

Proceduralno programiranje kroz programski jezik Pajton

prof. dr Marko Đukanović, vanredni profesor

Avgust 2026

Sadržaj

1	Uvod u osnove teorije algoritama	3
1.1	Računarski problem	4
1.2	Računarski program i pojam algoritma	6
1.2.1	Vrste algoritama	9
1.3	Teorijski RAM model	10
1.3.1	Instrukcije u RAM modelu	11
1.3.2	Formalizacija programa na RAM-u	13
1.4	Pojam složenosti algoritma	15
1.4.1	O -notacija	17
1.4.2	Neka svojstva O -notacije	21
1.4.3	Uputstva za procjenu složenosti algoritama	21
1.5	Prostorna složenost	25
1.6	Pseudokôd	27
1.7	O valjanosti algoritma	30
1.8	Euklidov algoritam	32
1.9	Master teorema	34
2	Uvod u programski jezik Pajton	41
2.1	Osnovne karakteristike	44
2.2	Pregled ugrađenih tipova podataka	45
2.3	Deklarisanje varijabli i memorijska organizacija	46
2.4	Ispis poruka i komentari	50
2.5	Osnovni numerički tipovi podataka	51

2.5.1	Cjelobrojni i realni brojevi	51
2.5.2	Kompleksni brojevi	52
2.6	Naredbe grananja i petlje	53
2.6.1	Uslovna naredba <code>if</code>	53
2.6.2	<code>For</code> i <code>While</code> petlje	54
2.7	Složene strukture podataka	56
2.7.1	Stringovni tip podataka	56
2.7.2	Liste	60
2.7.3	Nizovi: modul <code>array</code>	65
2.7.4	Rječnici	67
2.7.5	Skupovi	71
2.7.6	Torke	74
2.8	Funkcije	77
2.8.1	Funkcije kao argumenti	80
2.8.2	Anonimne funkcije.	81
2.9	Neki programski koncepti i često korištene ugrađene funkcije	82
2.9.1	Komprehenzija liste	82
2.9.2	Funkcije <code>map</code> , <code>filter</code> i <code>reduce</code>	83
2.9.3	Funkcije <code>zip</code> , <code>enumerate</code> i <code>sort</code>	85
2.9.4	Sortiranje lista	86
2.9.5	Korisne ugrađene funkcije	88
2.10	Moduli	88
2.10.1	Modul <code>math</code>	90
2.10.2	Modul <code>random</code>	92
2.10.3	Modul <code>time</code>	93
2.10.4	Modul <code>os</code>	93
2.10.5	Modul <code>copy</code>	95
2.11	Izuzeci	96
2.11.1	Blok <code>try-except</code>	97
2.11.2	Naredba <code>assert</code>	98
2.11.3	Hijerarhija izuzetaka	99
2.12	Rad sa datotekama	99
2.12.1	Modul <code>json</code>	101
2.13	Regularni izrazi	102

2.13.1	Osnovni koncepti regularnih izraza	102
2.13.2	Modul <code>re</code>	104
2.14	Moduli <code>sys</code> i <code>getopt</code>	106
2.14.1	Modul <code>sys</code>	107
2.14.2	Modul <code>getopt</code>	108
2.15	Modul <code>numpy</code>	110
2.16	Modul <code>ctypes</code>	115
2.17	Pisanje korisničkih modula	117
2.18	Postavljanje korisnički kreiranih modula na PyPI repozitorij	118
2.19	Kompajliranje i izvršavanje programa	120
2.19.1	Stek i hip memorija	121
2.19.2	Sakupljanje otpadaka	122
2.19.3	Proces prevođenja i izvršavanja programa	122
2.19.4	Pajtonova virtuelna mašina	124
3	Osnovne algoritamske paradigme	129
3.1	Rekurzivni algoritmi	132
3.1.1	Jednostavan primjer rekurzije	132
3.1.2	Rekurzivno računanje faktoriijela	133
3.1.3	Prekoračenje steka	134
3.2	Vrste rekurzije	135
3.2.1	Rekurzivni i iterativni pristup: poređenje	136
3.2.2	Primjene rekurzije	137
3.3	Algoritmi grube sile	147
3.4	Algoritmi pretraživanja	151
3.4.1	Linearna pretraga	152
3.4.2	Binarna pretraga	154
3.4.3	Pretraga preskakanjem	155
3.4.4	Interpolacijska pretraga	157
3.4.5	Eksponencijalna pretraga	159
3.5	Podijeli pa zavlada	161
3.6	Algoritamska paradigma vraćanja unazad	165
3.7	Pohlepni algoritmi	171
3.8	Dinamičko programiranje	184

3.8.1	Memoizacija (pristup odozgo prema dolje)	185
3.8.2	Rekurzija naspram dinamičkog programiranja	187
3.8.3	Primjene dinamičkog programiranja	188
4	Primjena algoritamskih paradigmi na probleme iz aritmetike	207
4.1	Eratostenovo sito	208
4.2	Faktorizacija broja	212
4.3	Izračunavanje binomnih koeficijenata	214
4.4	Hornerov metod	222

Predgovor

Ovaj udžbenik namijenjen je prvenstveno studentima prve godine Studijskog programa Matematika i informatika (smjer Informatika) na Prirodno-matematičkom fakultetu Univerziteta u Banjoj Luci, u okviru predmeta Proceduralno programiranje. Za savladavanje ovog gradiva, potrebno je da student vlada osnovama programiranja u barem jednom programskom jeziku.

Sadržaj udžbenika organizovan je u četiri cjeline. U prvom dijelu predstavljene su osnovni pojmovi iz oblasti algoritama i računanja, uključujući programe, algoritme, modele izračunavanja i osnovne koncepte složenosti algoritama. Pri tome se naglasak stavlja na intuitivno razumijevanje navedenih pojmova, bez ulaženja u sitne detalje koji se sistematski obrađuju na naprednijim kursovima iz algoritmike. Drugi dio posvećen je osnovama programskog jezika Pajton. Obrađuju se osnovni i složeni tipovi podataka, rad sa ugrađenim modulima, kao i razvoj sopstvenih modula te zvanično postavljanje na repozitorijumu PyPI. Treći dio uvodi najvažnije algoritamske paradigme, uključujući iscrpnu pretragu, paradigmu podijeli pa vladaj, pohlepne algoritme i dinamičko programiranje. Posebna pažnja posvećena je njihovoj primjeni na rješavanje različitih klasa problema i razvoju algoritamskog načina razmišljanja.

Četvrti dio bavi se problemima iz elementarne aritmetike i teorije brojeva. Prikazani problemi rješavaju se primjenom prethodno predstavljenih algoritamskih paradigmi i tehnika, pri čemu se poseban naglasak stavlja na proceduralni pristup modelovanja i implementacije rješenja.

Na kraju svake glave nalaze se zadaci za samostalni rad i provjeru usvojenog znanja. Udžbenik sadrži više od četrdeset zadataka za vježbanje, kao i oko trideset detaljno riješenih primjera. Za većinu njih data je kompletna implementacija u programskom jeziku Pajton, uz objašnjenje ključnih koraka u procesu dizajniranja i razvoja algoritama.

Ovaj udžbenik može koristiti širokom krugu čitalaca, uključujući studente nižih godina studija iz oblasti informatike, matematike i tehničkih nauka, kao i učenike srednjih škola zainteresovanih za algoritmiku i takmičarsko programiranje.

Iako su svi algoritmi implementirani u programskom jeziku Pajton, njihovi koncepti i struktura mogu se jednostavno prebaciti na druge programske jezike, naročito na jezike nižeg nivoa kao što su C i C++. Izbor jezika Pajton nije motivisan težnjom ka maksimalnim performansama izvršavanja, već njegovom preglednošću, jednostavnošću i izražajnošću. Na taj način studentima se omogućava da se usredsrede na razumijevanje algoritama i principa njihovog dizajniranja, umjesto na tehničke detalje implementacije.

Cilj ovog udžbenika nije samo upoznavanje čitalaca sa osnovama programiranja i algoritmike, već i razvijanje sposobnosti analitičkog razmišljanja, modelovanja problema i izbora odgovarajućih algoritamskih tehnika za efikasno rješavanje razmatranih problema.

Nadam se da će ova knjiga biti od koristi učenicima, studentima i mnogim entuzijastima koji imaju strast prema algoritmima te rješavanju netrivijalnih, relanih problema pomoću kompjuterskog kôda.

Glava 1

Uvod u osnove teorije algoritama

U ovom poglavlju uvode se osnovni pojmovi teorije algoritama i računanja. Najprije se definiše pojam računarskog problema, zatim pojam algoritma kroz apstraktni model računanja, a potom osnovni koncepti vremenske i prostorne složenosti algoritama. Na kraju se razmatra pitanje ispravnosti algoritama, sa posebnim osvrtom na metodu invarijantne petlje.

Nakon uspješnog savladavanja sadržaja ove glave student će biti u stanju da:

- definiše računarski problem, instancu problema, rješenje, računarski program i algoritam;
- razlikuje osnovne vrste računarskih problema i algoritamskih pristupa;
- objasni osnovne pretpostavke teorijskog RAM modela;
- odredi veličinu ulaza za jednostavne računarske probleme;
- analizira broj elementarnih operacija koje algoritam izvršava te odredi vremensku i prostornu složenost jednostavnih algoritama;

- razumije asimptotske notacije O , Θ , Ω i o pri analizi složenosti;
- predstavi algoritam korištenjem pseudokoda;
- objasni pojam ispravnosti algoritma i primijeni metod invarijante petlje na jednostavne algoritme;

1.1 Računarski problem

Računarski problem (eng. *computational problem*) predstavlja zadatak koji je potrebno riješiti uz pomoć računara.

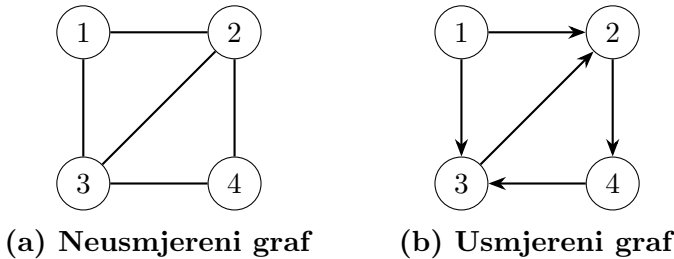
Formalna definicija ovog pojma zahtijeva uvođenje više netrivialnih teorijskih koncepata, među kojima je i pojam jezika. U opštem slučaju, ulaz računarskog problema može se predstaviti kao binarno kodiran niz, odnosno element skupa $L = \{0, 1\}^*$. Izlaz predstavlja rješenje (opet element skupa L) koje mora zadovoljiti određena svojstva definisana samim problemom. Drugim riječima, računarski problem određuje skup uslova koje izlaz mora zadovoljiti za svaki dopustiv ulaz.

Ulaz problema često se naziva *instancom problema*, dok se odgovarajući izlaz naziva *rješenjem instance*. Jedan od najpoznatijih primjera računarskog problema jeste problem sortiranja. U ulazu je dat niz objekata dužine n , dok je cilj proizvesti uređenu verziju tog niza prema unaprijed definisanom kriterijumu uređenja. Pri tome objekti ne moraju biti brojevi; dovoljno je da je nad njima definisana relacija totalnog uređenja.

Na računarski problem možemo gledati kao na preslikavanje između ulaznih i izlaznih objekata. Ulazi i izlazi mogu biti predstavljeni različitim matematičkim strukturama, kao što su brojevi, skupovi, nizovi ili grafovi. U tom kontekstu govori se o generičkoj instanci problema i odgovarajućem generičkom rješenju.

Prije razmatranja različitih tipova računarskih problema uvedimo pojam grafa, jednog od najvažnijih matematičkih objekata u računarstvu. Graf se može posmatrati kao skup čvorova povezanih granama (linijama). U ovoj knjizi podrazumijevamo proste grafove, odnosno grafove koji ne

sadrže petlje niti višestruke grane između istog para čvorova. Graf je *usmjeren* ukoliko svaka grana ima pridružen smjer, odnosno uređeni par krajnjih čvorova. U suprotnom, graf se naziva *neusmjerenim*. Primjeri usmjerenog i neusmjerenog grafa prikazani su na Slici 1.1.



Slika 1.1: Primjeri neusmjerenog i usmjerenog grafa.

Postoji više vrsta računarskih problema. U nastavku izdvajamo četiri najčešće klase koje se pojavljuju u literaturi: probleme odlučivanja, probleme pretraživanja, probleme optimizacije i probleme prebrojavanja.

Problem *odlučivanja* zahtijeva odgovor tipa DA/NE (*True/False*) za dati ulaz $x \in \{0, 1\}^*$. Drugim riječima, potrebno je utvrditi da li ulazna instanca posjeduje određeno svojstvo. Kao primjer može poslužiti problem *3-bojenja* grafa (3-COL). U ulazu je dat neusmjereni graf, a potrebno je odrediti da li postoji dodjela boja iz skupa 1, 2, 3 svim čvorovima tako da nijedna dva susjedna čvora nemaju istu boju.

Prikladan način formalizacije problema odlučivanja jeste određivanje skupa svih ulaza za koje je odgovor potvrđan. Takav skup označavamo sa $L \subseteq \{0, 1\}^*$ i nazivamo ga jezikom. Na taj način svaki problem odlučivanja može biti predstavljen odgovarajućim jezikom, a svaki jezik određuje jedan problem odlučivanja. Na primjer, skup svih 3-obojivih grafova predstavlja jezik koji opisuje problem 3-bojenja.

Kod problema *pretraživanja*, za dati ulaz $x \in \{0, 1\}^*$ potrebno je pronaći objekat $y \in \{0, 1\}^*$ koji zadovoljava određeni odnos sa ulazom. Takvi problemi formalizuju se relacijom $R \subseteq \{0, 1\}^* \times \{0, 1\}^*$, pri čemu $(x, y) \in R$ označava da je y dopustivo rješenje za instancu x .

Posmatrajmo ponovo problem 3-bojenja. Neka je dat neusmjereni graf $G = (V, E)$. Potrebno je pronaći, ukoliko postoji, funkciju bojenja $c: V \rightarrow \{1, 2, 3\}$, takvu da za svaku granu $(u, v) \in E$ važi $c(u) \neq c(v)$. Za razliku od verzije odlučivanja, ovdje nije dovoljno utvrditi postojanje rješenja; potrebno je i eksplicitno konstruisati jedno takvo rješenje.

Problemi *optimizacije* predstavljaju jednu od najvažnijih klasa računarskih problema. Glavni cilj je pronalaženje najboljeg rješenja iz skupa svih dopustivih rješenja prema unaprijed definisanoj funkciji cilja koja daje apsolutnu mjeru “kvaliteta” svakog rješenja. U zavisnosti od problema, ciljna funkcija se maksimizuje ili minimizuje uz poštovanje zadanih ograničenja.

Klasičan primjer predstavlja problem trgovačkog putnika. Potrebno je pronaći najkraću kružnu rutu (konturu) koja prolazi kroz sve zadate gradove (čvorovi grafa) tačno jednom i vraća se u početni grad. Cilj je minimizovati ukupnu pređenu udaljenost.

Problemi *prebrojavanja* bave se određivanjem broja mogućih rješenja određenog problema. Njihova složenost može varirati od veoma jednostavnih primjera do izuzetno zahtjevnih kombinatornih zadataka. Pretpostavimo da raspolažemo sa četiri košulje i tri para pantalona različitih boja. Na koliko različitih načina se osoba može obući ukoliko bira jednu košulju i jedne pantalone? Primjenom principa množenja dobijamo da je ukupan broj mogućih kombinacija jednak $4 \cdot 3 = 12$. Za kraj ovog poglavlja dajemo formalnu definiciju računarskog problema.

Definicija 1.1 *Računarski problem je funkcija $f: \Sigma^* \rightarrow 2^{\Sigma^*}$. Za dati ulaz x , skup $f(x)$ predstavlja skup svih ispravnih odgovora za instancu x .*

1.2 Računarski program i pojam algoritma

Računarski program predstavlja konačan niz instrukcija koje računar može izvršavati. Instrukcije određuju redoslijed operacija potrebnih za rješavanje određenog problema ili izvršavanje konkretnog zadatka. Programi mogu biti namijenjeni obradi teksta, analizi podataka, pretraživanju informacija na internetu, komunikaciji sa korisnikom, upravljanju uređajima ili brojnim drugim primjenama.

Programi se izvršavaju na različitim platformama, uključujući personalne računare, mobilne uređaje, serverske sisteme i ugrađene računarske uređaje. Za njihovu implementaciju koriste se različiti programski jezici, među kojima su Pajton, Java, *C*, *C++* itd. Nakon što je program napisan, potrebno ga je prevesti u mašinski kôd razumljiv procesoru. U zavisnosti od programskog jezika i načina izvršavanja, taj proces može uključivati kompajliranje, interpretaciju ili kombinaciju oba pristupa.

Algoritam predstavlja precizno definisan konačan niz koraka kojim za dati ulaz proizvodi odgovarajući izlaz, odnosno rješava određeni problem. U informatici se algoritmi koriste kao formalni opisi postupaka koje računarski program izvršava kako bi ostvario željenu funkcionalnost.

Može se posmatrati kao recept za rješavanje problema: algoritam definiše koje operacije treba izvršiti, kojim redoslijedom i pod kojim uslovima. Jednostavna analogija iz svakodnevnog života jeste priprema jela prema receptu. Recept opisuje niz koraka koje je potrebno izvršiti kako bi se od početnih sastojaka dobio željeni rezultat. Na sličan način algoritam transformiše ulazne podatke u odgovarajući izlaz.

Primjeri dobro poznatih algoritama uključuju algoritme sortiranja, koji uređuju podatke prema zadatom kriterijumu, algoritme pretraživanja za pronalaženje informacija u velikim skupovima podataka, algoritme kompresije i dekompresije podataka, kao i algoritme koji se koriste u kriptografiji i zaštiti informacija.

Uloga algoritama u računarstvu je višestruka:

- Omogućavaju efikasno rješavanje složenih problema.
- Predstavljaju osnovu automatizacije procesa, čineći ih bržim, pouzdanijim i jednostavnijim za izvršavanje.
- Omogućavaju računarima izvršavanje zadataka koje bi čovjek teško ili praktično nemoguće mogao obaviti ručno u realnom vremenu.

Međutim, svaki niz instrukcija ne predstavlja nužno algoritam. Da bi određeni postupak bio algoritam, potrebno je da zadovoljava nekoliko osnovnih osobina:

- *Dobro definisani ulazi.* Ukoliko algoritam koristi ulazne podatke, oni moraju biti jasno definisani i unaprijed određeni.
- *Dobro definisani izlazi.* Za svaki dopustiv ulaz algoritam mora proizvesti jasno određen rezultat.
- *Konačnost.* Algoritam mora završiti nakon konačnog broja koraka za svaki dopustiv ulaz.
- *Izvodljivost.* Svaka operacija algoritma mora biti izvršiva unutar raspoloživih računarskih resursa.
- *Nezavisnost od programskog jezika.* Algoritam predstavlja apstraktan postupak koji se može implementirati u različitim programskim jezicima.
- *Nedvosmislenost.* Svaka instrukcija mora biti precizno definisana i jednoznačno interpretirana.

Navedene osobine mogu se sažeti kroz nekoliko ključnih karakteristika algoritama:

- Algoritam se završava u konačnom vremenu.
- Za isti ulaz daje isti izlaz/rezultat, pod pretpostavkom determinističkog izvršavanja.
- Svaki njegov korak predstavlja efektivnu i izvršivu operaciju.

Veličina ulaza predstavlja mjeru količine podataka koje algoritam obrađuje. Način određivanja dimenzije ulaza zavisi od prirode problema. Kod problema sortiranja veličina ulaza odgovara broju elemenata koji se sortiraju. Kod problema nad grafovima može se definisati brojem čvorova i grana, dok se kod numeričkih problema često izražava brojem bitova potrebnih za predstavljanje ulaznih vrijednosti.

1.2.1 Vrste algoritama

Tokom razvoja računarstva razvijen je veliki broj algoritamskih paradigmi za rješavanje različitih klasa problema. U nastavku navodimo neke od najvažnijih pristupa koji će biti detaljnije obrađeni u kasnijim poglavljima.

1. *Iscrpna pretraga* (eng. *brute force*) predstavlja jedan od najjednostavnijih pristupa rješavanju problema. Osnovna ideja sastoji se u razmatranju svih mogućih kandidata za rješenje i provjeri koji od njih zadovoljavaju uslove problema.
2. *Rekurzivni algoritmi* (eng. *recursive algorithms*) zasnivaju se na razlaganju problema na manje podprobleme istog tipa. Rješenje se dobija ponovnim pozivanjem iste procedure sve dok se ne dostigne trivijalan slučaj koji se može direktno riješiti.
3. *Vraćanje unazad* (eng. *backtracking*) predstavlja tehniku sistematskog pretraživanja prostora rješenja. Kandidati za rješenje grade se postepeno, pri čemu se djelimična rješenja koja ne mogu dovesti do validnog konačnog rješenja odbacuju čim se to utvrdi.
4. *Algoritmi pretraživanja* (eng. *search algorithms*) koriste se za pronalaženje elemenata ili informacija unutar određene strukture podataka.
5. *Algoritmi sortiranja* (eng. *sorting algorithms*) uređuju podatke prema unaprijed definisanom kriterijumu uređenja.
6. *Podijeli pa vladaj* (eng. *divide-and-conquer*) razlaže problem na manje podprobleme, nezavisno ih rješava i zatim kombinuje njihova rješenja u konačan rezultat.
7. *Pohlepni algoritmi* (eng. *greedy algorithms*) grade rješenje korak po korak birajući u svakom trenutku lokalno najpovoljniju odluku prema unaprijed definisanom kriterijumu.
8. *Dinamičko programiranje* (eng. *dynamic programming*) koristi već izračunata rješenja podproblema kako bi se izbjegla višestruka iz-

računavanja istih vrijednosti. Posebno je efikasno kod problema sa preklapajućim podproblemima i optimalnom podstrukturom.

Većina navedenih pristupa predstavlja opšte algoritamske paradigme koje se mogu primijeniti na širok spektar problema. U trećoj glavi knjige detaljno će biti obrađene najvažnije paradigme, zajedno sa njihovim teorijskim osnovama i praktičnim primjenama.

Projektovanje efikasnih algoritama često predstavlja složen i kreativan proces koji zahtijeva dobro razumijevanje problema, poznavanje postojećih tehnika i sposobnost prepoznavanja odgovarajućih obrazaca rješavanja. Upravo zbog toga algoritamske paradigme imaju značajnu ulogu, jer pružaju opšte strategije koje se mogu prilagoditi velikom broju različitih problema.

1.3 Teorijski RAM model

Prilikom projektovanja i analize algoritama prirodno se nameće nekoliko važnih pitanja:

1. Da li je algoritam dovoljno efikasan?
2. Kako uporediti efikasnost dva različita algoritma za isti problem?
3. Kako se vrijeme izvršavanja algoritma ponaša kada veličina ulaza postane veoma velika?

Odgovor na ova pitanja zahtijeva precizan i nepristrasan matematički okvir za opisivanje procesa računanja. Zbog toga se u teorijskoj analizi algoritama uvode apstraktni modeli računara koji omogućavaju formalno mjerenje vremena i prostora potrebnih za izvršavanje algoritama.

Jedan od najčešće korištenih modela jeste *mašina sa slučajnim pristupom* (eng. *Random Access Machine* – RAM). Ovaj model predstavlja idealizovanu apstrakciju računara koja omogućava formalnu analizu algoritama nezavisno od konkretne hardverske arhitekture.

RAM model pretpostavlja postojanje glavne memorije organizovane kao niz adresabilnih memorijskih ćelija. Svakoj ćeliji može se pristupiti direktno,

pri čemu se njen sadržaj može čitati ili mijenjati u konstantnom vremenu. Takođe se pretpostavlja da se instrukcije izvršavaju sekvencijalno, odnosno da ne postoji paralelizam.

Glavna prednost RAM modela ogleda se u tome što omogućava precizno brojanje elementarnih operacija koje algoritam izvršava. Na taj način moguće je formalno definisati vremensku i prostornu složenost algoritama. Naravno, RAM model ne predstavlja potpuno vjeran opis stvarnih računarskih sistema. U realnim računarima vrijeme pristupa memoriji zavisi od različitih faktora, kao što su keš memorija, hijerarhija memorijskih nivoa, brzina pristupa sekundarnoj memoriji i drugi hardverski mehanizmi. Uprkos tome, RAM model pruža dovoljno preciznu aproksimaciju za većinu teorijskih analiza i zbog toga predstavlja standardni model u teoriji algoritama.

1.3.1 Instrukcije u RAM modelu

Krenimo sa formalizacijom teorijskog modela koji se standardno koristi za analizu algoritama. Posmatrano na visokom nivou apstrakcije, osnovne karakteristike RAM modela mogu se sažeti na sljedeći način:

- Glavna memorija predstavljena je nizom $M = (M[0], \dots, M[S-1])$ od w -bitnih riječi. Svaka riječ može se interpretirati kao broj iz intervala $\{0, \dots, 2^w - 1\}$.
- Registri su predstavljeni nizom $R = (R[0], \dots, R[r-1])$, gdje je r broj registara. Registri služe za privremeno skladištenje podataka i memorijskih adresa tokom izvršavanja programa.
- Algoritam može neposredno pristupati samo registrima. Pristup glavnoj memoriji ostvaruje se indirektnim adresiranjem, pri čemu se sadržaj registra tumači kao adresa memorijske lokacije. Zbog toga zahtijevamo da važi $S \leq 2^w$.
- Program predstavlja konačan niz instrukcija.

- Skup instrukcija uključuje osnovne aritmetičke, logičke i bitovske operacije, operacije dodjele, uslovne skokove, pristup memoriji i instrukciju zaustavljanja izvršavanja.
- Model dopušta dinamičku alokaciju memorije putem operacije **MALLOC**.
- Početna konfiguracija memorije definisana je na sljedeći način:
 1. Ulaz veličine n smješta se u memorijske lokacije $M[0], \dots, M[n-1]$.
 2. Veličina riječi određuje se kao $w = \lfloor \log_2(\max\{n+1, k+1\}) \rfloor$, gdje je k najveća vrijednost koja se može pojaviti u ulazu ili kao konstanta programa.
 3. Registar $R[0]$ inicijalizuje se vrijednošću w , dok se registar $R[1]$ inicijalizuje vrijednošću n .

Formalno, RAM model podržava sljedeće instrukcije za proizvoljne indekse $i, j, k, m \in \mathbb{N}$:

- $R[i] \leftarrow m$
- $R[i] \leftarrow R[j] \pm R[k]$
- $R[i] \leftarrow R[j] \cdot R[k]$
- $R[i] \leftarrow R[j] / R[k]$
- $R[i] \leftarrow R[j] \wedge R[k]$
- $R[i] \leftarrow \neg R[j]$
- $R[i] \leftarrow M[R[j]]$
- $M[R[j]] \leftarrow R[i]$
- IF $R[i] = 0$, goto l
- **MALLOC**

- HALT
- $\vdots$

Definicija 1.2 *w-RAM program predstavlja bilo koji konačan niz instrukcija $P = (P_1, P_2, \dots, P_q)$ prethodno navedenog tipa. Broj registara r implicitno je određen najvećim indeksom registra koji se pojavljuje u programu.*

1.3.2 Formalizacija programa na RAM-u

Da bismo formalno opisali izvršavanje programa, uvodimo pojam konfiguracije.

Konfiguracija programa predstavlja potpun opis stanja računanja u određenom trenutku izvršavanja. Drugim riječima, konfiguracija sadrži sve informacije neophodne za jednoznačno određivanje narednog koraka izvršavanja programa.

Definicija 1.3 *Konfiguracija w-RAM programa P je torka $K = (l, S, w, R, M)$, gdje je:*

- $l \in \{1, \dots, q\}$ programski brojač,
- $S \in \mathbb{N}$ količina korištene memorije,
- $w \in \mathbb{N}$ veličina riječi,
- $R = (R[0], \dots, R[r-1])$ niz registara,
- $M = (M[0], \dots, M[S-1])$ sadržaj glavne memorije.

Prelazak iz jedne konfiguracije u narednu određuje se pravilima izvršavanja instrukcija programa.

Definicija 1.4 *Za konfiguraciju $K = (l, S, w, R, M)$ programa $P = (P_1, \dots, P_q)$ definišemo narednu konfiguraciju $K' = (l', S', w', R', M')$ u oznaci $K \Rightarrow_P K'$. Transformacija zavisi od instrukcije P_l koja se trenutno izvršava. Na primjer:*

- *Uslovni skok mijenja vrijednost programskog brojača u skladu sa zadatim uslovom.*
- *Operacije dodjele i aritmetičke operacije mijenjaju sadržaj odgovarajućih registara.*
- *Operacije čitanja i upisa u memoriju mijenjaju sadržaj memorijskih ćelija.*
- *Operacija MALLOC proširuje raspoloživi memorijski prostor.*
- *Instrukcija HALT zaustavlja izvršavanje programa.*

Napomenimo da su ovdje prikazana samo najvažnija pravila tranzicije između konfiguracija. Preostale instrukcije definišu se na potpuno analogan način.

Sada možemo formalno definisati pojam rješavanja računarskog problema.

Definicija 1.5 *RAM program $P = (P_1, \dots, P_q)$ rješava računarski problem $f : \mathbb{N}^* \rightarrow 2^{\mathbb{N}^*}$ ako za svaki dopustiv ulaz postoji konačan niz konfiguracija $K_0, K_1, \dots, K_t$ takav da:*

- *K_0 predstavlja početnu konfiguraciju programa,*
- *za svako $i = 1, \dots, t$ važi $K_{i-1} \Rightarrow_P K_i$,*
- *posljednja konfiguracija odgovara zaustavljenom programu i sadrži ispravno rješenje problema.*

Komentar. Ključne karakteristike teorijskog RAM modela su sljedeće:

- *Uniformnost.* Zahtijeva se postojanje jednog konačnog programa koji može obrađivati ulaze proizvoljne veličine.
- *Elementarne operacije.* Računanje se modeluje kao niz osnovnih operacija nad registrima i memorijom.

- *Neograničenost resursa.* Teorijski model ne postavlja unaprijed ograničenja na vrijeme izvršavanja niti količinu memorije koju algoritam može koristiti.

U okviru teorijskog modela algoritam se smatra ispravnim bez obzira na količinu vremena i memorije koju koristi. Međutim, u praktičnim primjenama efikasnost algoritma predstavlja jedan od ključnih kriterijuma njegove upotrebljivosti.

Pri analizi vremena izvršavanja programa posmatra se broj elementarnih operacija koje algoritam izvrši. Takve operacije nazivaju se *jediničnim instrukcijama* i obuhvataju aritmetičke, logičke i bitovske operacije, operacije dodjele, kao i pristup memoriji.

Osnovna pretpostavka RAM modela jeste da se svaka jedinična instrukcija izvršava u konstantnom vremenu, predstavljajući tzv. *jedinični model kompleksnosti*. Zbog toga se vremenska složenost algoritma može izraziti ukupnim brojem izvršenih jediničnih instrukcija u toku izvršavanja. Ovaj koncept predstavlja osnovu za definisanje asimptotskih mjera složenosti koje će biti obrađene u narednoj sekciji.

1.4 Pojam složenosti algoritma

Podsjetimo da se u teorijskom RAM modelu svaka elementarna operacija izvršava u jediničnom vremenu. Ova pretpostavka predstavlja pojednostavljenje u odnosu na stvarne računarske sisteme, gdje različite operacije mogu zahtijevati različito vrijeme izvršavanja. Ipak, takav model se pokazao izuzetno korisnim jer omogućava formalnu analizu algoritama nezavisno od konkretne hardverske platforme.

Definišimo sada pojam vremena izvršavanja programa na RAM modelu.

Definicija 1.6 *Vrijeme izvršavanja programa P za ulaz $x \in \mathbb{N}^*$ predstavlja broj koraka koje program izvrši prije dostizanja zaustavne konfiguracije (HALT).*

Formalno, to je najmanji broj t za koji postoji niz konfiguracija $K_0 \Rightarrow_P K_1 \Rightarrow_P \dots \Rightarrow_P K_t$, pri čemu je K_0 početna konfiguracija programa za ulaz

x , dok je K_t zaustavna konfiguracija.

Razmotrimo sljedeći primjer.

```

algoritam max(a, n)
    maximum = prvi element niza a
    for svi elementi x niza a:
        if x > maximum then
            maximum = x
    vrati maximum

```

Pretpostavimo da je ulaz $a = [1, 5, 2, 10, 7]$, $n = 5$. U teorijskom RAM modelu svaka dodjela, poređenje i pristup elementu niza posmatraju se kao elementarne operacije. Brojanjem svih izvršenih instrukcija može se odrediti tačno vrijeme izvršavanja algoritma za konkretnu instancu ulaza.

Međutim, određivanje tačnog broja instrukcija za svaki mogući ulaz često je nepraktično, naročito kod složenijih algoritama. Zbog toga se uvodi pojam *najgoreg vremena izvršavanja* (eng. *worst-case running time*), koji predstavlja standardnu mjeru efikasnosti algoritama.

Definicija 1.7 *Najgore vrijeme izvršavanja programa predstavlja funkciju $T : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$, gdje je $T(n, k)$ maksimalno vrijeme izvršavanja programa nad svim ulazima dužine n čije se vrijednosti nalaze u skupu $\{0, \dots, k\}$.*

Primjer 1.1 *Posmatrajmo sljedeći algoritam:*

```

algorithm sort(a, n):
    for i od 0 do duzine niza -1:
        for j od početka (0) do i-1
            if i-ti element veći od j-tog elementa then
                swap(a[i], a[j])
    vrati a

```

Najgori slučaj nastupa kada se tijelo `if` naredbe izvršava pri svakoj iteraciji unutrašnje petlje. Na primjer, to se može dogoditi kada su elementi ulaznog niza raspoređeni u obrnutom poretku u odnosu na željeni.

Za svaku vrijednost i , unutrašnja petlja izvršava se tačno i puta, za svaki od prvih i elemenata niza a . Ukupan broj izvršavanja njenog tijela jednak je

$$\sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i \frac{n(n-1)}{2}.$$

Ako svaka iteracija zahtijeva konstantan broj elementarnih operacija, ukupan broj izvršenih instrukcija proporcionalan je vrijednosti $\frac{n(n-1)}{2}$, što pokazuje da vrijeme izvršavanja raste kvadratno sa veličinom ulaza.

Najgore vrijeme izvršavanja koristi se zato što predstavlja gornju granicu vremena potrebnog za obradu bilo koje instance određene veličine. Takva analiza ne zavisi od konkretne raspodjele ulaznih podataka i omogućava objektivno poređenje različitih algoritama.

Pored toga, najgori slučaj je često jednostavnije identifikovati i analizirati nego određivati tačan broj koraka za svaku pojedinačnu instancu. Iz tog razloga analiza najgoreg slučaja predstavlja standardni pristup u teoriji algoritama.

Ipak, čak i određivanje najgoreg vremena izvršavanja može biti zahtjevno za složenije algoritme. Zbog toga se uvodi pojam *asimptotske složenosti*, pri kojem se ne posmatra tačan broj instrukcija, već brzina rasta vremena izvršavanja u zavisnosti od veličine ulaza.

Kako se veličina ulaza povećava, broj operacija koje algoritam izvršava uglavnom raste. Stoga je od posebnog interesa proučavanje ponašanja algoritama za velike vrijednosti n , jer razlike između različitih algoritamskih pristupa tada postaju najizraženije.

U narednoj podsekciji uvodimo formalno pojam asimptotske složenosti i najvažniju notaciju koja se koristi u njenom opisivanju – O -notaciju.

1.4.1 O -notacija

U prethodnoj sekciji vidjeli smo da se vrijeme izvršavanja algoritma može izraziti brojem elementarnih instrukcija koje program izvrši shodno zadatom ulazu. Međutim, čak i za relativno jednostavne algoritme, tačno određivanje broja izvršenih instrukcija može biti nepraktično. Zbog toga se

u analizi algoritama koristi *asimptotska analiza*, koja proučava ponašanje algoritama za velike vrijednosti ulaza.

Jedan od najvažnijih alata asimptotske analize jeste O -notacija (eng. *Big-O notation*). Njena osnovna ideja sastoji se u tome da se zanemare detalji koji ne utiču značajno na ponašanje algoritma za velike vrijednosti ulaza, te da se pažnja usmjeri na dominantne članove funkcije vremena izvršavanja.

Posmatrajmo sljedeće funkcije:

- $T_1(n) = 2n^{3/2} + 3n + 10$
- $T_2(n) = 3n \log^3 n + \lceil \sqrt{n} \rceil + 15$

Na prvi pogled nije jednostavno uporediti ove dvije funkcije. Međutim, za velike vrijednosti n dominantni član funkcije T_1 jeste $n^{3/2}$, dok je dominantni član funkcije T_2 jednak $n \log^3 n$.

Kako funkcija $n \log^3 n$ raste sporije od funkcije $n^{3/2}$, zaključujemo da će za dovoljno velike vrijednosti ulaza algoritam čije je vrijeme izvršavanja dato funkcijom T_2 biti efikasniji.

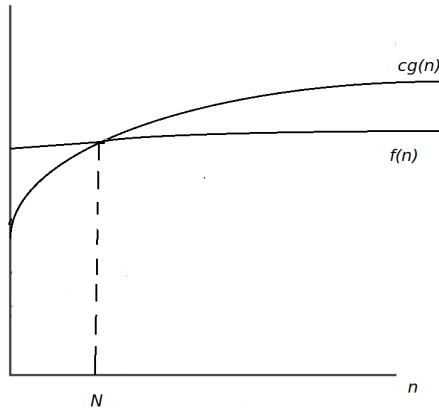
Da bismo formalizovali ovakva poređenja, uvodimo dvije osnovne konvencije:

1. Konstantni faktori se zanemaruju.
2. Posmatra se samo dominantni član funkcije, odnosno član koji najbrže raste kada n teži beskonačnosti.

Prema tome, $T_1(n) \sim n^{3/2}$, dok je $T_2(n) \sim n \log^3 n$.

U nastavku pretpostavljamo da posmatrane funkcije imaju nenegativne vrijednosti za dovoljno velike vrijednosti argumenta.

Definicija 1.8 *Neka su date funkcije f i g . Kažemo da je $f(n) = O(g(n))$ ako postoje konstante $c > 0$ i $N > 0$ takve da za svako $n \geq N$ važi $f(n) \leq c \cdot g(n)$. U tom slučaju kažemo da je g asimptotska gornja granica funkcije f .*



Slika 1.2: O -notacija: vizuelni prikaz asimptotske gornje granice.

Primjer 1.2 Vrijedi $T_1(n) = O(n^{3/2})$, ali i $T_1(n) = O(n^2)$. Takođe, $T_2(n) = O(n \log^3 n)$.

Dominantni član funkcije određuje njeno asimptotsko ponašanje. Kada je n dovoljno veliko, doprinos ostalih članova postaje zanemarljiv u odnosu na dominantni član. Zbog toga se složenost algoritma najčešće izražava upravo pomoću dominantnog člana funkcije vremena izvršavanja.

Iako je O -notacija izuzetno korisna, ona ne daje uvijek dovoljno preciznu informaciju. Na primjer, $n^2 + n = O(n^2)$, ali istovremeno važi i $n^2 + n = O(n^3)$.

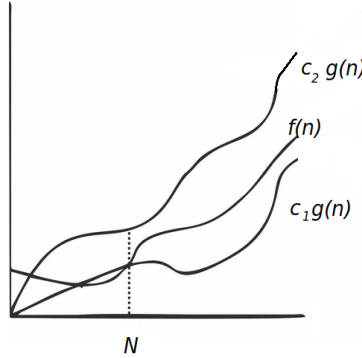
Oba iskaza su matematički tačna, ali prvi mnogo preciznije opisuje stvarno ponašanje funkcije. Zbog toga uvodimo pojam Θ -notacije.

Definicija 1.9 Neka su date funkcije f i g .

Kažemo da je $f(n) = \Theta(g(n))$ ako postoje konstante $c_1, c_2 > 0$ i $N > 0$ takve da za svako $n \geq N$ važi

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$

U tom slučaju kažemo da funkcije f i g imaju isto asimptotsko ponašanje.



Slika 1.3: Θ -notacija: funkcija f je ograničena odozgo i odozdo funkcijom g .

Primjer 1.3 Vrijedi $T_1(n) = \Theta(n^{3/2})$, dok je $T_2(n) = \Theta(n \log^3 n)$.

Pored O i Θ notacije, često se koriste i Ω i o notacije.

Definicija 1.10 Neka su date funkcije f i g . Kažemo da je $f(n) = \Omega(g(n))$ ako postoje konstante $c > 0$ i $N > 0$ takve da za svako $n \geq N$ važi $f(n) \geq c \cdot g(n)$. Funkcija g tada predstavlja asimptotsku donju granicu funkcije f .

Primjer 1.4 Vrijedi $n^3 = \Omega(n^2 + n)$, kao i $n \log n = \Omega(n)$.

Definicija 1.11 Neka su date funkcije f i g .

Kažemo da je $f(n) = o(g(n))$ ako za svako $c > 0$ postoji broj $N > 0$ takav da za svako $n \geq N$ važi $0 \leq f(n) < c \cdot g(n)$. Drugim riječima, funkcija f raste strogo sporije od funkcije g .

Neformalno posmatrano, uslov $f(n) = o(g(n))$ znači da odnos $\frac{f(n)}{g(n)}$ teži nuli kada n teži beskonačnosti.

Primjer 1.5 *Vrijedi $n = o(n^2)$, kao i $n = o(n^3)$. Međutim, $n \neq o(n)$.*

Asimptotske notacije predstavljaju osnovni jezik analize algoritama. Njihova najveća prednost ogleda se u tome što omogućavaju poređenje algoritama nezavisno od konkretne implementacije, programskog jezika ili hardverske platforme. Na taj način fokus se prebacuje na fundamentalno pitanje: kako vrijeme izvršavanja raste sa povećanjem veličine ulaza.

1.4.2 Neka svojstva O -notacije

Sljedeća teorema navodi nekoliko korisnih pravila koja se često koriste pri računanju asimptotske složenosti algoritama.

Teorema 1.1 *Neka su f , g i h pozitivne funkcije definisane za dovoljno velike vrijednosti argumenta. Tada vrijede sljedeće tvrdnje:*

- $f = O(c \cdot f)$ za svako $c > 0$ (konstantni faktori su irelevantni);
- Ako je $f = O(g + h)$ i $h = O(g)$, tada je $f = O(g)$ (pravilo dominantnog člana);
- $O(f + g) = O(f) + O(g)$ (pravilo zbira);
- $O(f \cdot g) = O(f) \cdot O(g)$ (pravilo proizvoda);
- Ako je $f = O(g)$ i $g = O(h)$, tada je i $f = O(h)$ (pravilo tranzitivnosti).

Dokaz Sve tvrdnje direktno slijede iz definicije O -notacije.

1.4.3 Uputstva za procjenu složenosti algoritama

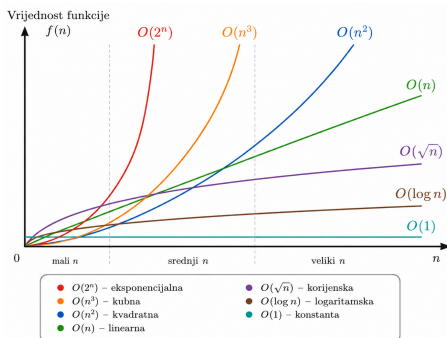
U prethodnoj sekciji uvedene su osnovne asimptotske notacije koje se koriste za procjenu složenosti algoritama. One omogućavaju da se vrijeme izvršavanja algoritma posmatra kao funkcija veličine ulaza, pri čemu se zanemaruju detalji koji ne utiču značajno na ponašanje algoritma za velike vrijednosti ulaza.

Najčešće funkcije koje se pojavljuju u analizi algoritama prikazane su u Tabeli 1.1.

Tabela 1.1: Najčešće klase asimptotske složenosti.

Funkcija	Klasa složenosti
1	konstantna
$\log n$	logaritamska
n	linearna
$n \log n$	log-linearna
n^2	kvadratna
n^3	kubna
2^n	eksponencijalna
$n!$	faktorijelska

Rast navedenih funkcija u zavisnosti od veličine ulaza prikazan je na Slici 1.4.



Slika 1.4: Poređenje najčešćih klasa složenosti.

U praksi se algoritmi eksponencijalne i faktorijelske složenosti smatraju neefikasnim za ulaze velikih dimenzija, jer vrijeme njihovog izvršavanja veoma brzo raste sa povećanjem veličine ulaza. Sa druge strane, logaritamska, linearna i log-linearna složenost najčešće se smatraju poželjnim, budući da

omogućavaju obradu veoma velikih instanci problema u razumnom vremenu.

Kvadratna složenost često predstavlja granicu praktične upotrebljivosti algoritma. Za neke probleme ona je sasvim prihvatljiva, dok za druge može biti nedovoljno efikasna, posebno kada se obrađuju veoma veliki skupovi podataka.

U nastavku navodimo nekoliko praktičnih pravila koja olakšavaju procjenu složenosti algoritama. Neka je sa $T(P)$ označena vremenska složenost programa ili programskog segmenta P .

Tabela 1.2: Pravila za procjenu složenosti algoritama.

Konstrukcija	Složenost
Jedinična instrukcija	1
$S : N_1; N_2$	$T(S) = T(N_1) + T(N_2)$
IF C THEN N_1 ELSE N_2	$T(S) = T(C) + \max\{T(N_1), T(N_2)\}$
WHILE U DO N_1	$T(S) = n \cdot (T(U) + T(N_1))$
FOR $i = j$ TO p DO N_1	$T(S) = n \cdot T(N_1)$

Pri procjeni složenosti algoritama najčešće se zanemaruju konstantni faktori i niži redovi rasta, a pažnja se usmjerava na dominantni član izraza. Na taj način moguće je brzo odrediti asimptotsko ponašanje algoritma bez potrebe za detaljnim brojanjem svih izvršenih instrukcija.

Postavlja se pitanje: kada možemo reći da je jedan algoritam efikasniji od drugog?

Odgovor daju upravo asimptotske notacije, prvenstveno O i Θ notacija. Kažemo da je algoritam X *asimptotski efikasniji* od algoritma Y ukoliko za dovoljno velike vrijednosti ulaza vrijeme izvršavanja algoritma X raste sporije od vremena izvršavanja algoritma Y .

Drugim riječima, algoritam X pripada nižoj klasi složenosti od algoritma Y . Na primjer, algoritam složenosti $O(n \log n)$ asimptotski je efikasniji od algoritma složenosti $O(n^2)$.

Važno je naglasiti da asimptotska efikasnost opisuje ponašanje algoritama za velike vrijednosti ulaza. Za male ulaze moguće je da algoritam sa lošijom asimptotskom složenošću bude brži zbog manjih konstantnih faktora ili jednostavnije implementacije. Ipak, kako veličina ulaza raste, prednost algoritma sa boljom asimptotskom složenošću postaje sve izraženija.

U narednom primjeru razmotrićemo jedan klasičan problem, prikazati dva različita algoritamska pristupa njegovom rješavanju i analizirati njihovu vremensku složenost.

Primjer 1.6 (*Problem podniza maksimalne sume*)

Dat je niz brojeva. Potrebno je odrediti neprekidni podniz početnog niza čija je suma elemenata najveća.

Rješenje. Neka je dat niz $\mathbf{a} = [1, 4, -2, 2, 10, -7, 1]$. Primjer jednog neprekidnog podniza jeste $[-2, 2, 10]$, dok niz $[-2, -7, 1]$ nije neprekidan podniz jer njegovi elementi nisu uzastopni u izvornom nizu. Najprije razmotrimo naivan pristup. Definišimo funkciju

$$S(i, j) = \sum_{k=i}^j a[k],$$

pri čemu je $S(i, i) = a[i]$. Dakle, funkcija $S(i, j)$ predstavlja sumu elemenata neprekidnog podniza čiji su početni i završni indeks redom i i j .

Za izračunavanje vrijednosti $S(i, j)$ potrebno je sabrati $j - i + 1$ elemenata, što zahtijeva $O(n)$ operacija u najgorem slučaju. Da bismo pronašli podniz maksimalne sume, potrebno je razmotriti sve moguće parove indeksa (i, j) za koje važi $0 \leq i \leq j < n$. Broj takvih parova jednak je $\binom{n}{2} + n = \frac{n(n+1)}{2}$, što pripada klasi $O(n^2)$. Kako se za svaki od tih parova funkcija $S(i, j)$ računa u vremenu $O(n)$, ukupna složenost algoritma iznosi $O(n) \cdot O(n^2) = O(n^3)$. Dakle, dobijamo algoritam koji ima kubnu složenost.

Pokušajmo sada konstruisati efikasniji pristup. Primijetimo da između vrijednosti $S(i, j)$ i $S(i, j + 1)$ postoji jednostavna rekurzivna veza $S(i, j + 1) = S(i, j) + a[j + 1]$.

Drugim riječima, ukoliko je suma podniza od pozicije i do pozicije j već poznata, suma narednog podniza može se dobiti dodavanjem samo jednog novog elementa. Definišimo $S(i) = \max\{S(i, i), S(i, i + 1), \dots, S(i, n - 1)\}$.

Vrijednost $S(i)$ može se izračunati prolaskom kroz niz počevši od pozicije i , pri čemu se tekuća suma ažurira u konstantnom vremenu. Zbog toga je složenost izračunavanja funkcije $S(i)$ jednaka $O(n)$.

Pošto se funkcija $S(i)$ poziva za svaku početnu poziciju $i \in 0, \dots, n - 1$, ukupna složenost algoritma iznosi $n \cdot O(n) = O(n^2)$. Time smo složenost smanjili sa $O(n^3)$ na $O(n^2)$, što predstavlja značajno poboljšanje u efikasnosti algoritma.

1.5 Prostorna složenost

Pored vremena izvršavanja, važan kriterijum za procjenu kvaliteta algoritma jeste količina memorije koju algoritam koristi tokom rada.

Pri tome treba razlikovati pojmove *pomoćni prostor* i *prostorna složenost*. Pomoćni prostor predstavlja dodatnu memoriju koju algoritam koristi tokom izvršavanja, ne računajući prostor potreban za smještanje ulaznih podataka. Sa druge strane, *prostorna složenost* algoritma predstavlja ukupnu količinu memorije potrebne za njegovo izvršavanje, uključujući i prostor za ulazne podatke i pomoćne memorijske strukture.

Kao i kod vremenske složenosti, prostorna složenost izražava se kao funkcija veličine ulaza. Na primjer:

- kreiranje niza dužine n zahtjeva $O(n)$ memorijskog prostora;
- kreiranje matrice dimenzija $n \times n$ zahtijeva $O(n^2)$ memorijskog prostora.

Efikasnost algoritma najčešće se procjenjuje na osnovu dva kriterijuma: vremena izvršavanja i količine korištene memorije. U mnogim situacijama postoji kompromis između ova dva resursa. Često je moguće ubrzati algoritam korišćenjem dodatne memorije, dok smanjenje memorijskih zahtijeva može dovesti do povećanja vremena izvršavanja.

Primjer 1.7 *Posmatrajmo sljedeće procedure.*

```

1: procedure PAIRSUM( $x, y$ )
2:   return  $x + y$ 
3: end procedure

```

```

1: procedure ADDSEQUENCE( $n$ )
2:    $sum \leftarrow 0$ 
3:   for  $i = 0$  to  $n - 1$  do
4:      $sum \leftarrow sum + \text{PAIRSUM}(i, i + 1)$ 
5:   end for
6:   return  $sum$ 
7: end procedure

```

Odredimo prostornu složenost procedure ADDSEQUENCE.

Iako se funkcija PAIRSUM poziva $O(n)$ puta, ne dolazi do akumuliranja dodatnih memorijskih struktura. Tokom izvršavanja koristi se samo konstantan broj promjenljivih (sum , i , te parametri funkcije), pa je prostorna složenost algoritma jednaka $O(1)$.

Drugim riječima, količina memorije ne zavisi od veličine ulaza n .

Napomena. U praktičnom programiranju raspoloživa memorija je ograničena. Na mnogim sistemima tipična memorijska ograničenja iznose između 128 MB i 512 MB po procesu. Zbog toga nije uvijek moguće kreirati veoma velike nizove ili matrice.

Takođe, lokalne promjenljive funkcija smještaju se na stek, čija je veličina obično znatno manja od ukupne raspoložive memorije procesa. Zbog toga se veoma velike strukture podataka često alociraju u globalnoj memoriji ili na hipu (eng. *heap*), umjesto unutar tijela funkcije.

1.6 Pseudokôd

Pseudokod predstavlja neformalni način zapisivanja algoritama pomoću kombinacije prirodnog jezika i programerskih konstrukcija. Njegova svrha nije da bude izvršiv na računaru, već da na jasan i nedvosmislen način predstavi logiku algoritma.

Za razliku od programskog kôda, pseudokôd ne posjeduje strogo definisanu sintaksu i ne može se kompajlirati niti interpretirati. Upravo zbog toga omogućava da se pažnja usmjeri na suštinu algoritma, bez opterećivanja detaljima konkretnog programskog jezika.

Pseudokôd predstavlja koristan međukorak između ideje rješenja i njegove implementacije. Njime se opisuje redoslijed koraka koje je potrebno izvršiti kako bi se riješio određeni problem, pri čemu zapis ostaje dovoljno prost da ga mogu razumjeti i osobe koje nisu upoznate sa svim detaljima implementacije.

Prednosti korišćenja pseudokôda su višestruke:

- Poboljšava čitljivost i razumijevanje algoritma.
- Omogućava fokusiranje na logiku rješenja umjesto na sintaksu programskog jezika.
- Predstavlja prirodnu vezu između idejnog rješenja, dijagrama toka i implementacije.
- Služi kao oblik dokumentacije koji olakšava kasnije održavanje i unapređivanje programa.

Osnovni cilj pseudokoda jeste da nedvosmisleno opiše svaki korak algoritma. Time se olakšava implementacija i smanjuje vjerovatnoća nastanka grešaka tokom razvoja programa.

Konstrukcija pseudokoda započinje detaljnim razumijevanjem problema koji se rješava. Nakon identifikacije potrebnih koraka, koraci se zapisuju u obliku jednostavnih i jasno formulisanih instrukcija.

Prilikom pisanja pseudokoda preporučuje se pridržavanje sljedećih smjernica:

- Koristiti jasne i opisne nazive promjenljivih i funkcija.
- Svaki korak algoritma opisati dovoljno precizno da ne postoji mogućnost različitog tumačenja.
- Posebnu pažnju posvetiti uslovima zaustavljanja algoritma.
- Jasno naglasiti strukturu grananja i petlji korišćenjem uvlačenja naredbi.
- Izbjegavati nepotrebne implementacione detalje i zadržati fokus na logici rješenja.

Primjer jednostavnog pseudokoda:

```
tip = unos sa tastature

ako je tip = "1"
  ispiši "Unesen je broj 1"

ako je tip = "2"
  ispiši "Unesen je broj 2"
```

Primijetimo da ovakav zapis ne predstavlja program u konkretnom programskom jeziku, već opisuje logiku koju bi program trebalo da realizuje.

Primjer 1.8 *Napisati pseudokod za pronalaženje najvećeg elementa niza.*

Postavi najveći element na prvi element niza.

Prolazi kroz elemente niza jedan po jedan.

Ako je trenutni element veći od najvećeg do sada pronađenog,

ažuriraj vrijednost najvećeg elementa.

Kraj prolaska kroz niz.

Prikaži najveći element.

U nastavku prikazujemo formalniji oblik pseudokôda, kakav se često koristi u literaturi iz algoritama. Posmatrajmo algoritam za rješavanje problema podniza maksimalne sume iz prethodne sekcije.

Algoritam 1 Funkcija $S_i(i)$

```

1: procedure  $S_i(i, niz)$ 
2:    $s_{\max} \leftarrow niz[i]$ 
3:    $s \leftarrow niz[i]$ 
4:   for  $j = i + 1$  to  $n - 1$  do
5:      $s \leftarrow s + niz[j]$ 
6:     if  $s > s_{\max}$  then
7:        $s_{\max} \leftarrow s$ 
8:     end if
9:   end for
10:  return  $s_{\max}$ 
11: end procedure

```

Algoritam 2 Funkcija S

```

1: procedure  $S(niz)$ 
2:    $s_{\max} \leftarrow niz[0]$ 
3:   for  $i = 0$  to  $n - 1$  do
4:      $s_i \leftarrow S_i(i, niz)$ 
5:     if  $s_i > s_{\max}$  then
6:        $s_{\max} \leftarrow s_i$ 
7:     end if
8:   end for
9:   return  $s_{\max}$ 
10: end procedure

```

Ovakav način zapisivanja pseudokoda predstavlja standard u većini udžbenika i naučnih radova iz oblasti algoritama. Iako je formalniji od prethodnih primjera, i dalje ostaje nezavisan od konkretnog programskog jezika, što omo-

gućava da se algoritam jednostavno implementira u Pajtonu, Javi, C-u, C++-u ili bilo kojem drugom jeziku.

1.7 O valjanosti algoritma

Pored analize vremenske i prostorne složenosti, važno je pokazati da algoritam zaista rješava problem za koji je konstruisan. Drugim riječima, potrebno je dokazati njegovu *valjanost* (ispravnost).

Postoji više pristupa dokazivanju ispravnosti algoritama, među kojima izdvajamo sljedeće:

- *Matematički dokaz.* Predstavlja najrigorozniji pristup dokazivanju ispravnosti algoritma. Cilj je pokazati da algoritam za svaki dopustiv ulaz vraća ispravno rješenje problema. Jedna od najčešće korištenih tehnika jeste metoda *invarijante petlje*, koju ćemo razmotriti u nastavku.
- *Formalna verifikacija.* Podrazumijeva modelovanje algoritma pomoću formalnog jezika i korištenje specijalizovanih alata za automatsku provjeru željenih svojstava. Ovakve metode imaju značajnu primjenu u razvoju kritičnih softverskih sistema, gdje čak i mala greška može imati ozbiljne posljedice.
- *Testiranje.* Iako ne predstavlja formalni dokaz ispravnosti, testiranje je nezaobilazan korak u razvoju softvera. Pokretanjem algoritma nad velikim brojem pažljivo odabranih testnih primjera može se steći visok stepen povjerenja u njegovu ispravnost.

Metoda *invarijante petlje* predstavlja jednu od najvažnijih tehnika za dokazivanje ispravnosti iterativnih algoritama.

Definicija 1.12 *Invarijanta petlje je tvrdjenje koje važi prije prve iteracije petlje, ostaje tačno nakon svake iteracije i važi u trenutku završetka petlje.*

Dokaz ispravnosti algoritma pomoću invarijante petlje obično se sastoji od tri koraka:

1. **Inicijalizacija.** Pokazuje se da invarijanta važi prije prve iteracije petlje.
2. **Održavanje.** Dokazuje se da, ukoliko invarijanta važi prije određene iteracije, ona ostaje tačna i nakon izvršavanja te iteracije.
3. **Završetak.** Nakon izlaska iz petlje koristi se invarijanta kako bi se pokazalo da rezultat koji algoritam vraća zaista predstavlja rješenje problema.

Prednost ove metode ogleda se u tome što omogućava formalno dokazivanje ispravnosti algoritama koji koriste petlje, bez potrebe da se zasebno analiziraju svi mogući ulazi.

Primjer 1.9 *Razmotrimo sljedeći algoritam za određivanje najvećeg zajedničkog djelioca (NZD) dva prirodna broja.*

Algoritam 3 Naivni algoritam za određivanje NZD-a.

```

1: procedure NZD( $n, m$ )
2:    $d \leftarrow \min(n, m)$ 
3:   while  $n \bmod d \neq 0$  or  $m \bmod d \neq 0$  do
4:      $d \leftarrow d - 1$ 
5:   end while
6:   return  $d$ 
7: end procedure

```

Neka je sa d^* označen najveći zajednički djelilac brojeva n i m . Posmatrajmo tvrđenje $d^* \leq d$. Pokazaćemo da ono predstavlja invarijantu petlje.

- **Inicijalizacija.** Prije prve iteracije važi $d = \min(n, m)$.

Pošto najveći zajednički djelilac ne može biti veći od manjeg od dva broja, slijedi da je $d^* \leq d$. Dakle, invarijanta važi prije ulaska u petlju.

- **Održavanje.** Pretpostavimo da invarijanta važi na početku neke iteracije. U tijelu petlje vrijednost promjenljive d smanjuje se za jedan.

Sve dok se petlja izvršava, trenutna vrijednost d nije zajednički djelilac oba broja. Budući da je d^* najveći zajednički djelilac, mora važiti $d^* < d$.

Nakon izvršavanja naredbe $d \leftarrow d - 1$ i dalje ostaje $d^* \leq d$, pa je invarijanta očuvana.

- **Završetak.** Petlja se zaustavlja kada trenutna vrijednost d dijeli i broj n i broj m . Dakle, tada je d zajednički djelilac. S druge strane, prema invarijanti petlje važi $d^* \leq d$.

Pošto algoritam ispituje kandidate silaznim redoslijedom, prvi pronađeni zajednički djelilac mora biti upravo najveći zajednički djelilac. Dakle, $d = d^*$.

Time je pokazano da algoritam zaista vraća najveći zajednički djelilac ulaznih brojeva.

U praksi je pronalaženje odgovarajuće invarijante često najzahtjevniji dio dokaza. Kod složenijih algoritama invarijanta može biti veoma komplikovana, pa formalni dokaz postaje težak za izvođenje i razumijevanje. Zbog toga se u razvoju softvera formalni dokazi često kombinuju sa opsežnim testiranjem i empirijskom provjerom ispravnosti.

1.8 Euklidov algoritam

Jedan od najstarijih i najpoznatijih algoritama u matematici jeste *Euklidov algoritam* za određivanje najvećeg zajedničkog djelioca (NZD) dva prirodna broja. Algoritam je dobio ime po starogrčkom matematičaru Euklidu, koji ga je opisao u djelu *Elementi* prije više od dvije hiljade godina.

Osnovna ideja algoritma zasniva se na sljedećem svojstvu:

$$\text{NZD}(a, b) = \text{NZD}(b, a \bmod b).$$

Drugim riječima, najveći zajednički djelilac brojeva a i b jednak je najvećem zajedničkom djeliocu brojeva b i ostatka pri dijeljenju broja a brojem b . Ovo svojstvo omogućava da se problem postepeno svodi na manje ulaze, sve dok jedan od tih ulaza ne dobije vrijednost nula.

Algoritam 4 Euklidov algoritam.

```
1: procedure NZD( $a, b$ )
2:   while  $b \neq 0$  do
3:      $r \leftarrow a \bmod b$ 
4:      $a \leftarrow b$ 
5:      $b \leftarrow r$ 
6:   end while
7:   return  $a$ 
8: end procedure
```

Primjer 1.10 *Odredimo najveći zajednički djelilac brojeva $a = 400$ i $b = 24$.*

Rješenje. Algoritam izvršava sljedeće transformacije: $(400, 24) \rightarrow (24, 16) \rightarrow (16, 8) \rightarrow (8, 0)$. Pošto je u posljednjem koraku drugi broj jednak nuli, algoritam se zaustavlja i vraća vrijednost prvog broja. Dakle, $\text{NZD}(400, 24) = 8$.

Za razliku od naivnog algoritma iz prethodne sekcije, koji u najgorem slučaju ispituje sve brojeve od $\min(a, b)$ do 1, Euklidov algoritam veoma brzo smanjuje vrijednosti brojeva nad kojima radi. Zbog toga se u praksi koristi gotovo isključivo Euklidov pristup.

Može se pokazati da broj iteracija Euklidovog algoritma raste proporcionalno logaritmu manjeg od ulaznih brojeva. Preciznije, broj iteracija pripada klasi $O(\log \min(a, b))$.

Pri analizi algoritama često se veličina ulaza ne mjeri samim brojevima a i b , već brojem bitova potrebnih za njihov zapis. Ako sa n označimo ukupan broj bitova potrebnih za predstavljanje ulaznih brojeva, tada približno važi $n \approx \log_2(a) + \log_2(b)$. Prema tome, broj iteracija algoritma raste linearno u odnosu na veličinu ulaza izraženu u bitovima, odnosno pripada klasi $O(n)$.

Pored male vremenske složenosti, Euklidov algoritam koristi svega nekoliko pomoćnih promjenljivih (a , b i r), pa je njegova prostorna složenost jednaka $O(1)$.

Zahvaljujući svojoj jednostavnosti i efikasnosti, Euklidov algoritam predstavlja jedan od najvažnijih algoritama u teoriji brojeva i računarstvu. Njegove modifikacije koriste se u brojnim oblastima, uključujući kriptografiju, računarsku algebru i teoriju algoritama.

Napomena. Valjanost Euklidovog algoritma direktno slijedi iz svojstva $\text{NZD}(a, b) = \text{NZD}(b, a \bmod b)$, koje se primjenjuje u svakoj iteraciji algoritma. Budući da se vrijednost drugog argumenta strogo smanjuje, algoritam se nakon konačnog broja koraka zaustavlja, a posljednja nenulta vrijednost predstavlja najveći zajednički djelilac ulaznih brojeva.

1.9 Master teorema

Kod analize algoritama veoma često se pojavljuju rekurzivne relacije koje opisuju vrijeme izvršavanja algoritma. Ovakve relacije su naročito karakteristične za algoritamske paradigme zasnovane na rekurzivnom principu, kao što je paradigma *podijeli pa zavlada*.

Pretpostavimo da algoritam problem veličine n dijeli na a podproblema, pri čemu svaki podproblem ima veličinu približno n/b . Pored vremena potrebnog za rješavanje podproblema, algoritam troši dodatno vrijeme za podjelu problema, kombinovanje parcijalnih rješenja ili druge operacije. Tada se vremenska složenost algoritma često može opisati rekurentnom relacijom oblika

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

gdje je:

- $a \geq 1$ broj podproblema koji se rekurzivno rješavaju;
- $b > 1$ faktor za koji se smanjuje veličina svakog podproblema;

- $f(n)$ vrijeme potrebno za operacije koje se izvršavaju izvan rekurzivnih poziva, kao što su podjela problema ili kombinovanje rješenja.

Određivanje asimptotske složenosti ovakvih rekurentnih relacija može biti zahtjevno. *Master teorema* daje jednostavan način za određivanje asimptotske složenosti velikog broja rekurzija ovog tipa.

Teorema 1.2 (Master teorema) *Neka je vremenska složenost algoritma opisana relacijom*

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

gdje su $a \geq 1$ i $b > 1$ konstante. Vrijednost

$$n^{\log_b a}$$

predstavlja osnovnu veličinu sa kojom se poredi funkcija $f(n)$.

Tada, u najčešćim slučajevima, vrijedi:

1. Ako funkcija $f(n)$ raste asimptotski sporije od $n^{\log_b a}$, tada je

$$T(n) = \Theta\left(n^{\log_b a}\right).$$

2. Ako funkcija $f(n)$ raste istom asimptotskom brzinom kao $n^{\log_b a}$, tada je

$$T(n) = \Theta\left(n^{\log_b a} \log n\right).$$

3. Ako funkcija $f(n)$ raste asimptotski brže od $n^{\log_b a}$, tada je, pod odgovarajućim dodatnim uslovima,

$$T(n) = \Theta(f(n)).$$

Drugim riječima, osnovna ideja Master teoreme jeste poređenje troška rekurzivnih poziva sa troškom operacija koje se izvršavaju izvan rekurzije.

Primjer 1.11 *Neka je data rekurzija*

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1).$$

Ovdje je $a = 1$, $b = 2$ i $f(n) = \Theta(1)$. Pošto je

$$n^{\log_2 1} = n^0 = 1,$$

funkcija $f(n)$ ima isti red rasta kao $n^{\log_b a}$. Primjenom drugog slučaja Master teoreme dobijamo

$$T(n) = \Theta(\log n).$$

Dakle, vremenska složenost binarne pretrage je logaritamska.

Primjer 1.12 *Neka je data rekurentna relacija*

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$

U ovom slučaju je $a = 2$, $b = 2$ i

$$n^{\log_2 2} = n.$$

Pošto je $f(n) = \Theta(n)$, primjenjuje se drugi slučaj Master teoreme, pa je

$$T(n) = \Theta(n \log n).$$

Primjer 1.13 *U slučaju rekurzicne relacije*

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

imamo $a = 3$ i $b = 2$, pa je

$$n^{\log_2 3} \approx n^{1.585}.$$

Kako funkcija n raste sporije od $n^{\log_2 3}$, primjenjuje se prvi slučaj Master teoreme. Prema tome,

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right).$$

Master teorema predstavlja veoma koristan alat za brzu procjenu vremenske složenosti algoritama zasnovanih na rekurzivnom razlaganju problema. U ovoj knjizi korist ćemo je prvenstveno za analizu jednostavnih algoritama zasnovanih na paradigmi podijeli pa zavladaaj.

Detaljan dokaz Master teoreme, njene preciznije formulacije, dodatni uslovi primjene i slučajevi koji nisu obuhvaćeni prethodno navedenim pravilima obrađuju se na naprednijim kursevima iz algoritama i analize algoritama na višim godinama studija.

Zadaci

1. Izračunati vremensku složenost sljedećeg programa:

```
1: procedure ZAD1( $N$ )
2:    $a \leftarrow 0$ 
3:   for  $i = 0$  to  $N$  do
4:     for  $j = N$  downto  $i$  do
5:        $a \leftarrow a + i + j$ 
6:     end for
7:   end for
8: end procedure
```

2. Izračunati vremensku složenost sljedećeg programa

```
1: procedure ZAD2( $n$ )
2:    $i, j, k \leftarrow 0$ 
3:   for  $i = n/2$  to  $n$  do
4:     for  $j = 2; j \leq n; j = j * 2$  do
5:        $k \leftarrow k + n/2$ 
6:     end for
7:   end for
8: end procedure
```

3. Izračunati vremensku složenost sljedećeg programa

```
1: procedure ZAD3( $N$ )
2:    $a \leftarrow 0, i \leftarrow N$ 
3:   while  $i > 0$  do
4:      $a \leftarrow a + i$ 
5:      $i \leftarrow i/2$ 
6:   end while
7: end procedure
```

4. Izračunati vremensku složenost sljedećeg programa

```
1: procedure ZAD4( $n$ )
2:    $k \leftarrow 3$ 
3:   for  $i = 0$  to  $n - 1$  do
4:      $i \leftarrow i * k$ 
5:   end for
6: end procedure
```

5. Izračunati vremensku složenost sljedećeg programa

```

1: procedure ZAD5( $n$ )
2:    $sum \leftarrow 0$ 
3:   for  $i = 1$  to  $n$  do
4:     for  $j = 1$  to  $i \cdot i$  do
5:       if  $j \bmod i == 0$  then
6:         for  $k = 1$  to  $j$  do
7:            $sum \leftarrow sum + 1$ 
8:         end for
9:       end if
10:    end for
11:  end for
12: end procedure

```

6. Izračunati (ukupnu) složenost sljedeće rekurzije:

```

1: procedure BUBBLESORT( $arr[]$ ,  $size$ )
2:   for  $i = 0; i < size - 1; ++i$  do
3:     for  $j = 0; j < size - 1; ++j$  do
4:       if  $arr[j] > arr[j + 1]$  then
5:          $swap(arr[j], arr[j + 1])$ 
6:       end if
7:     end for
8:   end for
9: end procedure

```

7. Izračunati složenost sljedeće rekurzije:

$$T(n) = \begin{cases} 3 \cdot T(n - 1), & \text{ako } n > 0, \\ 1, & \text{inače} \end{cases}$$

8. Izračunati složenost sljedeće rekurzije:

$$T(n) = \begin{cases} 2T(n-1) - 1, & \text{ako } n > 0, \\ 1, & \text{inače} \end{cases}$$

9. Izračunati prostornu složenost programa datog sljedećim pseudokodom

```
1: procedure SUM(arr[], N)
2:   ans ← 0
3:   for i = 0 to N do
4:     ans ← ans + arr[i]
5:   end for
6:   ispiši ans
7: end procedure
```

10. Izračunati prostornu složenost programa datog sljedećim pseudokodom

```
1: procedure FAKTORIJEL(N)
2:   fact ← 1
3:   for i = 1 to N do
4:     fact ← fact * i
5:   end for
6:   return fact
7: end procedure
```

Glava 2

Uvod u programski jezik Pajton

Pajton (*Python*) je interpretirani, objektno-orijentisani programski jezik visokog nivoa opšte namjene. Razvio ga je holandski programer Guido van Rossum, a prva javna verzija jezika objavljena je u februaru 1991. godine. Naziv jezika potiče od britanske humorističke grupe *Monty Python*, čiji je rad inspirisao autora tokom razvoja jezika.

Od svog nastanka Pajton je prošao kroz više razvojnih faza i danas predstavlja jedan od najrasprostranjenijih programskih jezika u akademskom i industrijskom okruženju. Njegova popularnost rezultat je prije svega jednostavne sintakse, velike čitljivosti koda i bogatog ekosistema biblioteka koje omogućavaju primjenu jezika u različitim oblastima. Pregled popularnosti programskih jezika prikazan je na Slici 2.1¹.

¹Preuzeto sa <https://www.tiobe.com/tiobe-index/>

Pos. Jan 2022	Pos. Jan 2021	Programming Language	Ratings	Chart Ratings	Variations
1	3	Python	13.58%		+1.86%
2	1	C	12.44%		-4.94%
3	2	Java	10.66%		-1.30%
4	4	C++	8.29%		+0.73%
5	5	C#	5.68%		+1.73%
6	6	Visual Basic	4.74%		+0.90%
7	7	JavaScript	2.09%		-0.11%
8	11	Assembly language	1.85%		+0.21%
9	12	SQL	1.80%		+0.19%
10	13	Swift	1.41%		-0.02%
11	8	PHP	1.40%		-0.60%
12	9	R	1.25%		-0.65%
13	14	Go	1.04%		-0.37%
14	19	Delphi/Object Pascal	0.99%		+0.20%
15	20	Classic Visual Basic	0.98%		+0.19%

Slika 2.1: Statistički pregled najpopularnijih programskih jezika (2021–2022)

Početak razvoja Pajtona vezuje se za 1989. godinu, kada je Guido van Rossum započeo rad na novom programskom jeziku inspirisanom jezikom ABC. Cilj je bio kreiranje jezika koji bi zadržao jednostavnost i preglednost ABC-a, ali istovremeno pružio veću fleksibilnost i mogućnosti za razvoj složenijih softverskih sistema.

Prva verzija jezika, Pajton 0.9.0, već tada sadrži brojne koncepte koji i danas predstavljaju osnovu jezika, uključujući funkcije, klase, izuzetke i module.

Pajton 1.0 objavljen je 1994. godine i donio je dodatne funkcionalnosti inspirisane funkcionalnim programiranjem, kao što su funkcije **map**, **filter**, **reduce** i lambda izrazi. Verzija 2.0, objavljena 2000. godine, uvela je značajna unapređenja poput komprehenzije lista (*list comprehensions*) i automatskog upravljanja memorijom putem sakupljača otpadaka (*garbage collector*).

Velika prekretnica u razvoju jezika dogodila se 2008. godine objavljivanjem verzije Pajton 3.0. Ova verzija donijela je niz izmjena koje nisu bile u

potpunosti kompatibilne sa prethodnim verzijama jezika, ali su omogućile jednostavniji, konzistentniji i pregledniji dizajn. Sve savremene verzije Pajtona zasnovane su na grani Pajton 3.

Danas se Pajton koristi u velikom broju oblasti, uključujući:

- razvoj veb aplikacija;
- naučno i numeričko računarstvo;
- obradu i analizu podataka;
- mašinsko učenje i vještačku inteligenciju;
- automatizaciju zadataka i sistemsko programiranje;
- razvoj obrazovnog softvera.

Veliki doprinos popularnosti jezika daje i njegova izuzetno aktivna zajednica korisnika i programera. Tokom godina razvijen je veliki broj biblioteka i razvojnih okruženja koja značajno proširuju mogućnosti jezika. Među najpoznatijim bibliotekama nalaze se NumPy, Pandas, Matplotlib, SciPy, TensorFlow i PyTorch, dok su Django i Flask među najpopularnijim okvirima za razvoj veb aplikacija.

Pajton se posebno ističe jednostavnošću sintakse i čitljivošću programskog kôda. Za razliku od mnogih drugih programskih jezika, naglasak je stavljen na preglednost i jasnoću programa, zbog čega se često preporučuje kao prvi programski jezik za učenje programiranja. Istovremeno, njegova izražajnost i bogat skup biblioteka čine ga pogodnim i za razvoj složenih profesionalnih softverskih sistema.

U ovoj knjizi Pajton će biti korišćen kao alat za implementaciju i eksperimentisanje sa različitim algoritamskim tehnikama. Njegova jednostavna sintaksa omogućava da se pažnja usmjeri na razumijevanje algoritama i struktura podataka, a ne na tehničke detalje implementacije karakteristične za jezike nižeg nivoa.

Nakon uspješnog savladavanja sadržaja ove glave student će biti u stanju da:

- objasni osnovne karakteristike programskog jezika Pajton i način predstavljanja podataka u memoriji;
- koristi osnovne numeričke i složene tipove podataka, uključujući stringove, liste, nizove, rječnike, skupove i torke;
- primijeni uslovne naredbe i petlje pri implementaciji jednostavnih proceduralnih algoritama;
- definiše i koristi funkcije, parametre, povratne vrijednosti i ugrađene funkcionalnosti za obradu kolekcija podataka;
- organizuje programski kod korištenjem modula i koristi odabrane module standardne biblioteke i eksternih biblioteka;
- prepoznaje, generiše i obrađuje izuzetke te koristi mehanizme za provjeru ispravnosti izvršavanja programa;
- koristi datoteke, regularne izraze za učitavanje, obradu i zapisivanje strukturiranih podataka;
- objasni osnovne faze izvršavanja programa napisanog u programskom jeziku Pajton, uključujući generisanje bajtkoda, ulogu virtuelne mašine i osnovne principe upravljanja memorijom.

2.1 Osnovne karakteristike

Pajton posjeduje niz osobina koje ga čine pogodnim kako za učenje programiranja, tako i za razvoj složenih softverskih sistema. U nastavku navodimo njegove najvažnije karakteristike.

- *Jezik visokog nivoa* (eng. *high-level language*). Pajton omogućava programiranje na visokom nivou apstrakcije, pri čemu se programer može fokusirati na rješavanje problema, a ne na detalje rada računara. Sintaksa jezika je jednostavna i pregledna, zbog čega je kôd relativno lako čitati i održavati.

- *Dinamički tipiziran jezik.* Tip promjenljive određuje se tokom izvršavanja programa. Zbog toga nije potrebno unaprijed deklarirati tip svake promjenljive. Na primjer, ista promjenljiva može u jednom trenutku sadržavati cijeli broj, a kasnije tekstualnu vrijednost.
- *Objektno-orijentisani jezik.* U Pajtonu su podaci i operacije nad njima organizovani kroz objekte i klase. Objektno-orijentisano programiranje omogućava modelovanje problema pomoću objekata i njihovih međusobnih interakcija, što često dovodi do preglednijeg i modularnijeg kôda.
- *Interpretirani i skriptni jezik.* Programi napisani u Pajtonu izvršavaju se posredstvom interpretera, bez potrebe za klasičnim procesom kompajliranja kakav postoji kod jezika poput C ili C++. Zbog toga je razvoj programa brz i pogodan za automatizaciju različitih zadataka.
- *Automatsko upravljanje memorijom.* Pajton posjeduje ugrađene mehanizme za automatsko oslobađanje memorije koja više nije potrebna programu (*garbage collection*). Time se značajno smanjuje mogućnost grešaka povezanih sa ručnim upravljanjem memorijom.

Zahvaljujući ovim karakteristikama, Pajton se danas koristi u obrazovanju, naučnim istraživanjima, analizi podataka, vještačkoj inteligenciji, razvoju veb aplikacija i mnogim drugim oblastima.

2.2 Pregled ugrađenih tipova podataka

Pajton posjeduje veliki broj ugrađenih (*built-in*) tipova podataka koji su dostupni bez potrebe za instalacijom dodatnih biblioteka. Svaki tip podataka definiše način predstavljanja i manipulacije određenom vrstom informacija.

Najvažniji ugrađeni tipovi podataka su:

- numerički tipovi: `int`, `float`, `complex`;

““

- tekstualni tipovi: `str`;
- sekvencijalni tipovi: `list`, `tuple`, `range`;
- mapirajući tipovi: `dict`;
- skupovni tipovi: `set`, `frozenset`;
- logički tip: `bool`;
- binarni tipovi: `bytes`, `bytearray`, `memoryview`. ““

Ovi tipovi predstavljaju osnovu za izgradnju složenijih struktura podataka i implementaciju različitih algoritama. U nastavku poglavlja detaljno ćemo razmotriti najvažnije od njih.

2.3 Deklarisanje varijabli i memorijska organizacija

Promjenljive u programskom jeziku Pajtonu kreiraju se dodjelom vrijednosti pomoću operatora dodjele `=`. Za razliku od mnogih drugih programskih jezika, nije potrebno unaprijed deklarirati tip promjenljive. Pajton automatski određuje odgovarajući tip na osnovu dodijeljene vrijednosti.

Na primjer, promjenljiva `x` kojoj se dodjeljuje cijeli broj 5 kreira se na sljedeći način:

```
x = 5
```

Slično tome, promjenljivoj `y` možemo dodijeliti tekstualnu vrijednost:

```
y = "abc"
```

Prilikom izvršavanja programa Pajton kreira odgovarajuće objekte u memoriji, dok promjenljive predstavljaju reference na te objekte. Zbog toga promjenljiva ne sadrži nužno samu vrijednost, već informaciju o tome gdje se odgovarajući objekat nalazi u memoriji.

2.3. DEKLARISANJE VARIJABLI I MEMORIJSKA ORGANIZACIJA 47

U narednim sekcijama detaljnije ćemo razmotriti numeričke, tekstualne i složene tipove podataka, kao i način njihovog korištenja u programima.

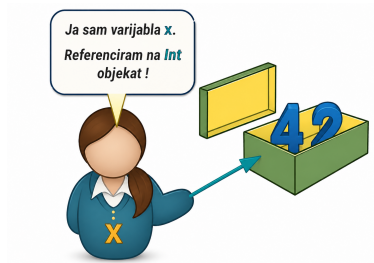
Pajton je objektno-orijentisani programski jezik, što znači da se sve vrijednosti u programu predstavljaju kao objekti. Drugim riječima, svaka promjenljiva zapravo referencira neki objekat smješten u memoriji.

Na primjer, promjenljiva koja sadrži cijeli broj predstavlja referencu na objekat tipa `int`, dok promjenljiva koja sadrži tekst predstavlja referencu na objekat tipa `str`. U tom smislu, promjenljive u Pajtonu ne čuvaju direktno vrijednosti, već reference na objekte koji sadrže te vrijednosti.

Posmatrajmo sljedeći primjer:

```
x = 42
```

Promjenljiva `x` referencira na objekat koji predstavlja broj 42, što se može ilustrovati kao na Slici 2.2.

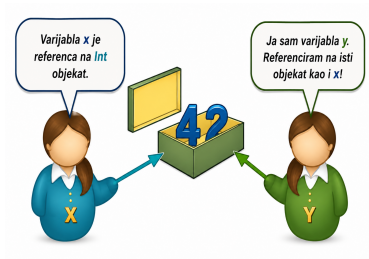


Slika 2.2: Promjenljiva `x` referencira objekat vrijednosti 42.

Ako zatim napišemo

```
y = x
```

promjenljiva `y` neće dobiti kopiju broja 42, već će referencirati isti objekat kao i promjenljiva `x`. Situacija je prikazana na Slici 2.3.

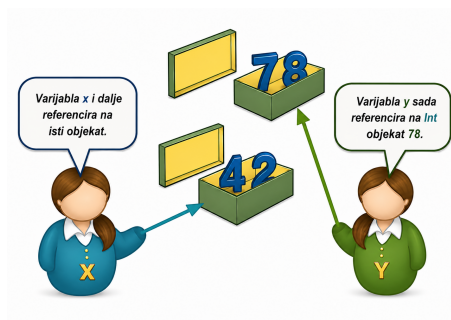


Slika 2.3: Dvije promjenljive referenciraju isti objekat.

Ukoliko nakon toga izvršimo naredbu

```
y = 72
```

promjenljiva y počinje referencirati novi objekat, dok x i dalje pokazuje na objekat koji predstavlja broj 42.

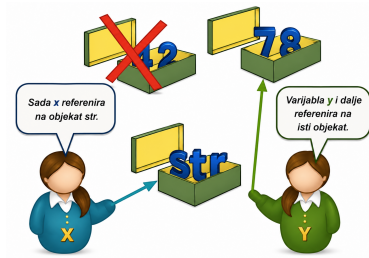


Slika 2.4: Promjenljive referenciraju različite objekte.

Slično tome, promjenljivoj možemo dodijeliti objekat potpuno drugog tipa:

```
x = "Text"
```

Nakon izvršavanja ove naredbe promjenljiva x više ne referencira objekat tipa `int`, već objekat tipa `str`.

Slika 2.5: Promjena reference promjenljive *x*.

Ovakvo ponašanje posljedica je činjenice da je Pajton dinamički tipiziran jezik. Tip se vezuje za objekat, a ne za promjenljivu. Zbog toga ista promjenljiva tokom izvršavanja programa može referencirati objekte različitih tipova.

Ipak, od verzije Pajton 3.5 moguće je koristiti oznake tipova (*type hints*) radi bolje čitljivosti programa:

```
x: str = "abc"
y: int = 23
```

Napomenimo da ove oznake ne mijenjaju način rada jezika, već prvenstveno služe kao dokumentacija i pomoć alatima za statičku analizu kôda.

Upravljanje memorijom

Pajton automatski upravlja memorijom. Svaki objekat posjeduje brojač referenci koji prati koliko promjenljivih ili drugih objekata trenutno pokazuje na njega.

Kada broj referenci nekog objekta postane jednak nuli, objekat više nije dostupan programu i njegova memorija može biti oslobođena. Pored toga, Pajton koristi i mehanizam sakupljanja otpadaka koji uklanja objekte koji više nisu potrebni, uključujući i složenije slučajeve kružnih referenci.

Zahvaljujući ovim mehanizmima, programer uglavnom ne mora ručno voditi računa o dodjeljivanju i oslobađanju memorije, što značajno pojednostavljuje razvoj programa.

Adresu objekta na koji referencira promjenljiva moguće je dobiti pomoću funkcije `id()`:

```
idnum = id(x)
idhex = hex(id(x))
```

Dobijena vrijednost predstavlja jedinstveni identifikator objekta tokom njegovog životnog vijeka.

2.4 Ispis poruka i komentari

Ispis rezultata na ekran vrši se pomoću funkcije `print()`:

```
print("Pozdrav svijete!")
print(x)
```

Funkcija `print()` može prikazati tekst, brojeve i druge objekte. Prije ispisa, objekat se po potrebi pretvara u tekstualni oblik.

Komentari služe za dokumentovanje programa i ne izvršavaju se tokom rada programa.

Jednolinijski komentar započinje znakom `#`:

```
# Ovo je komentar
```

```
x = 5
```

Za višelinijske komentare ili dokumentacione stringove najčešće se koriste trostruki navodnici:

```
"""
Ovo je višelinijski komentar
ili dokumentacioni string.
"""
```

Komentari povećavaju čitljivost programa i olakšavaju njegovo održavanje, naročito kod većih projekata.

2.5 Osnovni numerički tipovi podataka

Numerički tipovi podataka koriste se za predstavljanje brojeva i izvođenje aritmetičkih operacija nad njima. Pajton podržava nekoliko numeričkih tipova, od kojih su najvažniji `int`, `float` i `complex`.

2.5.1 Cjelobrojni i realni brojevi

Tip `int` (skraćeno od *integer*) koristi se za predstavljanje cijelih brojeva, dok tip `float` (skraćeno od *floating-point number*) služi za predstavljanje realnih brojeva.

Primjeri inicijalizacije ovih tipova dati su u nastavku:

```
x = 5          # int
y = -3         # int

z = 3.14       # float
w = -0.325     # float
```

Nad numeričkim tipovima moguće je primjenjivati standardne aritmetičke operacije kao što su sabiranje (+), oduzimanje (-), množenje (*) i dijeljenje (/).

Posebnu pažnju treba obratiti na operatore dijeljenja:

- `/` – realno dijeljenje;
- `//` – cjelobrojno dijeljenje.

Na primjer:

```
print(5 / 2)    # 2.5
print(5 // 2)   # 2
```

Pored osnovnih operatora, Pajton pruža i veliki broj ugrađenih funkcija za rad sa brojevima. Neke od najčešće korištenih su:

- `abs(x)` – apsolutna vrijednost broja;
- `round(x)` – zaokruživanje broja;
- `min(...)` – najmanja vrijednost;
- `max(...)` – najveća vrijednost.

Primjer 2.1

```
print(abs(-7))           # 7
print(round(3.14159,2))  # 3.14
print(min(2, 5, -1))     # -1
print(max(2, 5, -1))     # 5
```

2.5.2 Kompleksni brojevi

Tip `complex` koristi se za predstavljanje kompleksnih brojeva oblika $a + bj$, gdje su a i b realni brojevi, dok simbol j predstavlja imaginarnu jedinicu. Primjer rada sa kompleksnim brojevima:

```
x = 1 + 1j
y = 2 + 1j

z = x + y

print(z)           # (3+2j)
print(z.real)      # 3.0
print(z.imag)      # 2.0
```

Realni i imaginarni dio kompleksnog broja mogu se dobiti pomoću atributa `real` i `imag`. Kompleksni broj moguće je kreirati i korišćenjem funkcije `complex()`:

```
z = complex(3, 2)
print(z)          # (3+2j)
```

Prvi argument predstavlja realni dio, dok drugi argument predstavlja imaginarni dio kompleksnog broja.

2.6 Naredbe grananja i petlje

U ovoj sekciji upoznaćemo osnovne kontrolne strukture programskog jezika Pajton: naredbu grananja `if` te petlje `for` i `while`.

Prije toga, u Tabeli 2.1 dati su najčešće korišteni operatori poređenja.

Svaki izraz poređenja vraća logičku vrijednost `True` ili `False`, u zavisnosti od toga da li je odgovarajući uslov ispunjen.

Tabela 2.1: Osnovni operatori poređenja u Pajtonu.

Matematička notacija	Pajtonova notacija	Značenje
$a < b$	<code>a < b</code>	a je manje od b
$a \leq b$	<code>a <= b</code>	a je manje ili jednako od b
$a > b$	<code>a > b</code>	a je veće od b
$a \geq b$	<code>a >= b</code>	a je veće ili jednako od b
$a = b$	<code>a == b</code>	a je jednako b
$a \neq b$	<code>a != b</code>	a nije jednako b

2.6.1 Uslovna naredba `if`

Najjednostavniji oblik naredbe grananja dat je sljedećom sintaksom:

```
if uslov:
    naredba_1
else:
    naredba_2
```

Tok izvršavanja je jednostavan:

- ako je uslov tačan, izvršava se blok `naredba_1`;
- u suprotnom se izvršava blok `naredba_2`.

Posebno treba obratiti pažnju na uvlačenje koda (*indentation*). Za razliku od mnogih drugih programskih jezika, u Pajtonu uvlačenje nije samo pitanje stila, već predstavlja dio sintakse jezika.

Nakon izvršavanja odgovarajućeg bloka program nastavlja sa prvom naredbom koja se nalazi izvan `if`-strukture.

U situacijama kada postoji više alternativa koristi se naredba `elif`:

```
if uslov_1:
    naredba_1
elif uslov_2:
    naredba_2
elif uslov_3:
    naredba_3
else:
    naredba_n
```

Uslovi se provjeravaju redom, od vrha prema dnu. Čim se pronađe prvi tačan uslov, izvršava se odgovarajući blok naredbi, nakon čega se ostatak strukture preskače.

Ako nijedan od uslova nije zadovoljen, izvršava se blok pod granom `else`.

2.6.2 For i While petlje

Petlje predstavljaju kontrolne strukture koje omogućavaju višestruko izvršavanje istog bloka koda. Koriste se kada je potrebno ponoviti određene operacije unaprijed poznat broj puta ili sve dok je zadovoljen određeni uslov.

Pajton podržava dvije osnovne vrste petlji:

- `for` petlju;
- `while` petlju.

For petlja. Koristi se za prolazak kroz elemente iterabilnih objekata, kao što su liste, torke, stringovi, skupovi i rječnici.

Njena osnovna sintaksa je:

```
for item in iterable:
    naredba
```

Promjenljivoj `item` redom se dodjeljuju elementi kolekcije `iterable`, a za svaki element izvršava se odgovarajući blok koda.

Na primjer:

```
for broj in [1, 2, 3, 4]:
    print(broj)
```

Izvršavanjem ovog programa na ekran će biti ispisani brojevi 1, 2, 3 i 4.

Vrlo često se `for` petlja koristi zajedno sa funkcijom `range()`:

```
for i in range(5):
    print(i)
```

Ovaj kôd ispisuje brojeve od 0 do 4.

while petlja. `while` petlja izvršava blok koda sve dok je zadovoljen zadan logički uslov. Njena osnovna sintaksa je:

```
while uslov:
    naredba
```

Prije svake iteracije provjerava se vrijednost izraza `uslov`. Ako je njegova vrijednost *True*, izvršava se tijelo petlje. Kada uslov postane netačan (*False*), izvršavanje petlje se završava, a program nastavlja sa prvom naredbom nakon petlje.

Na primjer, prvih deset parnih prirodnih brojeva može se ispisati na sljedeći način:

```
i = 1
while i <= 10:
    print(2 * i)
    i += 1
```

Naredba

```
i += 1
```

predstavlja skraćeni zapis za

```
i = i + 1
```

`i` koristi se za povećavanje vrijednosti promjenljive `i` za jedan (inkrementiranje).

2.7 Složene strukture podataka

Pored osnovnih numeričkih tipova podataka, Pajton pruža i niz složenijih struktura podataka koje omogućavaju efikasno organizovanje i obradu većih količina informacija. Najvažnije među njima su:

- stringovi (`str`);
- liste (`list`);
- torke (`tuple`);
- skupovi (`set`);
- rječnici (`dict`).

Sve navedene strukture predstavljaju kolekcije objekata i koriste se u gotovo svakom ozbiljnijem programskom jeziku.

2.7.1 Stringovni tip podataka

Tip `str` predstavlja ugrađeni tip podataka namijenjen radu sa tekstom. String se može posmatrati kao uređena sekvenca Unicode znakova.

Jedna od najvažnijih osobina stringova jeste da su *nepromjenjivi* (*immutable*). Nakon što se string kreira, njegov sadržaj nije moguće direktno mijenjati. Svaka izmjena zapravo rezultuje kreiranjem novog string objekta.

Inicijalizacija stringova. String se može definisati korišćenjem jednostrukih ili dvostrukih navodnika:

```
string0 = 'hello'
string1 = "world"
```

Za višelinjske stringove koriste se trostruki navodnici:

```
multiline_str = """
Ovo je
višelinjski string.
"""
```

Osnovne operacije nad stringovima

- **Konkatenacija**

Stringovi se mogu spajati pomoću operatora +:

```
ime = "Marko"
prezime = "Markovic"

print(ime + " " + prezime)
```

- **Pristup pojedinačnim znakovima**

Znakovima stringa pristupa se pomoću operatora indeksiranja []. “

```
tekst = "Python"

print(tekst[0])    # P
print(tekst[-1])   # n
```

Pozitivni indeksi broje se od početka stringa, dok negativni indeksi broje od kraja stringa.

- **Rezanje stringa (slicing)**

Dio stringa može se izdvojiti pomoću operatora [:]:

```
tekst = "world"

print(tekst[1:3])  # or
print(tekst[:3])   # wor
print(tekst[2:])   # rld
```

- **Ponavljanje stringa**

Operator * omogućava višestruko ponavljanje stringa:

```
print("abc" * 3)
```

Rezultat je:

```
abccabccabc
```

- **Dužina stringa**

Broj znakova u stringu dobija se funkcijom `len()`:

```
print(len("Python"))
```

- **Brisanje reference**

Ključna riječ `del` uklanja referencu na objekat:

```
tekst = "abc"
del tekst
```

Korisne metode. Klasa `str` sadrži veliki broj ugrađenih metoda. Neke od najčešće korištenih su:

- `lower()` – pretvara sva slova u mala;

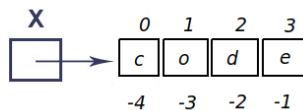
- `upper()` – pretvara sva slova u velika;
- `strip()` – uklanja praznine sa početka i kraja stringa;
- `replace()` – zamjenjuje dio stringa drugim tekstom;
- `find()` – pronalazi poziciju podstringa.

Memorijska organizacija. Stringovi se u Pajtonu predstavljaju kao uređene sekvence Unicode znakova smještene u neprekidnom dijelu memorije. Svakom znaku pridružen je odgovarajući Unicode kôd, što omogućava rad sa tekstom na velikom broju svjetskih jezika.

Pojednostavljen prikaz memorijske organizacije stringa

```
x = "code"
```

dat je na Slici 2.6.



Slika 2.6: Pojednostavljen prikaz memorijske organizacije stringa `code`.

Za razliku od nekih drugih programskih jezika, stringovi u Pajtonu su *nepromjenjivi* (*immutable*). To znači da se sadržaj postojećeg stringa ne može direktno mijenjati. Svaka operacija koja naizgled mijenja string zapravo kreira novi string objekt.

```
s = "code"
s1 = s + "!"
```

Nakon izvršavanja druge naredbe kreira se novi string `code!`, dok originalni string ostaje nepromijenjen.

Dodatna napomena. Pajton interno koristi različite optimizacije za rad sa stringovima. Jedna od njih je *interniranje stringova* (eng. *string interning*), pri čemu se identični stringovi po potrebi mogu dijeliti između više promjenljivih. Time se smanjuje potrošnja memorije i poboljšavaju performanse programa.

Formatiranje stringova. Prilikom ispisa rezultata često je potrebno kombinovati tekst i vrijednosti promjenljivih. Pajton nudi nekoliko načina za formatiranje stringova.

Jedan od tradicionalnih pristupa koristi metodu `format()`:

```
print("Moje ime je {0}, a prezime {1}".  
      format("Marko", "Marković"))
```

Vrijednost označena sa 0 zamjenjuje se prvim argumentom funkcije `format()`, dok se 1 zamjenjuje drugim argumentom.

Savremeniji i danas preporučeni način predstavljaju formatirani stringovi (*f-strings*):

```
ime = "Marko"  
godine = 32  
print(f"Ime: {ime}, godine: {godine}")
```

Kod *f*-stringova vrijednosti promjenljivih navode se direktno unutar vitičastih zagrada, što kôd čini preglednijim i jednostavnijim za pisanje.

Zbog svoje čitljivosti i efikasnosti, *f*-stringovi predstavljaju preporučeni način formatiranja stringova u savremenom Pajtonu.

2.7.2 Liste

Lista predstavlja uređenu, indeksiranu i promjenljivu kolekciju elemenata. U Pajtonu se implementira pomoću ugrađenog tipa podataka `list`.

Za razliku od mnogih drugih programskih jezika, elementi liste ne moraju biti istog tipa. Zbog toga kažemo da su liste *heterogene* kolekcije. Na primjer, ista lista može sadržavati brojeve, stringove, logičke vrijednosti

i druge objekte. Elementi liste indeksiraju se brojevima počevši od nule. Pored standardnog indeksiranja, Pajton podržava i negativne indekse. Pri tome indeks -1 označava posljednji element liste, -2 pretposljednji itd.

Inicijalizacija liste. Liste se mogu kreirati na više načina:

```
lista = list()
lista1 = ["a", "b", 2.0, 3]
lista2 = list(("abc", "def", 2))
```

Pristup elementima. Pojedinačnim elementima liste pristupa se pomoću operatora `[]`.

```
lista = [10, 20, 30]
print(lista[0])    # 10
print(lista[-1])   # 30
```

U prvom slučaju pristupa se prvom elementu liste, dok se u drugom slučaju koristi negativni indeks za pristup posljednjem elementu.

Rezanje liste (eng. slicing). Dio liste može se izdvojiti pomoću operatora

```
lista[indeks1:indeks2:korak]
```

pri čemu se uzimaju elementi od pozicije `indeks1` do pozicije `indeks2` (isključivo), uz zadani korak.

```
lista = [0, 1, 2, 3, 4, 5]
print(lista[1:4])    # [1, 2, 3]
print(lista[:3])     # [0, 1, 2]
print(lista[::2])    # [0, 2, 4]
```

Rezultat operacije rezanja uvijek je nova lista.

Provjera pripadnosti. Operator `in` koristi se za provjeru da li se određeni element nalazi u listi.

```
if 5 in lista:  
    print("Pronađen element")
```

Vrijednost izraza je *True* ako se element nalazi u listi, odnosno *False* u suprotnom.

Iteriranje kroz listu. Elementima liste može se pristupati jedan po jedan pomoću `for` petlje.

```
for x in lista:  
    print(x)
```

Ovakav način prolaska kroz listu predstavlja jednu od najčešće korištenih programskih konstrukcija u Pajtonu.

Dodavanje i brisanje elemenata. Novi element može se dodati na kraj liste metodom `append()`:

```
lista.append(10)
```

Element se uklanja metodom `remove()`, pri čemu se briše prvo pojavljivanje zadate vrijednosti:

```
lista.remove(10)
```

Posljednji element liste može se ukloniti metodom `pop()`:

```
lista.pop()
```

Metoda `pop()` vraća uklonjeni element, što je često korisno u praksi.

Umetanje elemenata. Element se može umetnuti na proizvoljnu poziciju korištenjem metode `insert()`:

```
lista.insert(2, 100)
```

Nakon izvršavanja ove naredbe vrijednost 100 nalaziće se na poziciji 2, dok će svi naredni elementi biti pomjereni za jedno mjesto udesno.

Obrtanje i kopiranje liste. Redoslijed elemenata liste može se obrnuti metodom `reverse()`:

```
lista.reverse()
```

Kopija liste kreira se metodom `copy()`:

```
nova_lista = lista.copy()
```

Na ovaj način dobija se nova lista koja sadrži iste elemente kao i originalna lista.

Najčešće korištene metode

Tabela 2.2 daje pregled najvažnijih metoda klase `list`.

Tabela 2.2: Najčešće korištene metode liste.

Metoda	Opis
<code>append(x)</code>	Dodaje element na kraj liste
<code>insert(i, x)</code>	Umeće element na poziciju <code>i</code>
<code>remove(x)</code>	Briše prvo pojavljivanje elementa
<code>pop()</code>	Uklanja i vraća posljednji element
<code>reverse()</code>	Obrće redoslijed elemenata
<code>copy()</code>	Kreira kopiju liste
<code>clear()</code>	Briše sve elemente liste
<code>sort()</code>	Sortira elemente liste

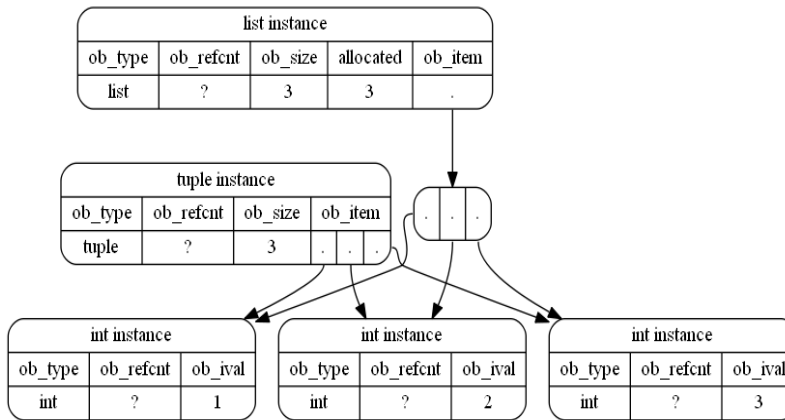
Memorijska organizacija

Liste su u Pajtonu implementirane kao *dinamički nizovi* (eng. *dynamic arrays*). To znači da se njihova veličina može mijenjati tokom izvršavanja programa, za razliku od klasičnih nizova čija je veličina unaprijed fiksirana.

Prilikom kreiranja liste interpreter rezerviše određeni memorijski prostor za smještanje elemenata. Kada se lista proširi i raspoloživi kapacitet više nije dovoljan za pohranu novih elemenata, automatski se dodjeljuje veći memorijski blok, nakon čega se postojeći elementi kopiraju u novu memorijsku oblast. Ovaj postupak naziva se *realokacija memorije*.

Kako bi se smanjio broj realokacija, Pajton obično rezerviše nešto više memorije nego što je trenutno potrebno. Zahvaljujući tome, dodavanje novih elemenata na kraj liste najčešće se izvršava veoma efikasno.

Pojednostavljen prikaz memorijske organizacije liste i torke dat je na Slici 2.7.



Slika 2.7: Pojednostavljen prikaz memorijske organizacije liste i torke.

U situacijama kada je unaprijed poznat maksimalan broj elemenata koji će se koristiti, često se kreira lista željene veličine:

```
lista = [None] * max_broj_elementa
```

Na ovaj način izbjegava se veliki broj naknadnih proširenja liste, što može pozitivno uticati na performanse programa.

Dodatne napomene. Za razliku od klasičnih nizova u programskom jeziku C, liste u Pajtonu mogu sadržavati elemente različitih tipova:

```
lista = [1, "tekst", 3.14, True]
```

Ova osobina čini liste veoma fleksibilnim i jednostavnim za upotrebu. Međutim, takva fleksibilnost ima svoju cijenu. Svaki element liste predstavlja zaseban Pajtonov objekat, koji pored stvarne vrijednosti sadrži i dodatne informacije, kao što su tip objekta i broj referenci na njega.

Interno posmatrano, lista ne sadrži same objekte već niz pokazivača na odgovarajuće objekte u memoriji. Zbog toga liste zauzimaju više memorijskog prostora nego homogeni nizovi implementirani u jezicima nižeg nivoa.

U većini praktičnih primjena liste predstavljaju osnovnu i najčešće korištenu strukturu podataka u Pajtonu. Zahvaljujući jednostavnoj sintaksi, podršci za dinamičko proširivanje i velikom broju ugrađenih metoda, one su često prvi izbor za organizaciju i obradu kolekcija podataka.

2.7.3 Nizovi: modul array

Pored lista, Pajton nudi i modul `array`, koji omogućava rad sa *homogenim* nizovima podataka. Po svojoj organizaciji i načinu korištenja, objekti tipa `array` slični su nizovima u programskim jezicima C i C++.

Za razliku od lista, koje mogu sadržavati elemente različitih tipova, svi elementi objekta `array` moraju biti istog tipa. Zbog toga se podaci mogu skladištiti efikasnije i zauzimati manje memorijskog prostora.

Elementi niza smješteni su u neprekidnom bloku memorije, što omogućava brz pristup proizvoljnom elementu korištenjem njegovog indeksa. Indeksiranje elemenata počinje od nule, kao i kod lista.

Prilikom kreiranja objekta tipa `array` potrebno je navesti oznaku tipa podataka (*typecode*) koji će se skladištiti. Najčešće korištene oznake prikazane su u Tabeli 2.3.

Tabela 2.3: Najčešće korišteni tipovi podataka u modulu `array`.

Typecode	Tip podatka	Pajtonov tip	Veličina (B)
'b' / 'B'	signed / unsigned char	int	1
'h' / 'H'	signed / unsigned short	int	2
'i' / 'I'	signed / unsigned int	int	4
'l' / 'L'	signed / unsigned long	int	4
'q' / 'Q'	signed / unsigned long long	int	8
'f'	float	float	4
'd'	double	float	8

Da bismo koristili modul `array`, potrebno ga je prethodno uključiti:

```
import array
```

Inicijalizacija. Objekat tipa `array` kreira se navođenjem oznake tipa i početnih vrijednosti:

```
from array import array
```

```
my_array = array('i', [1, 2, 3, 4, 5])
```

U ovom primjeru oznaka `'i'` označava cjelobrojne vrijednosti tipa `int`.
Osnovne operacije.

- **Pristup elementima**

```
print(my_array[0])    # 1
print(my_array[2])    # 3
```

- **Izmjena elemenata**

```
my_array[0] = 7
print(my_array)
# array('i', [7, 2, 3, 4, 5])
```

- **Dodavanje elementa na kraj niza**

```
my_array.append(10)
```

- **Dodavanje više elemenata**

```
from array import array

floats = array('f', [2.0, 3.2, 1.4])
floats.extend([1.1, 4.1, 5.2, 9.0])
```

- Umetanje elementa

```
my_array.insert(2, 100)
```

- Brisanje elementa

```
my_array.remove(100)
```

- Uklanjanje posljednjeg elementa

```
my_array.pop()
```

Modul `array` je posebno koristan kada radimo sa velikim kolekcijama numeričkih podataka istog tipa. U poređenju sa listama, objekti tipa `array` zauzimaju manje memorije i omogućavaju efikasnije skladištenje homogenih podataka.

Ipak, u naučnom računarstvu i analizi podataka danas se mnogo češće koriste strukture iz biblioteke `NumPy`, koje predstavljaju proširenje ideje homogenih nizova i nude znatno bolje performanse i bogatiji skup funkcionalnosti.

2.7.4 Rječnici

Rječnik (`dict`) predstavlja jednu od najvažnijih ugrađenih struktura podataka u Pajtonu. Riječ je o promjenljivoj kolekciji elemenata organizovanih u obliku parova *ključ-vrijednost*.

Za razliku od lista, gdje se elementima pristupa pomoću numeričkih indeksa, u rječnicima se pristup vrši preko ključeva. Svakom ključu pridružena je tačno jedna vrijednost, pri čemu ključevi moraju biti jedinstveni unutar rječnika.

Rječnici se često nazivaju i *heš mapama* (eng. *hash maps*), jer su interno implementirani korištenjem heš tabela.

Inicijalizacija. Rječnik se može kreirati na više načina.

```
# Kreiranje pomoću vitičastih zagrada
dct = {
    "ime": "Mirko",
    "godine": 30,
    "grad": "Banja Luka"
}

# Kreiranje pomoću konstruktora dict()

dct1 = dict(
    ime="Mirko",
    godine=30,
    grad="Banja Luka"
)
```

Osnovne operacije.

- **Pristup vrijednosti**

Vrijednost pridružena određenom ključu dobija se korištenjem operatora `[]`.

```
dct = {
    "ime": "Mirko",
    "godine": 30,
    "grad": "Banja Luka"
}

print(dct["ime"])      # Mirko
```

- **Dodavanje i izmjena elemenata:**

```
dct["godine"] = 31
```

Ako ključ već postoji, njegova vrijednost se ažurira. U suprotnom se dodaje novi par ključ–vrijednost.

- **Brisanje elementa:** Element se može ukloniti korištenjem ključne riječi `del` ili metode `pop()`.

```
del dct["grad"]
```

ili

```
dct.pop("grad")
```

- **Provjera postojanja ključa:** Operator `in` provjerava da li se određeni ključ nalazi u rječniku.

```
print("ime" in dct)      # True
print("pol" in dct)     # False
```

- **Iteriranje kroz rječnik:**

```
for kljuc, vrijednost in dct.items():
    print(kljuc, vrijednost)
```

- **Druge korisne metode:**

- `len(dct)` – vraća broj elemenata rječnika;
- `keys()` – vraća kolekciju svih ključeva;
- `values()` – vraća kolekciju svih vrijednosti;
- `items()` – vraća parove (ključ, vrijednost);
- `get(kljuc)` – sigurno dohvaća vrijednost bez generisanja greške ukoliko ključ ne postoji.

Memorijska organizacija. Rječnici su u Pajtonu implementirani korištenjem heš tabela. Osnovna ideja sastoji se u tome da se svaki ključ transformiše u cijeli broj primjenom heš funkcije. Dobijena vrijednost koristi se za brzo pronalaženje mjesta na kojem se nalazi odgovarajuća vrijednost.

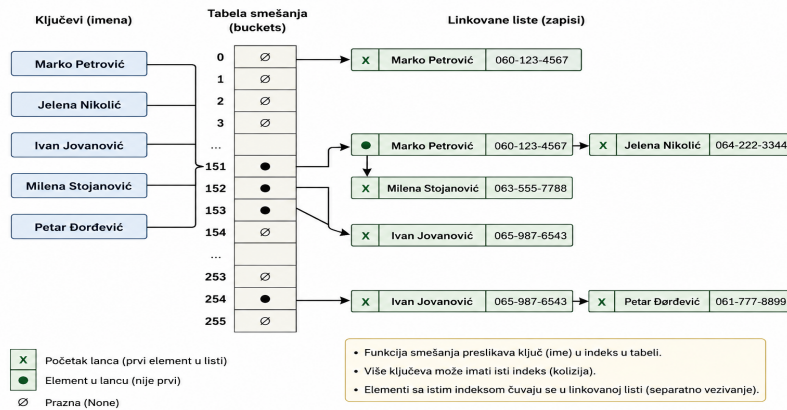
Zahvaljujući ovakvoj organizaciji, osnovne operacije nad rječnicima imaju očekivanu vremensku složenost:

- pristup elementu: $O(1)$,
- dodavanje elementa: $O(1)$,
- brisanje elementa: $O(1)$.

U najgorem slučaju ove operacije mogu biti sporije zbog pojave kolizija, ali se u praksi takve situacije rijetko javljaju.

Prilikom kreiranja rječnika interpreter rezerviše određeni broj mjesta u heš tabeli. Kako broj elemenata raste, tabela se po potrebi automatski proširuje kako bi se očuvala efikasnost pristupa.

Pojednostavljen prikaz organizacije rječnika dat je na Slici 2.8.



Slika 2.8: Pojednostavljen prikaz unutrašnje organizacije rječnika u Pajtonu.

Dodatne napomene. Ključevi u rječniku moraju biti nepromjenljivi objekti. Zbog toga se kao ključevi mogu koristiti objekti tipa `int`, `float`, `str` ili `tuple`, dok liste (`list`) ne mogu biti ključevi.

```
dct = {
    "ime": "Mirko",
    123: "identifikator",
```

```

        (1, 2): "par brojeva"
    }

```

Zbog veoma brzog pristupa podacima, rječnici predstavljaju jednu od najčešće korištenih struktura podataka u modernom programiranju. Posebno su korisni kada je potrebno efikasno pretraživanje, grupisanje ili indeksiranje podataka.

2.7.5 Skupovi

Skup (**set**) predstavlja neuređenu kolekciju jedinstvenih elemenata. Po svojim osobinama odgovara matematičkom pojmu skupa: redoslijed elemenata nije važan, a duplikati nisu dozvoljeni.

Osnovna namjena skupova jeste efikasna provjera pripadnosti elementa kolekciji. Zbog toga se često koriste u algoritmima koji zahtijevaju brzo pretraživanje ili eliminisanje duplikata.

Elementi skupa mogu biti objekti nepromjenljivih tipova podataka, kao što su `int`, `float`, `str`, `tuple` ili `complex`. Sa druge strane, promjenljive kolekcije poput lista (`list`), rječnika (`dict`) i drugih skupova (**set**) ne mogu biti elementi skupa.

Inicijalizacija. Skup se može kreirati na nekoliko načina:

```

skup = set()

skup1 = {1, 2, 3, 4}
skup2 = set((1, 2, "3"))
skup3 = set([1, 2, 3])

```

Prazan skup se kreira pomoću funkcije `set()`. Izraz

```
skup = {}
```

ne kreira prazan skup već prazan rječnik.

Osnovne metode.

- `add(elem)` – dodaje element u skup;

```
skup.add(5)
```

- `remove(elem)` – uklanja element iz skupa; generiše grešku ukoliko element ne postoji;

```
skup.remove(5)
```

- `discard(elem)` – uklanja element ako postoji, bez generisanja greške;

```
skup.discard(5)
```

- `pop()` – uklanja i vraća proizvoljan element skupa;
- `clear()` – uklanja sve elemente iz skupa;
- `union()` ili operator `|` – vraća uniju dva skupa;

```
A = {1, 2, 3}
```

```
B = {3, 4, 5}
```

```
print(A | B)
```

```
# {1, 2, 3, 4, 5}
```

- `intersection()` ili operator `&` – vraća presjek dva skupa;

```
print(A & B)
```

```
# {3}
```

- `difference()` ili operator `-` – vraća razliku skupova;

```
print(A - B)

# {1, 2}
```

- Operator `in` koristi se za provjeru pripadnosti elementa skupu;

```
print(3 in A)    # True
print(7 in A)    # False
```

Memorijska organizacija. Skupovi su u Pajtonu implementirani korištenjem heš tabela. Svakom elementu pridružuje se heš vrijednost koja omogućava veoma brzo pronalaženje njegove pozicije u memoriji.

Zahvaljujući takvoj organizaciji, osnovne operacije nad skupovima imaju očekivanu vremensku složenost:

- dodavanje elementa: $O(1)$;
- uklanjanje elementa: $O(1)$;
- provjera pripadnosti: $O(1)$.

U najgorem slučaju, zbog pojave većeg broja kolizija, složenost može porasti na $O(n)$, ali se takve situacije u praksi rijetko javljaju.

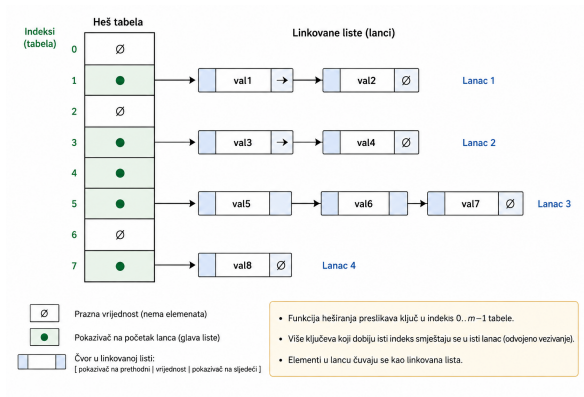
Sljedeći primjer pokazuje da je skup predstavljen ugrađenim tipom podataka `set`:

```
s = {1, 2, 3}

print(type(s))

# <class 'set'>
```

Pojednostavljen prikaz organizacije skupa dat je na Slici 2.9.



Slika 2.9: Pojednostavljen prikaz unutrašnje organizacije skupa u Pajtonu.

Dodatne napomene. Skupovi su naročito korisni kada je potrebno:

- ukloniti duplikate iz kolekcije podataka;
- efikasno provjeriti da li se element nalazi u kolekciji;
- izvršavati skupovne operacije poput unije, presjeka i razlike.

Zbog veoma brzih operacija pretraživanja, skupovi predstavljaju jednu od najvažnijih struktura podataka u algoritmici i često se koriste za optimizaciju vremena izvršavanja programa.

2.7.6 Torke

Torka (`tuple`) predstavlja uređenu, indeksiranu i nepromjenljivu kolekciju elemenata. U Pajtonu se implementira pomoću ugrađenog tipa podataka `tuple`.

Za razliku od lista, elementi torke se ne mogu mijenjati nakon kreiranja objekta. Torke mogu sadržavati elemente različitih tipova podataka i dopuštaju pojavljivanje duplikata.

Inicijalizacija. Torka se može kreirati na nekoliko načina:

```
my_tuple = ("aa", "bb", 2.0, 3)
my_tuple1 = tuple(["abc", 3.2, "a"])
```

Torka sa jednim elementom zahtijeva završni zarez:

```
t = (5,)
```

Bez zareza izraz (5) predstavlja običan cjelobrojni objekat.

Osnovne operacije.

- **Pristup elementima:** Elementima torke pristupa se pomoću operatora []:

```
my_tuple = (1, 2, 3, 2, 4, 2, 5)
print(my_tuple[2])    # 3
print(my_tuple[-1])   # 5
```

- **Broj pojavljivanja elementa** u torci se dobija metodom count():

```
my_tuple = (1, 2, 3, 2, 3, 5)
print(my_tuple.count(2)) # 2
```

- **Pronalaženje indeksa elementa** se vrši metodom index():

```
print(my_tuple.index(2)) # 1
```

- **Dužina torke** se vraća funkcijom len():

```
print(len(my_tuple)) # 6
```

- **Spajanje torki:** Dvije torke mogu se spojiti operatorom +.

```
t1 = ("a", "bb", 2)
t2 = ("ee", 2, 3.4)
t3 = t1 + t2
print(t3)
# ("a", "bb", 2, "ee", 2, 3.4)
```

- **Rezanje torke (slicing).** Kao i kod lista, moguće je izdvojiti dio torke

```
t = (0, 1, 2, 3, 4, 5)
print(t[1:4]) # (1, 2, 3)
```

Memorijska organizacija. Torke su nepromjenljive strukture podataka. Nakon kreiranja nije moguće dodavati, uklanjati niti mijenjati njihove elemente. Zbog toga Pajton može efikasnije upravljati memorijom nego kod promjenljivih struktura kao što su liste.

Objekat tipa `tuple` sadrži reference na svoje elemente. Pošto se broj elemenata ne može mijenjati nakon kreiranja torke, nije potrebno vršiti realokaciju memorije tokom izvršavanja programa.

Pristup elementima torke ostvaruje se direktnim indeksiranjem, pa je vremenska složenost operacije pristupa elementu jednaka $O(1)$.

Zbog svoje nepromjenljivosti, torke često zauzimaju manje memorije od lista sa istim brojem elemenata i mogu se koristiti kao ključevi u rječnicima ili kao elementi skupova, pod uslovom da svi njihovi elementi takođe budu nepromjenljivi. Poređenje memorijske organizacije lista i torki prikazano je na Slici 2.7.

Torke su pogodan izbor kada:

- podaci ne treba da se mijenjaju nakon kreiranja;
- želimo smanjiti potrošnju memorije;
- želimo koristiti kolekciju kao ključ u rječniku ili element skupa;
- želimo jasno naglasiti da određeni skup podataka predstavlja jednu logičku cjelinu.

Tipični primjeri upotrebe su predstavljanje koordinata tačke, datuma, RGB boja i drugih nepromjenljivih zapisa podataka.

2.8 Funkcije

Funkcije predstavljaju jedan od osnovnih mehanizama za organizaciju programa. Njihova uloga je da složen problem podijele na manje, logički zaokružene cjeline koje se mogu višestruko koristiti.

U Pajtonu je svaka funkcija objekat. To znači da se funkcija može dodijeliti varijabli, proslijediti kao argument drugoj funkciji ili vratiti kao rezultat izvršavanja funkcije. Kada interpreter obradi definiciju funkcije, kreira se objekat tipa `function` koji sadrži informacije o nazivu funkcije, parametrima, tijelu funkcije i drugim metapodacima.

Funkcija se definiše pomoću ključne riječi `def`. Nakon naziva funkcije navode se parametri u zagradama, a tijelo funkcije se piše u uvučenom bloku koda. Naredba `return` služi za vraćanje rezultata i završetak izvršavanja funkcije.

```
def proizvod_brojeva(x, y):  
    proizvod = x * y  
    return proizvod
```

Funkcija se poziva navođenjem njenog naziva i odgovarajućih argumenata:

```
rezultat = proizvod_brojeva(4, 3)  
print(rezultat) # 12
```

Parametrima funkcije mogu se dodijeliti podrazumijevane vrijednosti koje će biti korištene ukoliko korisnik ne navede odgovarajući argument.

```
def umnozi(tekst="Mirko", broj_ponavljanja=1):  
    print(tekst * broj_ponavljanja)  
    umnozi()
```

U ovom slučaju ispisuje se

```
Mirko
```

jer se koriste podrazumijevane vrijednosti parametara.

Pajton omogućava definisanje funkcija koje prihvataju proizvoljan broj pozicionih ili imenovanih argumenata. Operator `*` sakuplja dodatne pozicione argumente u toroku:

```
def ispisi_brojeve(*brojevi):  
    for broj in brojevi:  
        print(broj)
```

Operator `**` sakuplja imenovane argumente u rječnik:

```
def ispisi_podatke(**podaci):  
    for kljuc, vrijednost in podaci.items():  
        print(kljuc, ":", vrijednost)  
  
#main  
ispisi_podatke(  
    ime="Mirko",  
    prezime="Marković"  
)
```

Funkcija može vratiti više vrijednosti odjednom. U pozadini se takve vrijednosti pakuju u toroku.

```
def podjela(x, y):  
    kolicnik = x // y  
    ostatak = x % y  
  
    return kolicnik, ostatak  
  
#main  
k, o = podjela(10, 3)  
print(k, o)  # 3 1
```

Prosljeđivanje argumenata. U Pajtonu se funkcijama prosljeđuje referenca na objekat. Zbog toga ponašanje zavisi od toga da li je objekat promjenljiv ili nepromjenljiv.

Promjenljivi objekti (liste, rječnici i skupovi) mogu biti izmijenjeni unutar funkcije:

```
def dodaj_element(lista, element):
    lista.append(element)

#main
brojevi = [1, 2, 3]
dodaj_element(brojevi, 5)
print(brojevi)  # [1, 2, 3, 5]
```

Pošto funkcija i pozivalac referenciraju isti objekat liste, promjena je vidljiva i nakon povratka iz funkcije. Ako želimo izbjeći izmjenu originalne liste, potrebno je funkciji proslijediti kopiju:

```
dodaj_element(brojevi.copy(), 5)
```

Cijeli brojevi, decimalni brojevi, stringovi i torke predstavljaju nepromjenljive objekte. Njihova izmjena u tijelu funkcije zapravo podrazumijeva kreiranje novog objekta.

```
def inkrementiraj(x):
    x += 1
    print("Unutar funkcije:", x)

#main
y = 5
inkrementiraj(y)
print(y)
```

Rezultat izvršavanja je:

```
Unutar funkcije: 6
5
```

Vrijednost varijable `y` ostaje nepromijenjena jer izraz `x += 1` kreira novi objekat i lokalna varijabla `x` počinje da referencira taj novi objekat.

Globalne varijable. Ključna riječ `global` omogućava izmjenu pristupa varijabli unutar funkcije gdje ona postaje globalno vidljiva.

```
def my_func():
    global x
    x = 10
#main
x = 20
my_func()
print(x)  # 10
```

Ipak, prekomjerna upotreba globalnih varijabli se ne preporučuje jer otežava razumijevanje i održavanje programa.

2.8.1 Funkcije kao argumenti

Pošto su funkcije objekti, mogu se prosljeđivati drugim funkcijama kao argumenti.

```
def saberi(a, b):
    return a + b

def primijeni(funkcija, x, y):
    return funkcija(x, y)

#main
rezultat = primijeni(saber, 2, 3)
print(rezultat)  # 5
```

Ovakav pristup predstavlja osnovu mnogih naprednih tehnika programiranja, uključujući funkcionalno programiranje i rad sa bibliotekama za obradu podataka.

2.8.2 Anonimne funkcije.

Anonimne funkcije definišu se pomoću ključne riječi `lambda`, zbog čega se često nazivaju i *lambda funkcije*. Naziv “anonimne” potiče od činjenice da se funkcija može definisati bez eksplicitnog navođenja njenog imena, za razliku od standardnih funkcija koje se definišu pomoću ključne riječi `def`.

Opšta sintaksa lambda funkcije je:

```
lambda argumenti: izraz
```

Pri tome, *argumenti* predstavljaju ulazne parametre funkcije, dok *izraz* definiše operaciju koja se izvršava nad tim parametrima. Vrijednost izraza automatski predstavlja rezultat funkcije.

Sljedeći primjer definiše lambda funkciju za sabiranje dva broja:

```
suma = lambda x, y: x + y
print(suma(4, 3)) # Output: 7
```

U ovom primjeru lambda funkcija je dodijeljena varijabli `suma`. To je moguće zato što su funkcije u Pajtonu objekti, pa se mogu čuvati u varijabli, prosljeđivati drugim funkcijama ili vraćati kao rezultat izvršavanja funkcije.

Lambda funkcije su naročito korisne kada je potrebno definisati kratku funkciju koja se koristi samo na jednom mjestu u programu. U takvim situacijama često nije potrebno uvoditi funkciju sa punim imenom.

Na primjer, prethodna funkcija može se pozvati direktno:

```
print((lambda x, y: x + y)(4, 3))
```

Iako lambda funkcije mogu imati proizvoljan broj parametara, njihovo tijelo mora sadržavati samo jedan izraz. Zbog toga se uglavnom koriste za jednostavne operacije.

Lambda funkcije se često kombinuju sa ugrađenim funkcijama za obradu kolekcija, kao što su `map()`, `filter()` i `reduce()`. O ovim funkcijama biće više riječi u nastavku knjige, u Sekciji 2.9.

2.9 Neki programski koncepti i često korištene ugrađene funkcije

U ovoj sekciji predstavljamo nekoliko važnih koncepata i ugrađenih funkcija koje se često koriste u programskom jeziku Pajton. Posebna pažnja posvećena je komprehenziji liste (*list comprehension*) i funkcijama `map()`, `filter()` i `reduce()`, koje omogućavaju sažetije i preglednije pisanje programa te uvode određene elemente funkcionalnog programiranja.

2.9.1 Komprehenzija liste

Komprehenzija liste (*list comprehension*) predstavlja koncizan način kreiranja i popunjavanja lista. Omogućava formiranje nove liste u jednoj naredbi, bez potrebe za eksplicitnim korištenjem petlji i metoda za dodavanje elemenata.

Osnovna sintaksa je:

```
lista = [izraz for element in iterabilni_objekat]
```

Pri tome:

- *izraz* definiše vrijednost koja se upisuje u novu listu;
- *element* predstavlja trenutni element iterabilnog objekta;
- *iterabilni_objekat* može biti lista, torka, skup, string, generator ili drugi iterabilni objekat.

Na primjer, listu prvih deset parnih brojeva možemo kreirati na sljedeći način:

```
parni = [2 * i for i in range(1, 11)]
```

Isti rezultat mogao bi se dobiti i pomoću klasične petlje:

2.9. NEKI PROGRAMSKI KONCEPTI I ČESTO KORIŠTENE UGRAĐENE FUNKCIJE

```
parni = []  
for i in range(1, 11):  
    parni.append(2 * i)
```

Komprehenzija liste može sadržavati i uslov:

```
lista = [izraz for element in iterabilni_objekat if uslov]
```

Na primjer, izdvajanje parnih brojeva iz liste:

```
brojevi = [1, 2, 3, 4, 5, 6]  
parni = [x for x in brojevi if x % 2 == 0]  
print(parni) # [2, 4, 6]
```

Komprehenzija liste često rezultira kraćim i preglednijim kôdom, posebno kod jednostavnih transformacija i filtriranja podataka.

2.9.2 Funkcije map, filter i reduce

Funkcije `map()`, `filter()` i `reduce()` predstavljaju osnovne alate funkcionalnog programiranja. Njihova primjena omogućava obradu kolekcija podataka bez eksplicitnog pisanja petlji.

Funkcija `map()` prihvata funkciju i iterabilni objekat te primjenjuje zadatu funkciju na svaki element kolekcije.

```
brojevi = [1, 2, 3, 4, 5]  
dupli = map(lambda x: 2 * x, brojevi)  
print(list(dupli)) # [2, 4, 6, 8, 10]
```

U Pajtonu verzije 3 funkcija `map()` vraća iterator, pa je potrebno rezultat pretvoriti u listu pozivom funkcije `list()`.

Funkcija `filter()` služi za izdvajanje elemenata iterabilne kolekcije koji zadovoljavaju određeni uslov.

```
brojevi = [1, 2, 3, 4, 5, 6]
parni = filter(lambda x: x % 2 == 0, brojevi)
print(list(parni)) # [2, 4, 6]
```

Funkcija koja se prosljeđuje kao prvi argument mora vraćati logičku vrijednost (*True* ili *False*).

Funkcija `reduce()` sukcesivno kombinuje elemente kolekcije primjenom zadate funkcije.

Njena opšta sintaksa je:

```
reduce(funkcija, iterable[, pocetna_vrijednost])
```

Funkcija `reduce()` nije ugrađena u osnovni skup funkcija jezika Pajton, već se nalazi u modulu `functools`.

```
from functools import reduce

brojevi = [1, 2, 3, 4, 5]

proizvod = reduce(
    lambda x, y: x * y,
    brojevi,
    1)

print(proizvod) # 120
```

U ovom primjeru izvršavaju se sljedeće operacije:

$$\begin{aligned} 1 \cdot 1 &= 1, \\ 1 \cdot 2 &= 2, \\ 2 \cdot 3 &= 6, \\ 6 \cdot 4 &= 24, \\ 24 \cdot 5 &= 120. \end{aligned}$$

2.9. NEKI PROGRAMSKI KONCEPTI I ČESTO KORIŠTENE UGRAĐENE FUNKCIJE

Dakle, funkcija `reduce()` “redukuje” čitavu kolekciju na jednu konačnu vrijednost.

U savremenom Pajtonu mnoge operacije realizovane funkcijama `map()` i `filter()` često se zapisuju pomoću komprehenzije liste, jer je takav kôd pregledniji. Na primjer:

```
parni = [x for x in brojevi if x % 2 == 0]
```

predstavlja alternativu izrazu

```
parni = filter(lambda x: x % 2 == 0, brojevi)
```

Izbor između ova dva pristupa najčešće zavisi od stila programiranja i složenosti konkretnog problema.

2.9.3 Funkcije `zip`, `enumerate` i `sort`

Funkcije `zip()` i `enumerate()` spadaju među najčešće korištene ugrađene funkcije za obradu i iteriranje kroz kolekcije podataka.

Funkcija `zip()`

Funkcija `zip()` prima dva ili više iterabilnih objekata i vraća iterator koji generiše torke sastavljene od odgovarajućih elemenata tih objekata.

Njena opšta sintaksa je:

```
zip(iterable1, iterable2, ...)
```

gdje su *iterable1*, *iterable2*, ... proizvoljni iterabilni objekti. Na primjer:

```
lista1 = [1, 2, 3, 4]
lista2 = [5, 6, 7, 8]
parovi = zip(lista1, lista2)
print(list(parovi))
# [(1, 5), (2, 6), (3, 7), (4, 8)]
```

Funkcija `zip()` zaustavlja generisanje parova čim se iscrpi najkraći od prosljeđenih iterabilnih objekata.

Funkcija `enumerate()` omogućava istovremeno pristupanje elementima kolekcije i njihovim indeksima.

Sintaksa funkcije je:

```
enumerate(iterable, start=0)
```

gdje je:

- *iterable* iterabilni objekat;
- *start* početna vrijednost indeksa (podrazumijevano je 0).

```
voce = ["jabuka", "banana", "kruška", "breskva"]
for i, voca in enumerate(voce):
    print(i, voca)
```

Rezultat izvršavanja je:

```
0 jabuka
1 banana
2 kruška
3 breskva
```

Funkcija `enumerate()` često predstavlja pregledniju alternativu ručnom vođenju brojača unutar petlje.

2.9.4 Sortiranje lista

Pajton omogućava sortiranje elemenata pomoću metode `sort()` i ugrađene funkcije `sorted()`.

Metoda `sort()` sortira postojeću listu direktno u memoriji.

Njena sintaksa je:

```
lista.sort(reverse=False, key=None)
```

2.9. NEKI PROGRAMSKI KONCEPTI I ČESTO KORIŠTENE UGRAĐENE FUNKCIJE

Parametar `reverse` određuje redoslijed sortiranja:

- *False* – rastući poredak (podrazumijevano);
- *True* – opadajući poredak.

Parametar `key` omogućava zadavanje funkcije koja određuje kriterijum sortiranja. U nastavku slijedi jedan primjer.

```
brojevi = [6, 3, 8, 3, 1, 7]
brojevi.sort(reverse=True)
print(brojevi) # [8, 7, 6, 3, 3, 1]
```

Ako elementi liste nisu međusobno uporedivi, javlja se greška:

```
TypeError: '<' not supported between
instances of 'str' and 'int'
```

Za razliku od metode `sort()`, funkcija `sorted()` ne mijenja originalnu kolekciju, već vraća novu sortiranu kolekciju.

```
brojevi = [5, 2, 8, 3, 1, 7]
sortirani = sorted(brojevi)
print(sortirani)
# [1, 2, 3, 5, 7, 8]
print(brojevi)
# [5, 2, 8, 3, 1, 7]
```

U praksi se `sort()` koristi kada želimo izmijeniti postojeću listu, dok se `sorted()` koristi kada je potrebno sačuvati originalni redoslijed elemenata.

2.9.5 Korisne ugrađene funkcije

Pored navedenih funkcija, Pajton sadrži veliki broj korisnih ugrađenih funkcija. Neke od najčešće korištenih su:

- `sum(iterable)` – vraća sumu elemenata iterabilnog objekta;
- `max(iterable)` – vraća najveći element;
- `min(iterable)` – vraća najmanji element;
- `abs(x)` – vraća apsolutnu vrijednost broja;
- `round(x, n)` – zaokružuje broj na *n* decimalnih mjesta;
- `len(obj)` – vraća broj elemenata kolekcije;
- `eval(expr)` – izračunava izraz zapisan u obliku stringa.

2.10 Moduli

Modul predstavlja datoteku koja sadrži definicije funkcija, klasa, promjenljivih i drugih programskih elemenata. Modul sadrži jednu ili više datoteka sa ekstenzijom `.py`

Moduli služe organizovanju programa u manje cjeline i ponovnu upotrebu kôda u različitim projektima. Da bismo ga koristili, modul je potrebno prvo uključiti pomoću naredbe `import`.

Na primjer, ako postoji datoteka `mojmodul.py`, uključuje na sljedeći način:

```
import mojmodul
```

Nakon toga funkcijama, klasama i promjenljivama modula pristupamo pomoću notacije tačka:

```
mojmodul.moja_funkcija()
```

Moguće je uvesti i samo određene elemente modula:

```
from mojmodul import moja_funkcija
```

Tada se funkcija može pozivati direktno:

```
moja_funkcija()
```

Moduli predstavljaju jedan od osnovnih mehanizama organizacije programa u Pajtonu i omogućavaju razvoj većih i preglednijih softverskih sistema.

Pajton dolazi sa velikim brojem ugrađenih modula, među kojima se posebno izdvajaju `math`, `random`, `datetime` i `os`. Pored njih, u praksi se često koriste i brojni eksterni moduli i biblioteke, kao što je `numpy`, o kojima će biti više riječi u nastavku ovog poglavlja.

Moduli koji su dio standardne biblioteke dostupni su odmah nakon instalacije Pajtona i nije ih potrebno dodatno instalirati. Sa druge strane, eksterne module je potrebno prethodno instalirati.

Instalacija eksternih modula najčešće se vrši pomoću menadžera paketa `pip`, koji se standardno isporučuje uz većinu Pajton instalacija. Da bismo instalirali određeni modul, na terminalu izvršimo naredbu:

```
pip install naziv_modula
```

gdje `naziv_modula` predstavlja naziv modula koji želimo instalirati.

Na primjer, za instalaciju modula `requests` koristimo naredbu:

```
pip install requests
```

Nakon pokretanja naredbe, `pip` automatski preuzima traženi paket sa repozitorijuma PyPI (eng. *Python Package Index*) i instalira ga u trenutno Pajton okruženje.

PyPI predstavlja centralni repozitorijum Pajton paketa i sadrži stotine hiljada biblioteka koje su slobodno dostupne za korištenje. Zahvaljujući

njemu, funkcionalnost jezika može se značajno proširiti bez potrebe za implementacijom svega od početka.

Nakon uspješne instalacije, modul se može uključiti u program pomoću naredbe `import`:

```
import requests
```

Ponekad je potrebno instalirati tačno određenu verziju modula. To se postiže navođenjem broja verzije iza operatora `==`. Na primjer, za instalaciju verzije 2.2.1 modula `requests` koristimo:

```
pip install requests==2.2.1
```

Informacije o instaliranim modulima mogu se prikazati naredbom:

```
pip list
```

dok se detaljne informacije o konkretnom modulu dobijaju naredbom:

```
pip show requests
```

Savremeni projekti napisani u Pajtonu često koriste virtuelna okruženja (eng. *virtual environments*) kako bi se zavisnosti različitih projekata držale odvojeno. O ovom konceptu biće više riječi u kasnijim poglavljima.

2.10.1 Modul `math`

Modul `math` sadrži veliki broj matematičkih funkcija i konstanti koje se često koriste u naučnom računarstvu, obradi podataka i razvoju algoritama.

Prije korištenja potrebno ga je uključiti naredbom:

```
import math
```

Nakon toga funkcijama i konstantama modula pristupa se preko prefiksa `math`.

```
import math
# Kvadratni korijen
x = math.sqrt(25)
print(x)          # 5.0

# Konstanta pi
print(math.pi)
# Sinus ugla od 45 stepeni
print(math.sin(math.radians(45)))
```

Napomenimo da trigonometrijske funkcije modula `math` očekuju argument izražen u radijanima. Zbog toga se za pretvaranje stepeni u radijane često koristi funkcija `math.radians()`.

Neke od najčešće korištenih funkcija modula `math` su:

- `sqrt(x)` – kvadratni korijen broja;
- `pow(x,y)` – stepenovanje;
- `log(x)` – prirodni logaritam;
- `log10(x)` – dekadni logaritam;
- `exp(x)` – vrijednost e^x ;
- `floor(x)` – zaokruživanje naniže;
- `ceil(x)` – zaokruživanje naviše;
- `fabs(x)` – apsolutna vrijednost;
- `gcd(a,b)` – najveći zajednički djelilac;
- `dist(p,q)` – Euklidovo rastojanje između dvije tačke;
- `sin(x)`, `cos(x)`, `tan(x)` – trigonometrijske funkcije.

Pored funkcija, modul sadrži i korisne matematičke konstante kao što su `math.pi` i `math.e`.

2.10.2 Modul random

Modul `random` omogućava generisanje pseudo-slučajnih brojeva i slučajan izbor elemenata iz kolekcija podataka.

Uključuje se naredbom:

```
import random
```

Neke od najčešće korištenih funkcija su:

- `random()` – generiše realan broj iz intervala $[0, 1)$;
- `randint(a,b)` – generiše slučajan cijeli broj između a i b (uključujući granice);
- `choice(seq)` – vraća slučajno odabrani element kolekcije;
- `shuffle(seq)` – nasumično permutuje elemente liste;
- `sample(population,k)` – bira k različitih elemenata bez ponavljanja;
- `seed(x)` – postavlja početno stanje generatora slučajnih brojeva.

Primjer korištenja:

```
import random

print(random.random())
print(random.randint(1, 10))

imena = ["Ana", "Marko", "Petar"]
print(random.choice(imena))
```

Generator slučajnih brojeva u modulu `random` zasniva se na determinističkom algoritmu. To znači da se za isto početno stanje, zadato funkcijom `seed()`, uvijek generiše isti niz vrijednosti. Ova osobina je posebno korisna pri testiranju programa i reprodukciji eksperimenata.

2.10.3 Modul time

Modul `time` predstavlja dio standardne Pajton biblioteke i omogućava rad sa vremenom. Njegove funkcije koriste se za mjerenje trajanja izvršavanja programa, dobijanje trenutnog vremena, privremeno zaustavljanje izvršavanja programa i konverziju između različitih vremenskih formata.

Modul se uključuje naredbom:

```
import time
```

Neke od najčešće korištenih funkcija su:

- `time()` – vraća trenutno vrijeme izraženo kao broj sekundi proteklih od 1. januara 1970. godine (tzv. Unix epoha);
- `sleep(secs)` – privremeno zaustavlja izvršavanje programa na zadati broj sekundi;
- `localtime([secs])` – pretvara vrijeme izraženo u sekundama od epohe u lokalni datum i vrijeme;
- `ctime([secs])` – vraća tekstualni prikaz datuma i vremena;
- `perf_counter()` – vraća precizan brojač vremena koji se često koristi za mjerenje vremena izvršavanja algoritama.

Primjer:

```
import time
print(time.time())
time.sleep(2)
print("Program nastavljen.")
```

2.10.4 Modul os

Modul `os` omogućava interakciju sa operativnim sistemom. Njegove funkcije koriste se za rad sa datotekama i direktorijumima, pristup varijablama okruženja i izvršavanje određenih sistemskih operacija.

Modul se uključuje naredbom:

```
import os
```

Neke od najčešće korištenih funkcija i atributa su:

- `os.name` – vraća oznaku operativnog sistema;
- `os.getcwd()` – vraća trenutni radni direktorijum;
- `os.chdir(path)` – mijenja trenutni radni direktorijum;
- `os.listdir(path)` – vraća listu datoteka i direktorijuma;
- `os.mkdir(path)` – kreira novi direktorijum;
- `os.makedirs(path)` – kreira direktorijum zajedno sa svim potrebnim poddirektorijumima;
- `os.remove(path)` – briše datoteku;
- `os.rmdir(path)` – briše prazan direktorijum;
- `os.rename(src, dest)` – mijenja naziv datoteke ili direktorijuma;
- `os.path.exists(path)` – provjerava da li putanja postoji;
- `os.path.isfile(path)` – provjerava da li putanja predstavlja datoteku;
- `os.path.isdir(path)` – provjerava da li putanja predstavlja direktorijum.

Primjer:

```
import os

print(os.getcwd())
if os.path.exists("podaci.txt"):
    print("Datoteka postoji.")
```

2.10.5 Modul copy

Modul `copy` omogućava pravljenje kopija objekata. Posebno je koristan pri radu sa složenim strukturama podataka, kao što su liste, rječnici i ugniježdene kolekcije. Ovaj modul podržava dvije osnovne vrste kopiranja:

- **površinsko kopiranje** (*shallow copy*);
- **dubinsko kopiranje** (*deep copy*).

Modul se uključuje naredbom:

```
import copy
```

Površinsko kopiranje

Funkcija `copy()` kreira novi objekat, ali elementi koji se nalaze unutar njega ostaju zajednički sa originalnim objektom.

```
import copy

originalna_lista = [1, [2, 3], 4]
kopija = copy.copy(originalna_lista)
kopija[1][0] = 99
print(originalna_lista) # [1, [99, 3], 4]
```

Vidimo da je izmjena u unutrašnjoj listi vidljiva i u originalnom objektu.

Dubinsko kopiranje

Funkcija `deepcopy()` rekurzivno kopira sve objekte sadržane unutar originalnog objekta, čime nastaje potpuno nezavisna kopija. Naziva se još kopiranje po vrijednosti.

```
import copy
```

```
originalna_lista = [1, [2, 3], 4]
kopija = copy.deepcopy(originalna_lista)
kopija[1][0] = 99
print(originalna_lista) # [1, [2, 3], 4]
```

U ovom slučaju promjene napravljene nad kopijom ne utiču na originalni objekat.

Površinsko kopiranje je brže i troši manje memorije, dok dubinsko kopiranje zahtijeva više vremena i memorijskog prostora, ali garantuje potpunu nezavisnost kopije od originalnog objekta.

2.11 Izuzeci

Tokom izvršavanja programa mogu se pojaviti različite greške koje nije moguće otkriti u fazi prevođenja programa. Takve greške nazivaju se *izuzeci* (eng. *exceptions*). Tipični primjeri izuzetaka su:

- dijeljenje brojem nula;
- pristup nepostojećoj datoteci;
- pokušaj konverzije neispravnog podatka;
- pristup elementu izvan granica kolekcije;
- korištenje objekta neodgovarajućeg tipa.

Mehanizam izuzetaka omogućava programu da prepozna nastalu grešku i na nju reaguje na kontrolisan način, umjesto da se izvršavanje trenutno prekine.

2.11.1 Blok try-except

Osnovni mehanizam za obradu izuzetaka u Pajtonu zasniva se na konstrukciji `try-except`. Kôd koji potencijalno može izazvati izuzetak smješta se u blok `try`, dok se obrada greške definiše u odgovarajućem bloku `except`.

Opšti oblik ove konstrukcije je:

```
try:
    # kod koji može izazvati izuzetak
except ExceptionType:
    # obrada izuzetka
```

Na primjer:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Dijeljenje nulom nije dozvoljeno.")
```

U ovom slučaju program neće biti momentalno prekinut, već će se izvršiti odgovarajuća poruka o grešci.

Jedan blok `try` može imati više blokova `except`, pri čemu svaki obrađuje određenu vrstu greške.

```
try:
    # kod koji može izazvati izuzetak
except ValueError:
    print("Greška u vrijednosti.")
except TypeError:
    print("Greška u tipu podatka.")
except Exception:
    print("Dogodio se neočekivani izuzetak.")
```

Klasa `Exception` predstavlja baznu klasu većine izuzetaka i često se koristi za hvatanje svih neobrađenih grešaka.

Pored hvatanja postojećih izuzetaka, moguće je i eksplicitno generisati izuzetak pomoću naredbe `raise`.

```
raise ValueError("Neispravan ulazni podatak.")
```

Ovakav pristup je koristan kada želimo da sami signaliziramo pojavu greške ili neispravnog stanja u programu.

Ponekad je potrebno izvršiti određeni kod bez obzira na to da li je došlo do izuzetka ili ne. Za tu svrhu koristi se blok `finally`.

```
try:
    # kod koji može izazvati izuzetak
finally:
    # kod koji se uvijek izvršava
```

Blok `finally` često se koristi za zatvaranje datoteka, oslobađanje resursa ili izvršavanje operacija čišćenja nakon završetka rada programa.

2.11.2 Naredba `assert`

Još jedan koristan mehanizam predstavlja naredba `assert`, koja služi za provjeru da li određeni uslov važi. Njena sintaksa je:

```
assert uslov, "poruka o greški"
```

Ako je uslov tačan, program nastavlja izvršavanje. U suprotnom se generiše izuzetak tipa `AssertionError`.

Primjer.

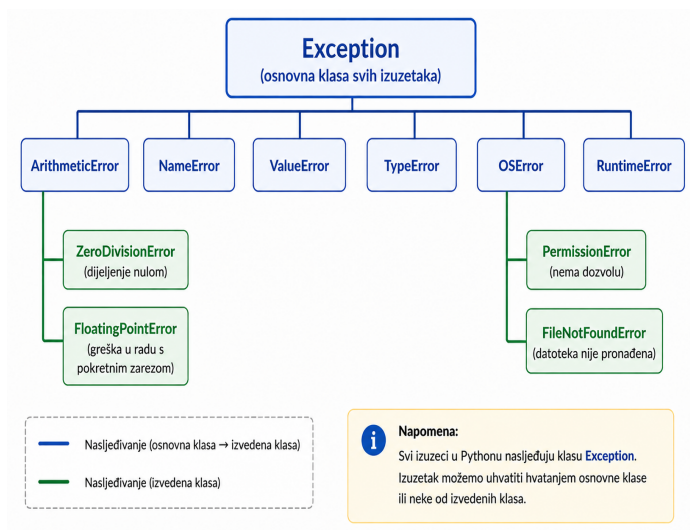
```
x = 5
assert x > 0, "Vrijednost x mora biti pozitivna."
```

Naredba `assert` posebno je korisna tokom razvoja i testiranja programa, kada želimo provjeriti da li određene pretpostavke važe u toku izvršavanja.

2.11.3 Hijerarhija izuzetaka

Sve klase izuzetaka u Pajtonu organizovane su hijerarhijski. Na vrhu hijerarhije nalazi se klasa `BaseException`, iz koje nasljeđuju sve ostale klase izuzetaka.

Pregled najvažnijih klasa izuzetaka prikazan je na Slici 2.10.



Slika 2.10: Hijerarhija najvažnijih klasa izuzetaka u Pajtonu.

Pravilna upotreba izuzetaka predstavlja važan dio razvoja pouzdanih programa. Obradom izuzetaka moguće je spriječiti neočekivani prekid izvršavanja, korisniku pružiti jasne informacije o nastaloj grešci i povećati robusnost softvera.

2.12 Rad sa datotekama

Pomoću svojih ugrađenih funkcija, Pajton omogućuje jednostavan rad sa ulaznim i izlaznim datotekama. U toku rada, ovaj modul interaguje sa

operativnim sistemom. U nastavku dajemo pregled najčešće korištenih metoda.

Čitanje iz datoteke. Koristi se funkcija `open`, koja prihvata dva argumenta – naziv datoteke (ili putanja do datoteke) i način u kojem se otvara datoteka (npr. za čitanje, pisanje, dodavanje, itd.). Pogledajmo sljedeći primjer:

```
# Otvaranje fajla za čitanje
file = open("datoteka.txt", "r")
# Čitanje sadržaja iz fajla
sadrzaj = file.read()
# Zatvaranje fajla
file.close()
# Ispisivanje sadržaja fajla
print(sadrzaj)
```

Dakle, prvo otvaramo datoteku `datoteka.txt` samo za čitanje. Zatim čitamo sadržaj datoteke koristeći metodu `read` i čuvamo ga u varijabli *sadrzaj*. Konačno, zatvaramo datoteku pomoću metode `close` i ispisujemo sadržaj datoteke.

Pisanje u fajl. Za pisanje u datoteku koristimo funkciju `open`, koja otvara fajl sa određenim nazivom, ali ovaj put sa načinom “w” (eng. `write`), čime se dopušta pisanje u fajl. Pogledajmo sljedeći primjer.

```
# Otvaranje fajla za pisanje
file = open("datoteka.txt", "w")
# Pisanje teksta u datoteku
file.write("Zdravo!")
# Zatvaranje fajla
file.close()
```

Dodavanje sadržaja u fajl. Da biste dodali sadržaj u postojeći sadržaj datoteke, otvaramo fajl sa načinom “a” (eng. *append*), a potom pomoću funkcije `write` upisujemo novi sadržaj na postojeći.

Napomenimo da je dobra praksa zatvoriti datoteku nakon što završite rad sa njom, kako bismo izbjegli bilo kakve potencijalne probleme sa

oštećenjem datoteke ili gubitkom podataka. Praksa je da se rad u fajlovima odvija pod `try-except` blokom. Klase izuzetaka koje se u tim slučajevima mogu koristiti su: `FileNotFoundError` i `OSError`.

2.12.1 Modul json

Modul `json` se koristi za enkodiranje i dekodiranje podataka sa i u JSON (eng. *Java Script Object Notation*) formatu. JSON je široko rasprostranjen format za razmjenu podataka, osobama jednostavan za čitanje i pisanje, a mašinama za raščlanjivanje i generisanje.

Modul `json` nudi dvije glavne metode za rad sa JSON podacima:

- `json.dump()` – koristi se za enkodiranje pajton objekata u JSON format i njegovo spremanje u objekat datoteka.
- `json.loads()` – koristi se za dekodiranje JSON niza u strukturu podataka (obično heš mapa).

Pogledajmo i jedan primjer upotrebe `json` modula.

```
import json

# Pajton objekat
objekat = {'ime': 'Mirko', 'godine': 30, 'grad': 'Banja Luka'}
# Enkoduj objekat u JSON string
json_string = json.dumps(objekat)
# Dekodiraj JSON string nazad u pajtonov objekat (dict)
dekodiran_objekat = json.loads(json_string)

print(type(json_string))    # <class 'str'>
print(type(dekodiran_objekat)) # <class 'dict'>
print(dekodiran_objekat)    # {'ime': 'Mirko', 'godine': 30,
                             # 'grad': 'Banja Luka'}
```

Primijetite da funkcija *dumps()* pretvara pajtonov objekat u string (kodiranje), dok funkcija *loads()* radi obrnutu akciju, od stringa pravi pajtonov objekat (enkodiranje).

2.13 Regularni izrazi

Regularni izraz (eng. *regular expression*, skraćeno *regex*) predstavlja šablon za opisivanje i pretraživanje tekstualnih nizova. Drugim riječima, regularni izraz definiše skup stringova koji zadovoljavaju određena sintakсна pravila, opisujući skup riječi koje se uklapaju u isti šablon.

Osnovna svrha regularnih izraza jeste da se veliki skup stringova opiše na koncizan način, bez potrebe za njihovim pojedinačnim nabranjanjem. Na primjer, regularnim izrazom možemo opisati skup svih binarnih nizova ili skup riječi {gray, grey} koristeći samo jedan obrazac.

Regularni izrazi imaju široku primjenu u obradi teksta, pretraživanju podataka, validaciji korisničkog unosa, uređivačima teksta i alatima kao što je *grep* u operativnom sistemu Unix.

2.13.1 Osnovni koncepti regularnih izraza

- **Alternacija**

Alternacija se označava simbolom `|` i predstavlja logičko “ili”.

Na primjer, `a|b|c` opisuje skup stringova {*a*, *b*, *c*}.

- **Grupisanje**

Zagrade služe za grupisanje dijelova izraza i određivanje područja djelovanja operatora.

Na primjer, `gr(a|e)y` opisuje riječi `gray`, `grey`.

- **Specijalni simboli i klase karaktera**

- `.` – odgovara proizvoljnom karakteru;
- `[abc]` – odgovara jednom od navedenih karaktera;

- $[a-z]$ – odgovara jednom malom slovu engleske abecede;
 - $[A-Z]$ – odgovara jednom velikom slovu;
 - $[a-zA-Z]$ – odgovara jednom slovu, bez obzira na veličinu;
 - $\backslash d$ – odgovara jednoj cifri (ekvivalentno izrazu $[0-9]$);
 - $[\sim abc]$ – odgovara bilo kojem karakteru osim navedenih;
 - $\backslash w$ – odgovara slovu, cifri ili znaku “_”;
 - $\backslash s$ – odgovara praznom znaku (razmak, tabulator, novi red);
 - $\backslash n$ – odgovara znaku novog reda;
 - $\backslash t$ – odgovara tabulatoru;
 - $\backslash b$ – odgovara granici riječi.
- **Kvantifikatori.** Kvantifikatori određuju koliko puta se izraz može pojaviti koji im neposredno prethodi.
- $?$ – prethodni izraz pojavljuje se 0 ili 1 put;
 - $*$ – prethodni izraz pojavljuje se 0 ili više puta;
 - $+$ – prethodni izraz pojavljuje se 1 ili više puta;
 - $\{m, n\}$ – prethodni izraz pojavljuje se najmanje m , a najviše n puta.

U nastavku slijedi nekoliko primjera regularnih izraza.

Registarske tablice. Pretpostavimo da registarska oznaka ima oblik: A12-B-345 tj. jedno veliko slovo, dvije cifre, povlaku, jedno veliko slovo, novu povlaku i tri cifre.

Odgovarajući regularni izraz je: $[A-Z]\backslash d\{2\}-[A-Z]-\backslash d\{3\}$

Datum u formatu DD.MM.GGGG. Ako se dan i mjesec uvijek zapisuju sa dvije cifre (npr. 01.02.2025.), odgovarajući regularni izraz je: `(\d{2})\.(\d{2})\.(\d{4})\.`

Ovaj izraz opisuje dvije cifre za dan, dvije cifre za mjesec i četiri cifre za godinu, razdvojene tačkama.

Regularni izrazi predstavljaju veoma moćan alat za obradu tekstualnih podataka. Iako njihova sintaksa na prvi pogled može djelovati složeno, pravilnom kombinacijom osnovnih operatora moguće je veoma precizno opisati i prepoznati složene obrasce u tekstu.

2.13.2 Modul `re`

Modul `re` je ugrađeni Pajtonov modul namijenjen radu sa regularnim izrazima. Omogućava definisanje, pretraživanje i obradu tekstualnih obrazaca (eng. *pattern*), pa predstavlja jedan od najvažnijih alata za manipulaciju tekstualnim podacima.

Da bismo koristili funkcionalnosti ovog modula, potrebno ga je prvo uključiti naredbom:

```
import re
```

Neke od najčešće korištenih funkcija modula `re` su:

- `search(pattern, string)`

Pretražuje string `string` i pronalazi prvo pojavljivanje obrasca `pattern`. Ukoliko se obrazac pronađe, funkcija vraća objekat podudaranja; u suprotnom vraća vrijednost `None`.

- `match(pattern, string)`

Provjerava da li se obrazac `pattern` nalazi na samom početku stringa. Ako postoji podudaranje, vraća odgovarajući objekat podudaranja.

- `sub(pattern, replacement, string)`

Zamjenjuje sva pojavljivanja obrasca `pattern` u stringu `string` novim tekstom `replacement`.

- `split(pattern, string)`

Razdvaja string na dijelove koristeći obrazac `pattern` kao separator i vraća listu dobijenih podstringova.

- `findall(pattern, string)`

Pronalazi sva podudaranja obrasca u stringu i vraća ih u obliku liste.

U nastavku je prikazan jednostavan primjer korištenja funkcije `search()`.

```
import re

# Definisanje obrasca
pattern = r'\b\w+'

# Tekst koji pretražujemo
text = 'Hello, World!'

# Pretraga
match = re.search(pattern, text)

# Ispis prvog pronađenog podudaranja
print(match.group())
```

Izlaz programa je:

```
Hello
```

Posmatrajmo detaljnije regularni izraz

```
r'\b\w+'
```

Njegove komponente imaju sljedeće značenje:

- `\b` označava granicu riječi;

- `\w` označava slovo, cifru ili znak `_`;
- `+` označava jedno ili više uzastopnih pojavljivanja prethodnog obrasca.

Dakle, izraz `\b\w+` odgovara prvoj riječi u tekstu, odnosno prvom nizu uzastopnih alfanumeričkih znakova koji počinje na granici riječi.

Napomenimo da se u primjeru koristi zapis

```
r'...'
```

koji predstavlja tzv. ravni string (eng. *raw string*). Kod ovakvog zapisa znak `\` se ne tretira kao početak specijalne sekvence, što pojednostavljuje pisanje regularnih izraza i smanjuje mogućnost greške.

2.14 Moduli `sys` i `getopt`

Pokretanje programa iz komandne linije

Pajton skripta se može pokrenuti iz terminala naredbom:

```
python myscript.py arg1 arg2
```

ili, na sistemima gdje je instalirano više verzija Pajtona,

```
python3 myscript.py arg1 arg2
```

Pri tome je:

- `myscript.py` naziv skripte;
- `arg1`, `arg2`, ... argumenti komandne linije.

Modul sys

Ovaj modul omogućava pristup informacijama i funkcijama vezanim za Pajtonov interpreter i izvršno okruženje. Neki od najčešće korištenih funkcija i atributa ovog modula su dati u nastavku.

- `sys.argv`: Lista argumenata komandne linije. Element na poziciji 0 sadrži naziv skripte, dok ostali elementi predstavljaju argumente koje je korisnik proslijedio programu.
- `sys.path`: Lista direktorijuma u kojima Pajton traži module za učitavanje.
- `sys.exit([kod])`: Prekida izvršavanje programa i vraća izlazni kod operativnom sistemu. Ukoliko se kod ne navede, podrazumijeva se vrijednost 0.
- `sys.version`: Vraća informaciju o verziji Pajtonovog interpretera.
- `sys.platform`: Vraća oznaku platforme na kojoj se program izvršava.

Sljedeći primjer prikazuje korištenje atributa `argv`.

```
import sys

if len(sys.argv) > 1:
    print("Argumenti komandne linije su:")

    for arg in sys.argv[1:]:
        print(arg)

else:
    print("Nema proslijeđenih argumenata.")
```

Ako program pokrenemo naredbom

```
python test.py arg1 arg2
```

dobijamo izlaz

```
Argumenti komandne linije su:
arg1
arg2
```

Modul getopt . Ovaj modul služi za parsiranje opcija i argumenata komandne linije. Njegova funkcionalnost slična je istoimenoj funkciji dostupnoj u Unix operativnim sistemima. Najvažnija funkcija modula je:

```
getopt.getopt(args, shortopts, longopts)
```

gdje su:

- **args** lista argumenata komandne linije;
- **shortopts** string koji definiše kratke opcije;
- **longopts** lista dugih opcija.

Funkcija vraća torku:

- listu pronađenih parova (**opcija**, **vrijednost**);
- listu preostalih argumenata.

Na primjer, zapis

```
"hi:o:"
```

definiše tri kratke opcije:

- **-h** ne zahtijeva dodatnu vrijednost;
- **-i** zahtijeva vrijednost;
- **-o** zahtijeva vrijednost.

Dvotačka nakon oznake opcije označava da opcija mora imati pridruženu vrijednost.

Za duge opcije koristi se lista stringova:

```
["ifile=", "ofile="]
```

Znak = na kraju naziva opcije označava da opcija zahtijeva argument.

Primjer

```
import getopt
import sys

def main(argv):

    inputfile = ""
    outputfile = ""

    try:
        opts, args = getopt.getopt(
            argv,
            "hi:o:", ["ifile=", "ofile="]
        )

    except getopt.GetoptError:
        print("test.py -i <inputfile> -o <outputfile>")
        sys.exit(2)

    for opt, arg in opts:

        if opt == "-h":
            print("test.py -i <inputfile> -o <outputfile>")
            sys.exit()

        elif opt in ("-i", "--ifile"):
            inputfile = arg

        elif opt in ("-o", "--ofile"):
            outputfile = arg

    if __name__ == "__main__":
        main(sys.argv[1:])
```

Program se može pokrenuti naredbom

```
python3 test.py -i ulaz.txt -o izlaz.txt
```

U slučaju da korisnik navede neispravne opcije ili izostavi obavezne argumente, aktivira se izuzetak `GetoptError`, koji se obrađuje u bloku `try-except`.

Napomena. U savremenim Pajton programima modul `argparse` uglavnom predstavlja preporučeni način za obradu argumenata komandne linije, jer nudi jednostavniju sintaksu, automatsko generisanje pomoći (`-h`) i bolju podršku za složenije aplikacije. Modul `getopt` je i dalje koristan za razumijevanje osnova parsiranja opcija i kompatibilnost sa Unix stilom komandne linije.

2.15 Modul numpy

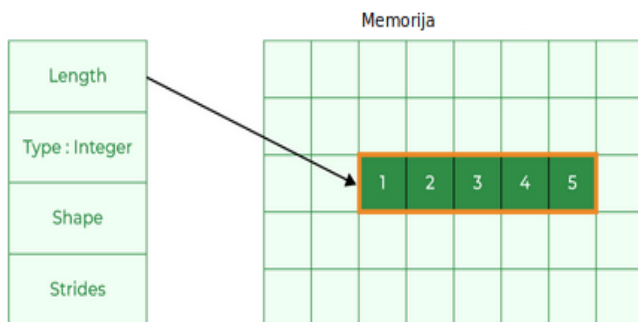
`NumPy` (skraćeno od *Numerical Python*) predstavlja jednu od najvažnijih biblioteka za naučno i numeričko računarstvo u programskom jeziku Pajton. Razvio ju je Travis Oliphant 2005. godine, a značajan dio implementacije napisan je u jezicima *C* i *C++*.

Osnovna struktura podataka biblioteke `NumPy` je višedimenzionalni niz (`ndarray`), koji omogućava efikasan rad sa vektorima, matricama i višedimenzionalnim tenzorima.

Za razliku od Pajtonovih lista:

- svi elementi `NumPy` niza moraju biti istog tipa;
- podaci se čuvaju u neprekidnom bloku memorije;
- aritmetičke operacije nad nizovima izvršavaju se veoma efikasno;
- zauzeće memorije je manje nego kod odgovarajućih lista.

Zbog toga su NumPy nizovi posebno pogodni za obradu velikih količina numeričkih podataka.



Slika 2.11: Pojednostavljen prikaz memorijske organizacije NumPy niza.

Biblioteka se najčešće uključuje sljedećom naredbom:

```
import numpy as np
```

Primjeri kreiranja jednodimenzionalnih i višedimenzionalnih nizova:

```
import numpy as np
```

```
# Jednodimenzionalni niz
```

```
arr = np.array([2, 4, 1, 10], dtype=np.int32)
```

```
# Matrica (2D niz)
```

```
matrica = np.array(  
    [[2, 5],  
     [1, 7]],  
    dtype=np.int32
```

```

)

# Trodimenzionalni niz

matrica3D = np.array(
[
[[2, 5], [1, 7]],
[[2, 3], [5, 5]]
],
dtype=np.int32
)

```

Argument `dtype` određuje tip podataka koji se koristi za skladištenje elemenata. Neki od najčešćih tipova su:

Tip	Značenje
<code>int32, int64</code>	cijeli brojevi
<code>float32, float64</code>	realni brojevi
<code>bool</code>	logičke vrijednosti
<code>complex128</code>	kompleksni brojevi
<code>str</code>	stringovi

Osnovne operacije

- *Pristup elementima:*

```

arr[2]           # element 1D niza
matrica[1, 0]    # element matrice

```

- *Dimenzije niza:*

```

arr.shape

```

- *Rezanje (slicing):*

```
arr[1:5]  
arr[:,2]
```

- *Kopiranje objekta:*

```
kopija = arr.copy()
```

- *Promjena oblika:*

```
arr.reshape((2, 2))
```

Broj elemenata mora ostati nepromijenjen.

- *Spajanje nizova:*

```
np.concatenate((arr1, arr2), axis=0)
```

- *Dijeljenje niza:*

```
np.array_split(arr, 3)
```

- *Pretraga elemenata:*

```
np.where(arr % 2 == 0)
```

Vraća indekse elemenata koji zadovoljavaju dati uslov.

- *Sortiranje:*

```
np.sort(arr)
```

- *Filtriranje:*

```
arr = np.array([41, 42, 43, 44])
```

```
filter_arr = arr > 42  
novi_arr = arr[filter_arr]
```

Rezultat je niz:

```
array([43, 44])
```

- *Kreiranje specijalnih nizova:*

```
np.zeros((3, 4))  
np.ones((2, 5))
```

Jedna od najvećih prednosti biblioteke NumPy jeste mogućnost izvođenja operacija nad cijelim nizovima bez eksplicitnog korištenja petlji.

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
c = a + b
```

Rezultat je

```
array([5, 7, 9])
```

Na isti način moguće je izvršavati množenje, dijeljenje, poređenja i brojne matematičke operacije nad kompletnim nizovima.

Memorijska organizacija. Objekti biblioteke NumPy implementirani su klasom `ndarray`. Pored samih podataka, svaki objekat sadrži i određene metapodatke:

- `dtype` – tip elemenata niza;
- `ndim` – broj dimenzija;
- `shape` – dimenzije niza;
- `strides` – broj bajtova koji je potrebno preskočiti da bi se došlo do sljedećeg elementa u određenoj dimenziji;
- `data` – pokazivač na memorijsku oblast u kojoj se nalaze elementi niza.

Pošto su elementi smješteni u neprekidnom bloku memorije i imaju isti tip, pristup podacima i izvođenje numeričkih operacija znatno su efikasniji nego kod standardnih Pajtonovih lista.

2.16 Modul ctypes

Modul `ctypes` omogućava pozivanje funkcija i korištenje tipova podataka definisanih u dijeljenim bibliotekama (eng. *shared libraries*) napisanim u jeziku *C* ili drugim jezicima nižeg nivoa. Na taj način moguće je koristiti postojeći kôd napisan u jeziku *C* direktno iz Pajtona, bez potrebe za ponovnom implementacijom.

Osnovni koraci za korištenje modula `ctypes` su sljedeći:

1. Napisati program ili biblioteku u jeziku *C*.
2. Prevesti (*kompajlirati*) program u dijeljenu biblioteku.
Na Linux i macOS sistemima to se obično radi pomoću kompajlera `gcc`, dok se na Windows sistemima mogu koristiti Visual Studio ili MinGW.
3. Učitati biblioteku u Pajton pomoću modula `ctypes`.
4. Definirati tipove argumenata i povratne tipove funkcija koje želimo pozivati.
5. Pozvati funkcije iz Pajtona kao da su obične Pajtonove funkcije.

Pretpostavimo da želimo koristiti jednostavnu funkciju za sabiranje napisanu u jeziku *C*.

```
// datoteka dodaj.c

int saberi(int a, int b)
{
    return a + b;
}
```

Na Linux sistemu biblioteku možemo kompajlirati naredbom:

```
gcc -shared -fPIC -o libdodaj.so dodaj.c
```

Nakon toga biblioteku učitavamo u Pajton:

```
import ctypes

lib_dodaj = ctypes.CDLL("./libdodaj.so")
```

Potom definišemo tipove argumenata i povratni tip funkcije:

```
saberi = lib_dodaj.saberi

saberi.argtypes = [ctypes.c_int, ctypes.c_int]
saberi.restype = ctypes.c_int
```

Konačno, funkciju možemo pozvati na uobičajen način:

```
rezultat = saberi(2, 3)

print(rezultat)    # Output: 5
```

Modul `ctypes` automatski vrši konverziju između Pajtonovih i *C* tipova podataka, što značajno pojednostavljuje komunikaciju između ova dva jezika.

2.17 Pisanje korisničkih modula

Jedna od prednosti programskog jezika Pajton jeste mogućnost organizovanja koda u module. Modul predstavlja datoteku sa ekstenzijom `.py` koja sadrži funkcije, klase, konstante ili druge definicije koje se mogu koristiti u više programa.

Na primjer, pretpostavimo da želimo napraviti modul za osnovne matematičke operacije. Kreirajmo datoteku `mathOps.py` sa sljedećim sadržajem:

```
def add(x, y):  
    return x + y  
  
def subtract(x, y):  
    return x - y  
  
def multiply(x, y):  
    return x * y  
  
def divide(x, y):  
    if y == 0:  
        raise ValueError("Dijeljenje nulom nije dozvoljeno.")  
    return x / y
```

Ovaj modul definiše četiri funkcije:

- `add` – sabiranje dva broja;
- `subtract` – oduzimanje dva broja;
- `multiply` – množenje dva broja;
- `divide` – dijeljenje dva broja.

Nakon što je modul sačuvan, može se koristiti u drugoj Pajton skripti pomoću naredbe `import`:

```
import mathOps  
  
print(mathOps.add(3, 4))  
# Output: 7  
print(mathOps.divide(10, 2))  
# Output: 5.0
```

Ako pokušamo izvršiti dijeljenje nulom:

```
mathOps.divide(10, 0)
```

aktiviraće se izuzetak:

```
ValueError: Dijeljenje nulom nije dozvoljeno.
```

Korištenje modula omogućava bolju organizaciju programa, veću ponovnu upotrebljivost koda i jednostavnije održavanje većih projekata. U praksi se složeni programi gotovo uvijek organizuju kao skup međusobno povezanih modula.

2.18 Postavljanje korisnički kreiranih modula na PyPI repozitorij

PyPI (*Python Package Index*) predstavlja centralni repozitorijum paketa napisanih u programskom jeziku Pajton. Objavljivanjem paketa na PyPI-u omogućava se njegovo jednostavno distribuiranje i instaliranje od strane drugih korisnika.

Da bismo objavili vlastiti paket na PyPI-u, potrebno je izvršiti nekoliko koraka.

Prije svega, potrebno je kreirati korisnički nalog na zvaničnoj PyPI stranici <https://pypi.org/account/register/>.

Nakon registracije preporučuje se kreiranje API tokena koji služi za sigurnu autentifikaciju prilikom postavljanja paketa na PyPI.

Priprema paketa. Pretpostavimo da je napisan kôd u okviru paketa pod nazivom `modul1`. U korijenskom direktorijumu projekta kreiramo datoteku `pyproject.toml` koja sadrži osnovne metapodatke o paketu:

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "modul1"
```

2.18. POSTAVLJANJE KORISNIČKI KREIRANIH MODULA NA PYPI REPOZITORIJI

```
version = "0.0.1"
authors = [
    {name = "Mirko Mirković"}
]
description = "Primjer korisničkog modula"
dependencies = [
    "numpy",
    "pandas"
]
```

Ova datoteka definiše naziv paketa, verziju, autora, opis i zavisnosti koje će biti automatski instalirane zajedno sa paketom.

Instalacija potrebnih alata

Za izgradnju i objavljivanje paketa potrebno je instalirati nekoliko pomoćnih alata:

```
pip install build twine
```

Paket `build` služi za kreiranje distribucionih datoteka, dok `twine` omogućava njihovo postavljanje na PyPI.

Kreiranje distribucije. Distribucione datoteke kreiraju se naredbom:

```
python -m build
```

Nakon uspješnog izvršavanja ove naredbe kreira se direktorijum `dist` koji sadrži:

- izvornu distribuciju (`.tar.gz`);
- binarnu distribuciju (`.whl`).

Prije objavljivanja modula preporučuje se provjera ispravnosti generisanih datoteka:

```
twine check dist/*
```

Paket se zatim objavljuje naredbom:

```
twine upload dist/*
```

Prilikom objavljivanja koristi se korisničko ime `--token` i prethodno generisani PyPI API token.

Nakon uspješnog postavljanja paket postaje dostupan svim korisnicima putem PyPI repozitorijuma.

Objavljeni paket može se instalirati standardnom `pip` naredbom:

```
pip install modul1
```

Nakon instalacije paket se može koristiti u bilo kojem programu napisanim u Pajtonu:

```
import modul1
```

Objavljivanje paketa na PyPI-u predstavlja standardni način distribucije Pajton softvera i omogućava jednostavno dijeljenje biblioteka sa širom programerskom zajednicom.

2.19 Kompajliranje i izvršavanje programa

U ovom odjeljku ukratko opisujemo način na koji se izvršava program napisan u programskom jeziku Pajton, kao i osnovne principe organizacije memorije tokom rada programa.

2.19.1 Stek i hip memorija

Tokom izvršavanja programa operativni sistem i Pajtonov interpreter koriste različite dijelove memorije. Među njima posebno su važni *stek* (eng. *stack*) i *hip*.

Stek predstavlja memorijski prostor koji se koristi za čuvanje informacija o aktivnim pozivima funkcija. Svaki put kada se pozove funkcija, na

steku se kreira novi zapis koji sadrži informacije potrebne za izvršavanje te funkcije.

Kada funkcija završi rad, njen zapis se uklanja sa steka. Zbog toga kažemo da je stek privremena memorijska struktura čiji je životni vijek vezan za trajanje poziva funkcije.

Posmatrajmo sljedeći primjer:

```
def func():  
    a = 10  
    b = []  
    c = "abc"
```

Prilikom poziva funkcije `func` kreira se novi zapis na steku. U njemu se čuvaju reference na objekte označene imenima `a`, `b` i `c`. Nakon završetka funkcije ove reference se uklanjaju iz memorije steka.

Međutim, stvarni objekti na koje reference pokazuju uglavnom se nalaze u drugom dijelu memorije koji se naziva *hip*.

Hip predstavlja dinamički dio memorije namijenjen skladištenju objekata čija veličina može biti poznata tek tokom izvršavanja programa. Gotovo svi Pajton objekti, uključujući liste, rječnike, skupove, torke, stringove i numeričke vrijednosti, smještaju se u hip memoriju.

Na primjer:

```
a = [0] * 10
```

U ovom slučaju lista od deset elemenata kreira se u hip memoriji, dok varijabla `a` predstavlja referencu na taj objekat.

Za razliku od jezika *C* i *C++*, gdje programer često mora ručno upravljati memorijom, u Pajtonu se dodjeljivanje i oslobađanje memorije obavlja automatski. Interpreter vodi računa o tome kada je potrebno kreirati novi objekat i kada se memorija može osloboditi.

2.19.2 Sakupljanje otpadaka

Pajton koristi mehanizam poznat pod nazivom sakupljanje otpadaka (eng. *garbage collection*). Njegov zadatak je automatsko uklanjanje objekata koji više nisu dostupni nijednom dijelu programa.

Osnovni mehanizam zasniva se na brojanju referenci. Svaki objekat vodi evidenciju o broju aktivnih referenci koje pokazuju na njega. Kada broj referenci postane jednak nuli, objekat se može ukloniti iz memorije.

Pored toga, Pajton koristi i dodatni algoritam za otkrivanje kružnih referenci, odnosno situacija u kojima dva ili više objekata međusobno referenciraju jedni na druge, ali više nisu dostupni ostatku programa.

Zahvaljujući ovim mehanizmima, programer u većini slučajeva ne mora voditi računa o ručnom oslobađanju memorije, što značajno pojednostavljuje razvoj softvera i smanjuje mogućnost grešaka u vezi sa memorijom.

2.19.3 Proces prevođenja i izvršavanja programa

Iako se Pajton često opisuje kao interpretirani programski jezik, proces izvršavanja programa zapravo uključuje i fazu prevođenja izvornog kôda. Zbog toga se može reći da je Pajton istovremeno i kompajlirani i interpretirani jezik.

Proces izvršavanja programa sastoji se od nekoliko osnovnih koraka.

1. Leksička analiza

Izvorni kôd najprije prolazi kroz leksički analizator koji ga razlaže na elementarne cjeline nazvane *tokeni*. Primjeri tokena su ključne riječi, identifikatori, operatori, literal i interpunkcijski simboli.

2. Sintaksna analiza

Dobijeni tokeni prosljeđuju se parseru koji na osnovu gramatičkih pravila jezika gradi *apstraktno sintakšno stablo* (*Abstract Syntax Tree* – *AST*). AST predstavlja hijerarhijsku strukturu programa i služi kao osnova za naredne faze prevođenja.

3. Generisanje bajtkoda

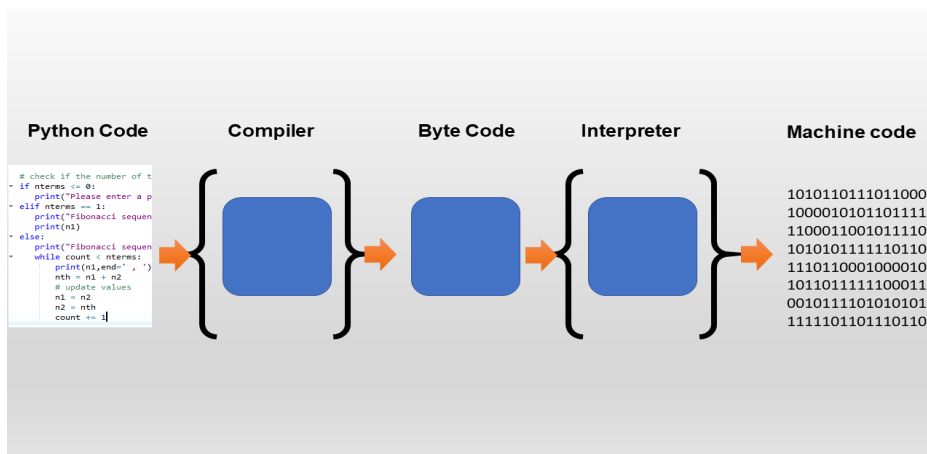
Na osnovu AST strukture Pajton generiše *bajtkod* (eng. *bytecode*), koji predstavlja međureprezentaciju nezavisnu od operativnog sistema i procesorske arhitekture.

Generisani bajtkod može biti sačuvan u datotekama sa ekstenzijom `.pyc`.

4. Izvršavanje bajtkoda

Bajtkod izvršava Pajtonova virtuelna mašina (eng. *Python Virtual Machine* – *PVM*). Virtuelna mašina čita instrukcije bajtkoda i izvršava ih jednu po jednu, upravljajući memorijom, objektima i tokom programa.

Pojednostavljen prikaz procesa izvršavanja prikazan je na Slici 2.12.



Slika 2.12: Pojednostavljen prikaz procesa prevođenja i izvršavanja Pajton programa.²

²Slika preuzeta sa: <https://www.astateofdata.com/python-programming/can-python-be-compiled/>

2.19.4 Pajtonova virtuelna mašina

Pajtonova virtuelna mašina (PVM) predstavlja centralnu komponentu izvršnog okruženja jezika Pajton. Njena osnovna uloga jeste izvršavanje bajtkoda generisanog tokom procesa prevođenja izvornog programa.

Najvažnije funkcije PVM-a su navedene u nastavku.

- **Prenosivost programa:** Bajtkod je nezavisan od konkretne hardverske platforme. Zbog toga se isti Pajton program može izvršavati na različitim operativnim sistemima bez izmjene izvornog koda.
- **Izvršavanje bajtkoda:** PVM interpretira instrukcije bajtkoda i prevodi ih u odgovarajuće operacije nad objektima i memorijom.
- **Dinamičko tipiziranje:** Pošto se tipovi objekata određuju tokom izvršavanja programa, PVM upravlja svim provjerama i operacijama vezanim za tipove podataka.
- **Upravljanje memorijom:** PVM koristi mehanizme brojanja referenci i sakupljanja otpadaka kako bi automatski oslobađala memoriju koja više nije potrebna.
- **Podrška za module i biblioteke:** PVM omogućava učitavanje i korištenje velikog broja ugrađenih i eksternih biblioteka pomoću naredbe `import`.

Treba napomenuti da standardna implementacija Pajtona (CPython) koristi interpretaciju bajtkoda. Postoje i druge implementacije, kao što su PyPy, Jython i IronPython, koje koriste drugačije tehnike izvršavanja i optimizacije programa.

Ukratko, Pajton prvo prevodi izvorni kôd u bajtkod, a zatim taj bajtkod izvršava pomoću virtuelne mašine. Zbog toga se često kaže da je Pajton hibrid između kompajliranih i interpretiranih programskih jezika.

Zadaci

1. Implementi igru pogađanja u kojoj korisnik treba da pogodi skriveni broj. Nakon pogađanja, program izvještava korisnika da li je njegov broj velik ili mali (ili je pogođen). Na kraju ispisati broj pokušaja iz kojeg je korisnik pogodio skriveni broj. Računa se jedan pokušaj ukoliko se unese isti broj više puta uzastopno.
2. Napisati program koji računa sljedeću sumu

$$4 \cdot \sum_{k=1}^{10^6} (-1)^{k-1} \frac{1}{2k-1} = 4 \cdot (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 \dots).$$

3. Napisati funkciju koja kombinira dvije liste naizmjenično uzimajući elemente jedne pa druge liste, Npr. za $[a, b, c]$, $[5, 6, 7]$ generišemo listu $[a, 5, b, 6, c, 7]$.
4. Napisati funkciju koja pomijera elemente liste za k elemenata. Npr. lista $[1, 2, 3, 4, 5, 6]$ rotirana za $k = 2$ je data sa $[3, 4, 5, 6, 1, 2]$. Riješiti zadatak bez kreiranja kopije liste.
5. Napisati funkciju koja uzima broj i vraća listu njegovih brojeva. Npr. za broj 2342 treba vratiti listu $[2, 3, 4, 2]$.
6. Napisati funkciju koja uzima listu brojeva, bazu b_1 u kojoj se brojevi liste interpretiraju, te ciljnu bazu b_2 . Kao rezultat vratiti listu brojeva konvertujući svaki od brojeva početne liste u bazu b_2 . Npr. za listu $[21, 10, 01]$ u bazi $b_1 = 3$ pretvaramo u listu brojeva po bazi $b_2 = 10$, pri čemu vraćamo listu $[7, 3, 1]$.
7. Napišite funkciju koja množi dvije matrice. Vratiti grešku (koristiti izuzetke) ukoliko su dimenzije matrica nekompatibilne.
8. Napisati regularni izraz koji prepoznaje registarske tablice. Testirati primjenu tog paterna uz pomoć modula `re`.

9. Napisati regularni izraz koji prepoznaje email adrese. Testirati primjenu tog paterna uz pomoć modula `re`.
10. Napisati regularni izraz koji prepoznaje realne brojeve (u decimalnom ili eksponencijalnom zapisu). Testirati primjenu uz pomoć modula `re`.
11. Kreirati datoteku `apsolventi.txt` i popuniti je podacima gdje je u svakom redu data informacija o studentu apsolventu u formi:

`< Ime_prezime >, < datum_upisa_na_fakultet >, < godine_starosti >`

Datum se zadaje u formi `<broj>.<broj>.<broj>`. Napisati skriptu koja se pokreće sa terminala čiji su argumenti komandne linije:

- `i`: koja zadaje ulaznu datoteku (apsolutna putanja) koja treba da bude učitana u program;
- `o`: koja zadaje izlaznu datoteku (apsolutna putanja) u kojoj se čuvaju rezultati izvršavanja programa;
- `z`: koji zadaje zadatak koji treba da bude izvršen i obuhvata sljedeći opseg vrijednosti:
 - 1: U izlaznoj datoteci treba da se ispišu svi apsolvenci stariji od 25 godina.
 - 2: U izlaznoj datoteci treba da se ispišu svi apsolvenci koji su upisali fakultet u periodu od 01.01.2014. do 01.01.2015. godine.
 - 3: U izlaznoj datoteci treba da se ispiše sortirani niz (koristiti `merge sort`) studenata po godini starosti.
 - 4: U izlaznoj datoteci treba da se ispiše za svakog studenta koliko godina studira/je studirao.
 - Za ostale inpute javiti poruku o neispravnoj opciji (unosu). Koristiti izuzetke.

Napomena. Studenta učitati uz pomoć regularnih izraza. Dakle, prazan karakter predstavlja invarijantna vrijednost što se postiže konstrukcijom odgovarajućeg regularnog izraza koji parsira sadržaj svakog reda datoteke.

12. Kreirati datoteku *pretplatnik.txt* i popuniti je podacima gdje je u svakom redu data informacija o pretplatniku u formi

< Ime_prezime >, < datum_ugovora >, < cijena_usluge >

Datum se zadaje u formi *<broj>.<broj>.<broj>*, a cijena je decimalni broj. Napisati skriptu koja se pokreće sa terminala sa sljedećim argumentima komandne linije:

- *i*: koja zadaje ulaznu datoteku (apsolutna putanja) koja treba da bude učitana u program;
- *o*: koja zadaje izlaznu datoteku (apsolutna putanja) koja treba da sačuva rezultate izvršavanja programa;
- *z*: koji zadaje zadatak koji treba da bude izvršen i obuhvata sljedeće vrijednosti:
 - 1: U izlaznoj datoteci treba da se ispišu svi korisnici koji koriste usluge čija je cijena veća od 15.4 KM. Eliminirati duplikate.
 - 2: U izlaznoj datoteci treba da se ispišu svi pretplatnici koji su potpisali ugovor u periodu od 01.01.2022. do 15.01.2023. godine.
 - 3: U izlaznoj datoteci treba da se ispiše zbir svih cijena za usluge koje koristi svaki od pretplatnika.
 - 4: U izlaznoj datoteci treba da se ispišu sortirane cijene od najmanje do najveće.
 - Za ostale inpute javiti poruku o neispravnoj opciji (unosu) koristeći izuzetke.

Napomena. Pretplatnika učitati uz pomoć regularnih izraza. Npr., smatra se da je unos Marko Markovic, 20.2.2022, 8, 4 isti kao Marko Markovic, 20. 2.2022, 8, 4, itd. Dakle, prazan karakter predstavlja invarijantnu vrijednost što se postiže konstrukcijom odgovarajućeg regularnog izraza koji parsiraju sadržaj svakog reda iz datoteke.

Glava 3

Osnovne algoritamske paradigme

Prilikom rješavanja računarskih problema često se koriste određeni obrasci razmišljanja i pristupi konstruisanju algoritama. Takvi pristupi nazivaju se *algoritamske paradigme*. Svaka paradigma predstavlja opšti metod za dizajniranje algoritama i može se primijeniti na širok spektar problema.

Prije nego što pređemo na algoritamske paradigme, vrijedi pomenuti da postoje i različite *programske paradigme*, odnosno stilovi programiranja koji određuju način organizacije i pisanja programskog kôda. Najpoznatije programske paradigme su:

- *imperativno programiranje*, zasnovano na sekvencijalnom izvršavanju naredbi;
- *funkcionalno programiranje*, koje naglasak stavlja na funkcije i transformaciju podataka;
- *logičko programiranje*, zasnovano na pravilima i zaključivanju;
- *objektno orijentisano programiranje*, koje modeluje probleme pomoću objekata i njihovih međusobnih interakcija.

Savremeni programski jezici često podržavaju više paradigmi istovremeno, omogućavajući programerima da odaberu pristup koji najbolje odgovara problemu koji rješavaju.

Kada je riječ o algoritmima, različiti problemi zahtijevaju različite strategije dizajniranja. Upravo te strategije nazivamo algoritamskim paradigmatama. U nastavku navodimo najvažnije među njima.

- **Rekurzivni algoritmi**

Zasnivaju se na razlaganju problema na manje podprobleme iste prirode. Rješenje se dobija rekurzivnim pozivanjem algoritma sve dok se ne dostigne jedan od baznih slučajeva.

- **Algoritmi grube sile** (eng. *brute force*)

Zasnivaju se na ispitivanju svih mogućih rješenja dok se ne pronade ispravno ili optimalno rješenje. Jednostavni su za implementaciju, ali često imaju visoku vremensku složenost.

- **Podijeli pa zavladaaj** (eng. *divide-and-conquer*)

Problem se dijeli na više manjih nezavisnih podproblema koji se rješavaju odvojeno, nakon čega se njihova rješenja kombinuju u rješenje originalnog problema.

- **Pohlepni algoritmi** (eng. *greedy algorithms*)

U svakom koraku donose lokalno najbolje moguće rješenje, uz pretpostavku da će takav izbor dovesti do globalno optimalnog rješenja. Karakterišu ih jednostavnost i mala potrošnja memorije, ali ne garantuju optimalnost za većinu problema.

- **Dinamičko programiranje** (eng. *dynamic programming*)

Problem se razlaže na podprobleme koji se međusobno preklapaju. Rezultati već riješenih podproblema memorišu se i ponovo koriste, čime se izbjegava višestruko računanje istih vrijednosti.

- **Vraćanje unazad** (eng. *backtracking*)

Rješenje se gradi postepeno. Kada se ustanovi da trenutno parcijalno rješenje ne može dovesti do validnog rješenja, algoritam se vraća na prethodni korak i pokušava alternativni izbor.

- **Randomizirani algoritmi**

U proces odlučivanja uvode element slučajnosti. Često se koriste za ubrzavanje algoritama ili za efikasno rješavanje problema velikih dimenzija.

Važno je naglasiti da ove paradigme nisu međusobno isključive. U praksi se veoma često kombinuju kako bi se dobili efikasniji algoritmi za rješavanje složenih računarskih problema. U nastavku knjige detaljnije ćemo proučiti nekoliko najvažnijih algoritamskih paradigmi i njihove primjene.

Ishodi učenja

Nakon uspješnog savladavanja sadržaja ove glave student će biti u stanju da:

- objasni princip rekurzije i razlikuje rekurzivni od iterativnog pristupa rješavanju problema;
- primijeni algoritme grube sile i osnovne algoritme pretraživanja na jednostavne probleme;
- objasni i primijeni paradigmu podijeli pa zavladaaj pri razlaganju problema na manje podprobleme;
- koristi paradigmu vraćanja unazad za sistematsko pretraživanje prostora rješenja i odbacivanje nedopustivih parcijalnih rješenja;
- objasni princip pohlepni algoritama i prepozna situacije u kojima lokalno najbolji izbor može voditi ka kvalitetnom ili optimalnom rješenju;

- primijeni dinamičko programiranje kroz definisanje podproblema, rekurentnih relacija i memorisanje već izračunatih rezultata te procijeni vremensku i prostornu složenost dobijenog algoritma.

3.1 Rekurzivni algoritmi

Rekurzija predstavlja jednu od osnovnih algoritamskih paradigmi. Sama riječ potiče od latinske riječi *recurrere*, što znači *vraćati se*. U programiranju se funkcija naziva *rekurzivnom* ukoliko u svom tijelu direktno ili indirektno poziva samu sebe.

Osnovna ideja rekurzije sastoji se u tome da se problem postepeno svodi na manje podprobleme iste prirode. Svaki rekurzivni poziv funkcije obrađuje jednostavniju instancu problema, sve dok se ne dostigne jedan od unaprijed definisanih *baznih slučajeva*. Bazni slučajevi predstavljaju trivijalne situacije za koje je rješenje poznato i služe za zaustavljanje rekurzivnog procesa.

Pravilno definisanje baznih slučajeva od ključnog je značaja. Ukoliko oni ne postoje ili nisu dostižni, rekurzivni pozivi će se nastaviti beskonačno, što na kraju dovodi do prekoračenja memorije rezervisane za stek.

3.1.1 Jednostavan primjer rekurzije

Posmatrajmo problem određivanja n -tog parnog broja. Neka funkcija $f(n)$ označava n -ti parni broj u skupu nenegativnih cijelih brojeva $\mathbb{N}_0$.

Pošto je prvi parni broj jednak nuli, definišemo $f(0) = 0$. Svaki naredni parni broj za dva je veći od prethodnog, pa važi rekurzivna relacija $f(n) = f(n-1) + 2, n \geq 1$. Na primjer, $f(3) = f(2) + 2 = f(1) + 2 + 2 = f(0) + 2 + 2 + 2 = 6$. Dakle, za izračunavanje vrijednosti $f(3)$ potrebno je izračunati $f(2)$, zatim $f(1)$ i konačno bazni slučaj $f(0)$. Nakon dostizanja baznog slučaja, rekurzija se vraća unazad i sukcesivno računa vrijednosti $f(1)$, $f(2)$ i $f(3)$.

Rekurzivna implementacija ove funkcije u Pajtonu izgleda ovako:

```
def even(n):
    if n == 0:
```

```

    return 0

    return even(n - 1) + 2

n = int(input("Unesite broj: "))
print(even(n))

```

Vremenska složenost. Funkcija izvršava tačno n rekurzivnih poziva, pa je njena vremenska složenost jednaka $O(n)$.

Međutim, u ovom primjeru rekurzija nije neophodna jer se lako može izvesti zatvorena formula $f(n) = 2n$.

3.1.2 Rekurzivno računanje faktoriijela

Jedan od najpoznatijih primjera primjene rekurzije jeste računanje faktoriijela prirodnog broja. Faktorijel broja n se može definisati rekurzivno sa $n! = n \cdot \underbrace{(n-1) \cdot (n-2) \cdots 2 \cdot 1}$, tj.

$$n! = n \cdot (n-1)!$$

Bazni slučaj je $0! = 1$. Implementacija računanja faktoriijela broja rekurzivnim načinom bi izgledala ovako:

```

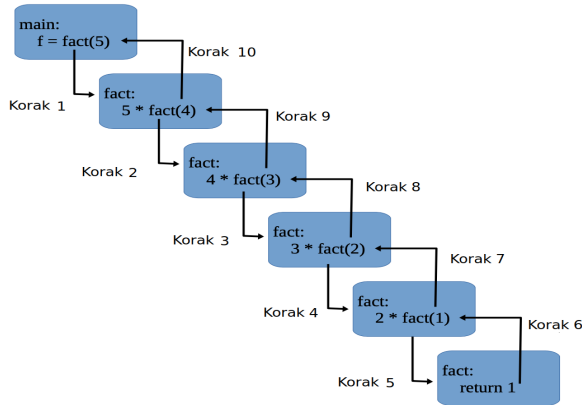
def fact(n):
    if n == 0:
        return 1

    return n * fact(n - 1)

```

Na primjer, za $n = 5$ izvršava se sljedeći niz poziva: $fact(5) \rightarrow fact(4) \rightarrow fact(3) \rightarrow fact(2) \rightarrow fact(1) \rightarrow fact(0)$.

Nakon dostizanja baznog slučaja, rezultati se vraćaju unazad kroz stek poziva.



Slika 3.1: Drvo rekurzivnih poziva za funkciju faktorijela.

Vremenska složenost. Broj rekurzivnih poziva proporcionalan je vrijednosti parametra n , pa je složenost $O(n)$.

3.1.3 Prekoračenje steka

Ako bazni slučaj nije pravilno definisan ili nije dostižan, rekurzija se neće završiti. Posmatrajmo sljedeći primjer:

```
def fact(n):

    if n == 100:
        return 1
    return n * fact(n - 1)
```

Ukoliko je početna vrijednost parametra manja od 100, funkcija će nastaviti da poziva samu sebe sa sve manjim vrijednostima argumenta i nikada neće dostići bazni slučaj. Posljedica je prekoračenje maksimalne dubine rekurzije i aktiviranje izuzetka:

```
RecursionError:
maximum recursion depth exceeded
```

Maksimalna dozvoljena dubina rekurzije može se provjeriti i po potrebi izmijeniti pomoću funkcija modula `sys`, iako se povećanje ove granice uglavnom ne preporučuje.

3.2 Vrste rekurzije

Rekurzivne funkcije mogu se klasifikovati na više načina.

Direktna i indirektna rekurzija

Funkcija je *direktno rekurzivna* ukoliko poziva samu sebe. Funkcija je *indirektno rekurzivna* ukoliko poziva drugu funkciju koja, direktno ili indirektno, ponovo poziva prvu funkciju.

```
# Direktna rekurzija

def direktna():
    direktna()

# Indirektna rekurzija
def funkcija1():
    funkcija2()

def funkcija2():
    funkcija1()
```

Repna i nerepna rekurzija

Rekurzija se naziva *repnom* (eng. *tail recursion*) ukoliko je rekurzivni poziv posljednja operacija koja se izvršava u funkciji.

```
def countdown(n):
```

```
if n == 0:
    return
return countdown(n - 1)
```

U suprotnom govorimo o *nerepnoj rekurziji*. Na primjer:

```
def fact(n):

    if n == 0:
        return 1

    return n * fact(n - 1)
```

Pošto se nakon povratka iz rekurzivnog poziva još uvijek izvršava operacija množenja, funkcija `fact()` predstavlja primjer nerepne rekurzije.

Iako neki programski jezici optimizuju repnu rekurziju, standardna implementacija Pajtona (CPython) ne primjenjuje ovu optimizaciju. Zbog toga duboke rekurzije mogu dovesti do povećane potrošnje memorije i pojave izuzetka `RecursionError`.

3.2.1 Rekurzivni i iterativni pristup: poređenje

Svaki rekurzivni algoritam može se implementirati i u iterativnom obliku, korištenjem petlji i odgovarajućih pomoćnih struktura podataka. Iako oba pristupa imaju istu računarsku moć, među njima postoje značajne razlike.

- Rekurzija se završava dostizanjem baznog slučaja, dok se iteracija završava kada uslov petlje postane netačan.
- Rekurzija je zasnovana na pozivima funkcija, dok se iteracija zasniva na petljama (`for` i `while`).

- Svaki rekurzivni poziv zauzima dodatni prostor u memoriji steka, dok iterativni algoritmi uglavnom zahtijevaju manje dodatne memorije.
- Rekurzivna rješenja često su kraća i bliža matematičkoj formulaciji problema, dok iterativna rješenja mogu biti nešto duža što se tiče kôda, ali su obično efikasnija.

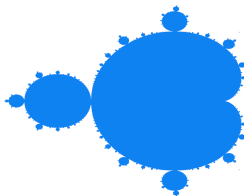
Rekurzivni programi najčešće zahtijevaju više memorije od ekvivalentnih iterativnih programa, jer se informacije o svakom pozivu funkcije čuvaju u steku sve do dostizanja baznog slučaja. Takođe, dodatno vrijeme izvršavanja troši se na pozivanje funkcija i povratak rezultata.

S druge strane, rekurzija često omogućava elegantnije i preglednije implementacije. Neki problemi prirodno imaju rekurzivnu strukturu, kao što su obilazak stabala, problem Hanojskih kula, generisanje kombinacija i permutacija, te različiti algoritmi zasnovani na paradigmi *podijeli pa zavladaš*. U takvim situacijama rekurzivni pristup često predstavlja intuitivnije rješenje.

3.2.2 Primjene rekurzije

Rekurzija se koristi u brojnim oblastima informatike i programiranja.

1. **Pretraga stabala i grafova:** Rekurzija se prirodno koristi za obilazak hijerarhijskih struktura podataka kao što su stabla i grafovi. Primjeri su algoritmi DFS (eng. *Depth-First Search*) i različiti obilaci binarnih stabala.
2. **Algoritmi sortiranja:** Mnogi poznati algoritmi sortiranja koriste rekurziju. Primjeri su Brzo sortiranje i Sort spajanjem, koji problem sortiranja razlažu na manje podprobleme i zatim kombinuju njihova rješenja.
3. **Generisanje fraktala:** Brojni fraktalni objekti mogu se generisati primjenom rekurzivnih pravila. Jedan od najpoznatijih primjera je Mandelbrotov skup.



Slika 3.2: Mandelbrotov skup.

4. **Algoritmi vraćanja unazad:** Ovi algoritmi postepeno grade rješenje problema. Kada se ustanovi da trenutno parcijalno rješenje ne može dovesti do validnog konačnog rješenja, algoritam se vraća na prethodno stanje i pokušava drugu mogućnost.
5. **Dinamičko programiranje i memoizacija:** Memoizacija predstavlja tehniku pohranjivanja prethodno izračunatih rezultata podproblema kako bi se izbjeglo njihovo ponovno računanje. Ova tehnika se često koristi zajedno sa rekurzijom i predstavlja osnovu mnogih algoritama dinamičkog programiranja.

U nastavku ćemo ilustrovati primjenu rekurzije na nekoliko konkretnih problema.

Primjer 3.1 *Pretpostavimo da se populacija zečeva razvija prema sljedećem pravilu. Par zečeva tokom prvog mjeseca života ne daje potomstvo. Počevši od drugog mjeseca života, svaki par na kraju svakog mjeseca proizvede jedan novi par zečeva. Koliko će novih parova zečeva biti nakon 12 mjeseci?*

Rješenje. Broj novorođenih parova po mjesecima formira niz

$$[0, 1, 1, 2, 3, 5, 8, 13, 21, \dots].$$

Poznat je kao Fibonačijev niz. Ako sa $f(n)$ označimo broj novorođenih parova na kraju n -tog mjeseca, tada važi rekurzivna relacija $f(n) = f(n - 1) + f(n - 2)$, $n \geq 2$, uz bazne slučajeve $f(0) = 0$, $f(1) = 1$.

Odgovarajuća rekurzivna implementacija glasi:

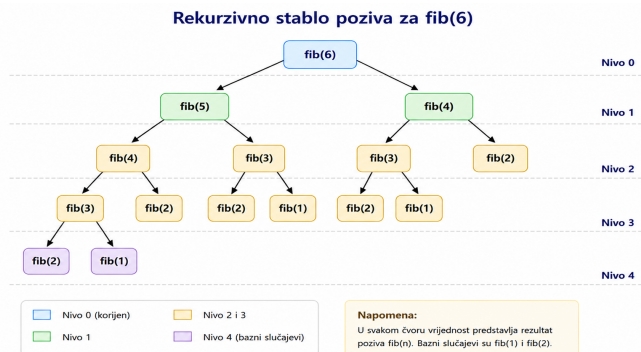
```
def fib(n):  
    if n == 0:  
        return 0  
  
    if n == 1:  
        return 1  
  
    return fib(n - 1) + fib(n - 2)
```

U svakom netrivialnom koraku funkcija generiše dva nova rekurzivna poziva. Zbog toga se broj poziva vrlo brzo povećava sa rastom vrijednosti parametra n .

Vremenska složenost. Za funkciju vrijedi rekurentna relacija $T(n) = T(n - 1) + T(n - 2) + O(1)$.

Rješenje ove rekurencije ima eksponencijalni rast, pa je vremenska složenost $T(n) = O(\varphi^n)$, gdje je $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ zlatni presjek. U literaturi se često koristi i grublja ocjena $T(n) = O(2^n)$.

Ovaj primjer pokazuje da rekurzija sama po sebi ne garantuje efikasan algoritam. U narednim poglavljima vidjećemo kako se primjenom memoizacije i dinamičkog programiranja složenost računanja Fibonačijevih brojeva može smanjiti sa eksponencijalne na linearnu.

Slika 3.3: Drvo rekurzije za `fib` rekurzivni metod.**Primjer 3.2 (Problem Hanojskih kula)**

Data su tri štapa i n diskova različitih dimenzija. Svi diskovi su inicijalno složeni na jednom štapu (A), pri čemu se manji diskovi nalaze iznad većih. Cilj je premjestiti kompletnu kulu diskova na drugi štap (C), koristeći treći štap (B) kao pomoćni.

Pri tome moraju biti zadovoljena sljedeća pravila:

- U jednom potezu dozvoljeno je premjestiti samo jedan disk.
- Disk se uvijek uzima sa vrha jednog štapa i postavlja na vrh drugog štapa.
- Veći disk nikada ne smije biti postavljen na manji disk.

Odrediti minimalan broj poteza potreban za premještanje svih diskova sa štapa A na štap C .

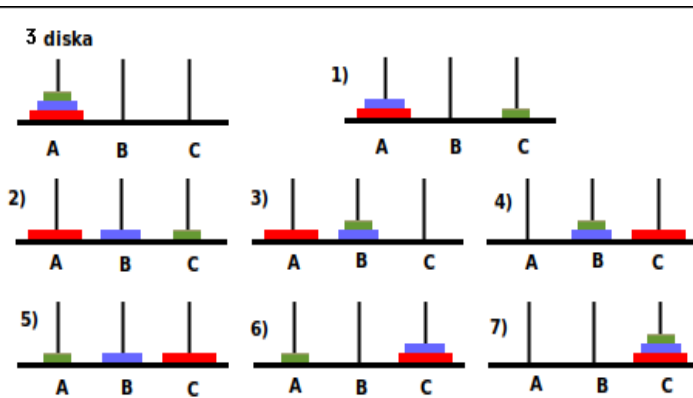
Rješenje. Najprije razmotrimo slučaj sa dva diska ($n = 2$). Pretpostavimo da se oba diska nalaze na štapu A , a da ih želimo premjestiti na štap C .

Rješenje se sastoji od tri poteza:

1. manji disk se premješta sa štapa A na štap B ;

2. veći disk se premješta sa štapa A na štap C ;
3. manji disk se premješta sa štapa B na štap C .

Posmatrajmo sada slučaj sa tri diska ($n = 3$), prikazan na Slici 3.4.



Slika 3.4: Problem Hanojskih kula za $n = 3$ diska.

Rješenje se dobija sljedećim redoslijedom poteza:

1. najmanji disk se premješta sa štapa A na štap C ;
2. srednji disk se premješta sa štapa A na štap B ;
3. najmanji disk se premješta sa štapa C na štap B ;
4. najveći disk se premješta sa štapa A na štap C ;
5. najmanji disk se premješta sa štapa B na štap A ;
6. srednji disk se premješta sa štapa B na štap C ;
7. najmanji disk se premješta sa štapa A na štap C .

Ukupno je potrebno sedam poteza.

Da bismo riješili opšti slučaj, označimo sa $T(n)$ minimalan broj poteza potreban za premještanje n diskova sa štapa A na štap C , koristeći štap B kao pomoćni.

Bazni slučaj je jednostavan: $T(1) = 1$. Za proizvoljno $n > 1$, najprije je potrebno premjestiti gornjih $n - 1$ diskova sa štapa A na štap B , koristeći štap C . Za to je potrebno $T(n - 1)$ poteza. Nakon toga najveći disk ostaje sam na štapu A i može se direktno premjestiti na štap C u jednom potezu.

Preostaje da se $n - 1$ diskova sa štapa B rekurzivno premjesti na štap C , za šta je ponovo potrebno $T(n - 1)$ poteza. Prema tome, ukupan broj poteza zadovoljava rekurentnu relaciju $T(n) = T(n - 1) + 1 + T(n - 1)$, odnosno $T(n) = 2T(n - 1) + 1, T(1) = 1$. Prethodni postupak opisuje rekurzivnu formulaciju problema Hanojskih kula.

Rješavanjem navedene rekurencije dobija se zatvorena forma $T(n) = 2^n - 1$. Dakle, minimalan broj poteza potreban za premještanje n diskova iznosi $2^n - 1$. Na primjer, za $n = 3$ dobijamo $T(3) = 2^3 - 1 = 7$, što se slaže sa prethodno izvedenim rješenjem.

Implementacija rekurzivnog pristupa problema Hanojskih kula je data sljedećim kôdom.

```
def TowerOfHanoi(n , izvor, cilj, pomocni):
    if n==1:
        print ("Pomaknite disk 1 sa štapa ", izvor," \
na štap ", cilj)
        return
    TowerOfHanoi(n-1, izvor, pomocni, cilj)
    print ("Pomakni disk ",n," sa štapa ", izvor," na štap \
", cilj) #pomijeranje najvećeg štapa
    TowerOfHanoi(n-1, pomocni, cilj, izvor)

#instanca:
```

```
n = 4 #broj diskova
TowerOfHanoi(n, 'A', 'C', 'B')
```

Vremenska kompleksnost. Vrlo lako dobijamo eksplicitnu funkciju koja odgovara rekurzivnoj relaciji problema: $T(n) = 2T(n-1) + 1 = 2 \cdot (2 \cdot T(n-2) + 1) + 1 = \dots = 2^{n-2} + 2^{n-1} + \dots + 2 + 1 = 2^n - 1$. Rekurzija se izvršava u eksponencijalnom $O(2^n)$ vremenu.

Primjer 3.3 *Sortiranje umetanjem (Insertion Sort) predstavlja jednostavan algoritam sortiranja koji niz gradi postepeno, održavajući jedan njegov dio sortiranim. Ideja algoritma sastoji se u tome da se svaki novi element umetne na odgovarajuću poziciju unutar već sortiranog dijela niza. Niz koji se sastoji od jednog elementa se smatra sortiranim.*

Rješenje. Algoritam sortiranja umetanjem može se prirodno formulirati i rekurzivno. Neka je dat niz

$$A = (a_0, a_1, \dots, a_{n-1})$$

od n elemenata koje je potrebno sortirati u neopadajućem poretku.

Rekurzivna formulacija zasniva se na sljedećem zapažanju. Pretpostavimo da smo već sortirali prvih $n-1$ elemenata niza:

$$(a_0, a_1, \dots, a_{n-2}).$$

Tada je za sortiranje kompletnog niza dovoljno posljednji element a_{n-1} umetnuti na odgovarajuću poziciju unutar već sortiranog prefiksa.

Prema tome, algoritam se sastoji od dvije osnovne faze:

1. rekurzivno sortirati prvih $n-1$ elemenata niza;
2. umetnuti posljednji element a_{n-1} na odgovarajuću poziciju unutar sortiranog prefiksa.

Bazni slučaj nastupa kada niz sadrži najviše jedan element. Takav niz je već sortirani, pa nije potrebno izvršavati dodatne operacije.

Pretpostavimo sada da su elementi

$$a_0, a_1, \dots, a_{n-2}$$

već sortirani. Vrijednost a_{n-1} privremeno čuvamo u promjenljivoj `key`. Zatim se, počevši od elementa a_{n-2} , krećemo unazad kroz sortirani dio niza. Svaki element koji je veći od vrijednosti `key` pomjera se za jednu poziciju udesno. Kada pronađemo prvi element koji nije veći od `key`, vrijednost `key` umećemo neposredno iza njega.

Na ovaj način nakon završetka postupka prvih n elemenata niza nalazi se u sortiranom poretku.

```
def RecursiveInsertionSort(A, n):
    if n <= 1:
        return

    # Rekurzivno sortiramo prvih n-1 elemenata
    RecursiveInsertionSort(A, n - 1)

    # Umetanje posljednjeg elementa
    key = A[n - 1]
    j = n - 2

    while j >= 0 and A[j] > key:
        A[j + 1] = A[j]
        j = j - 1

    A[j + 1] = key
```

Na primjer, posmatrajmo niz

$$A = [7, 3, 5, 2].$$

Rekurzivni algoritam najprije sortira prefiks dužine 3, odnosno niz $[7, 3, 5]$, koji nakon sortiranja postaje $[3, 5, 7]$. Preostaje umetanje posljednjeg elementa 2. Pošto je 2 manje od svih elemenata sortiranog prefiksa, elementi 7, 5 i 3 pomjeraju se za jednu poziciju udesno, nakon čega se vrijednost 2 postavlja na početak niza. Dobijamo konačno sortirani niz $[2, 3, 5, 7]$.

Ispravnost algoritma. Ispravnost rekurzivnog algoritma možemo pokazati indukcijom po veličini niza n . Za $n = 1$ niz sadrži samo jedan element i trivijalno je sortiran. Pretpostavimo sada da algoritam ispravno sortira svaki niz od $n - 1$ elemenata. Pri pozivu algoritma za niz od n elemenata, rekurzivni poziv sortira njegovih prvih $n - 1$ elemenata. Nakon toga posljednji element a_{n-1} umeće se na odgovarajuću poziciju unutar sortiranog prefiksa, pri čemu se svi elementi veći od njega pomjeraju za jednu poziciju udesno. Prema tome, nakon umetanja svih n elemenata nalazi se u neopadajućem poretku. Time je pokazano da algoritam ispravno sortira niz proizvoljne dužine n .

Vremenska složenost. Neka je $T(n)$ vrijeme potrebno za sortiranje niza od n elemenata.

Algoritam najprije rekurzivno sortira prvih $n - 1$ elemenata, što zahtijeva vrijeme $T(n - 1)$. Nakon toga posljednji element umeće se u već sortirani prefiks.

U najgorem slučaju posljednji element je manji od svih prethodnih elemenata. Tada je potrebno pomjeriti svih $n - 1$ elemenata za jednu poziciju udesno. Zbog toga operacija umetanja zahtijeva $\Theta(n)$ vremena.

Dobijamo rekurentnu relaciju

$$T(n) = T(n - 1) + \Theta(n),$$

uz bazni slučaj

$$T(1) = \Theta(1).$$

Razvijanjem rekurzije dobijamo

$$\begin{aligned} T(n) &= T(n - 1) + \Theta(n) \\ &= T(n - 2) + \Theta(n - 1) + \Theta(n) \\ &\vdots \\ &= T(1) + \Theta(2) + \Theta(3) + \cdots + \Theta(n). \end{aligned}$$

Kako vrijedi

$$1 + 2 + \cdots + n = \frac{n(n + 1)}{2},$$

slijedi

$$T(n) = \Theta(n^2).$$

Prema tome, vremenska složenost rekurzivnog algoritma sortiranja umetanjem u najgorem slučaju iznosi $\Theta(n^2)$.

Najgori slučaj nastupa kada je početni niz sortiran u opadajućem poretku. Tada se pri svakom umetanju trenutni element mora pomjeriti preko svih prethodno obrađenih elemenata.

U najboljem slučaju niz je već sortiran. Tada se pri svakom rekurzivnom nivou izvršava samo jedno poređenje prije nego što se utvrdi da se posljednji element već nalazi na odgovarajućoj poziciji. Rekurentna relacija tada ima oblik

$$T_{\text{best}}(n) = T_{\text{best}}(n-1) + \Theta(1),$$

odakle slijedi

$$T_{\text{best}}(n) = \Theta(n).$$

Prosječna vremenska složenost algoritma iznosi $\Theta(n^2)$.

Prostorna složenost. Sama operacija umetanja koristi samo konstantan broj dodatnih promjenljivih. Međutim, zbog rekurzivne implementacije na steku se istovremeno može nalaziti do n aktivnih poziva funkcije. Zbog toga dodatna prostorna složenost rekurzivnog algoritma iznosi $\Theta(n)$.

Kod standardne iterativne implementacije sortiranja umetanjem ovaj dodatni trošak ne postoji, pa je njena prostorna složenost $\Theta(1)$.

Sortiranje umetanjem nije efikasno za velike nizove zbog kvadratne prosječne i najgore vremenske složenosti. Ipak, algoritam ima nekoliko važnih prednosti. Jednostavan je za implementaciju, ne zahtijeva dodatnu memoriju za čuvanje elemenata i veoma je efikasan za male ili skoro sortirane nizove.

Rekurzivna formulacija posebno je korisna sa pedagoškog stanovišta, jer jasno pokazuje način na koji se problem veličine n može svesti na problem istog tipa veličine $n-1$. Time sortiranje umetanjem predstavlja jednostavan primjer primjene rekurzivnog načina razmišljanja na problem sortiranja.

3.3 Algoritmi grube sile

Algoritmi grube sile predstavljaju jednu od najjednostavnijih algoritamskih paradigmi za rješavanje računarskih problema. Osnovna ideja ove paradigme jeste sistematsko ispitivanje svih mogućih rješenja dok se ne pronađe rješenje koje zadovoljava postavljene uslove ili dok se ne odredi najbolje rješenje prema zadanom kriterijumu.

Iako je riječ o jednostavnom pristupu, algoritmi grube sile imaju značajnu ulogu u algoritamskom dizajnu. Često predstavljaju polaznu tačku za razvoj efikasnijih algoritama, a za probleme manjih dimenzija mogu dati zadovoljavajuća rješenja.

U svakodnevnom životu primjer algoritma grube sile je pokušaj otključavanja brave kada ne znamo koji ključ odgovara. U tom slučaju isprobavamo jedan po jedan ključ sve dok ne pronađemo odgovarajući. Slično tome, ukoliko ne znamo orijentaciju USB priključka, pokušavamo jednu mogućnost, a zatim drugu.

Paradigma grube sile može se koristiti za:

- probleme pretraživanja,
- probleme zadovoljenja ograničenja (SAT problemi),
- probleme optimizacije,
- verifikaciju ispravnosti složenijih algoritama.

Primjer 3.4 (*Problem najbližeg para tačaka*) Neka je dat skup od n tačaka u prostoru $\mathbb{R}^m$:

$$S = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}.$$

Potrebno je pronaći dvije tačke iz skupa S čija je međusobna Euklidova udaljenost najmanja.

Rješenje. Pristup grube sile podrazumijeva izračunavanje udaljenosti između svakog para tačaka iz skupa S . Tokom izvršavanja algoritma pamtimo najmanju do sada pronađenu udaljenost i odgovarajući par tačaka.

Jedna moguća implementacija prikazana je u nastavku.

```
import math

def min_dist_pair(S):

    min_dist = float("inf")
    point1 = None
    point2 = None

    for i in range(len(S)):
        for j in range(i + 1, len(S)):
            distance = math.dist(S[i], S[j])

            if distance < min_dist:
                min_dist = distance
                point1 = S[i]
                point2 = S[j]

    return point1, point2, min_dist
```

Analiza složenosti. Broj različitih parova tačaka jednak je $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$. Računanje Euklidove udaljenosti između dvije tačke u prostoru $\mathbb{R}^m$ se vrši u $O(m)$ vremenu. Kako je dimenzija prostora m fiksna, ukupna vremenska složenost algoritma iznosi $O(n^2)$.

Primjer 3.5 (*Problem ruksaka*). Neka je dato n predmeta. Svaki predmet i ima težinu w_i i vrijednost p_i , za svaki $i = 1, \dots, n$. Takođe je dat ruksak kapaciteta C .

Potrebno je odabrati predmete koje treba staviti u ruksak tako da njihova ukupna težina ne prelazi kapacitet ruksaka, a pri tome je ukupna vrijednost u ruksaku najveća moguća.

Rješenje. Posmatrajmo sljedeću instancu problema.

<i>Predmet</i>	<i>Težina</i>	<i>Vrijednost</i>
1	2	20
2	5	30
3	10	50
4	5	10

Neka je kapacitet ruksaka $C = 16$. Pristup grube sile sastoji se u generisanju svih mogućih podskupova skupa predmeta. Kako postoje četiri predmeta, ukupan broj mogućih rješenja jednak je $2^4 = 16$.

Za svaki podskup izračunava se ukupna težina i ukupna vrijednost.

Rješenje	Ukupna težina	Ukupna vrijednost
{1}	2	20
{2}	5	30
{3}	10	50
{4}	5	10
{1, 2}	7	50
{1, 3}	12	70
{1, 4}	7	30
{2, 3}	15	80
{2, 4}	10	40
{3, 4}	15	60
{1, 2, 3}	17	nedopustivo
{1, 2, 4}	12	60
{1, 3, 4}	17	nedopustivo
{2, 3, 4}	20	nedopustivo
{1, 2, 3, 4}	22	nedopustivo

Iz tabele se vidi da optimalno rješenje odgovara skupu {2, 3}, čija je ukupna težina 15, a ukupna pridružena vrijednost 80.

Pošto se svaki predmet može ili nalaziti ili ne nalaziti u ruksaku, prostor rješenja odgovara skupu svih podskupova skupa $\{1, \dots, n\}$. Broj mogućih rješenja jednak je 2^n .

Zbog toga je vremenska složenost algoritma grube sile eksponencijalna, $O(2^n)$.

Jedna moguća implementacija prikazana je u nastavku.

```
best_fitness = 0

# primjer instance
N = 4
w = [2, 5, 10, 5]
p = [20, 30, 50, 10]
C = 16

def objective(solution):

    weight = 0
    value = 0

    for i in solution:
        weight += w[i]
        value += p[i]

    if weight > C:
        return float("-inf")
    return value

def knapsack_brute_force(solution):

    global best_fitness

    best_fitness = max(
        best_fitness,
        objective(solution))

    for i in range(N):

        if len(solution) == 0:
            knapsack_brute_force([i])

        elif i > max(solution): #razbijanje simetrije
```

```
knapsack_brute_force(solution + [i])

knapsack_brute_force([])
print(best_fitness)    # 80
```

Algoritmi grube sile obično nisu praktični za velike instance problema zbog eksponencijalnog ili polinomijalno visokog vremena izvršavanja. Ipak, oni imaju značajnu teorijsku vrijednost jer garantuju pronalaženje optimalnog rješenja i često predstavljaju osnovu za razvoj efikasnijih algoritama zasnovanih na paradigmi *podijeli pa zavladaaj*, dinamičkom programiranju, grananju i ograničavanju (eng. *branch-and-bound*) ili metaheuristikama.

Kako primjećujemo na osnovu prethodne (rekurzivne) implementacije, svaki od rekurzivnih poziva, kojih je tačno 2^n , poziva funkciju `objective`. Preciznije, kompleksnost pristupa brutalne sile je $O(2^n)$. Napomenimo da postoje efikasniji pristupi za rješavanje problema ruksaka. Među njima, izdvajamo metod dinamičkog programiranja, o kojem će biti više riječi u nastavku.

3.4 Algoritmi pretraživanja

Algoritmi pretraživanja predstavljaju grupu algoritama čiji je zadatak pronalaženje određenog elementa unutar strukture podataka. Rezultat pretrage najčešće je informacija da li traženi element postoji, a ukoliko postoji, vraća se njegova pozicija ili druga relevantna informacija o njemu.

Algoritmi pretraživanja imaju široku primjenu u računarstvu, od rada sa nizovima i listama do pretraživanja baza podataka, tekstualnih dokumenata i složenih struktura podataka.

U zavisnosti od načina organizacije podataka i strategije pretraživanja, algoritmi pretraživanja mogu se podijeliti u dvije osnovne kategorije:

- *Sekvencijalna pretraga* prolazi kroz elemente strukture jedan po jedan sve dok ne pronađe traženi element ili ne iscrpi cijelu strukturu. Najpoznatiji predstavnik ove grupe je *linearna pretraga*.

- *Intervalna pretraga* koristi dodatne informacije o organizaciji podataka, najčešće činjenicu da su podaci sortirani. Umjesto provjeravanja svih elemenata, prostor pretrage se postepeno smanjuje, čime se postiže znatno veća efikasnost. Najpoznatiji primjer je *binarna pretraga*.

3.4.1 Linearna pretraga

Linearna pretraga (eng. *linear search*) predstavlja najjednostavniji algoritam pretraživanja. Ideja algoritma je da se elementi strukture podataka obilaze redom, od početka prema kraju, sve dok se ne pronađe tražena vrijednost.

U slučaju da se traženi element pronađe, algoritam vraća njegov indeks. Ukoliko se dođe do kraja strukture bez pronalaska elementa, vraća se posebna vrijednost koja označava neuspješnu pretragu.

Iterativna implementacija linearne pretrage prikazana je u nastavku.

```
def search(arr, x):  
  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
  
    return -1  
  
arr = [2, 3, 1, 11, 10, 20]  
x = 10  
res = search(arr, x)  
  
if res == -1:  
    print("Element nije pronađen.")  
else:  
    print("Element je pronađen na indeksu", res)
```

Analiza složenosti. U najgorem slučaju algoritam mora pregledati

svih n elemenata strukture podataka. Zbog toga je vremenska složenost linearne pretrage $O(n)$, gdje je n broj elemenata u strukturi. Dodatna memorija koja se koristi tokom izvršavanja algoritma je konstantna, pa je prostorna složenost jednaka $O(1)$.

Linearna pretraga može se implementirati i rekurzivno.

```
def linear_search(arr, x, i):  
  
    if i >= len(arr):  
        return -1  
    if arr[i] == x:  
        return i  
  
    return linear_search(arr, x, i + 1)  
  
# poziv funkcije  
res = linear_search(arr, x, 0)
```

Kod rekurzivne implementacije vremenska složenost ostaje nepromijenjena i iznosi $O(n)$. Međutim, svaki rekurzivni poziv zauzima dodatni prostor na steku, pa je prostorna složenost $O(n)$.

Prednosti i nedostaci. Glavna prednost linearne pretrage jeste njena jednostavnost. Algoritam ne zahtijeva nikakvu prethodnu obradu podataka niti sortiranje strukture nad kojom se vrši pretraga. Zbog toga se može primijeniti na bilo kojem nizu, listi ili drugoj sekvencijalnoj strukturi podataka.

Sa druge strane, linearna pretraga nije efikasna za velike skupove podataka, jer se broj potrebnih poređenja povećava proporcionalno veličini ulaza. Iz tog razloga koristi se uglavnom za manje strukture podataka ili u situacijama kada podaci nisu sortirani i kada jednostavnost implementacije ima prednost nad efikasnošću izvršavanja.

3.4.2 Binarna pretraga

Binarna pretraga (eng. *binary search*) predstavlja jedan od najefikasnijih algoritama za pretraživanje podataka. Za razliku od linearne pretrage, koja elemente provjerava redom, binarna pretraga u svakoj iteraciji eliminiše približno polovinu preostalog prostora pretrage.

Da bi se binarna pretraga mogla primijeniti, moraju biti zadovoljena dva osnovna uslova:

1. podaci moraju biti sortirani;
2. pristup proizvoljnom elementu strukture mora biti moguć u konstantnom vremenu.

Pretpostavimo da je niz sortiran u rastućem poretku. U svakoj iteraciji određuje se srednji element trenutnog intervala pretrage i poredi sa traženom vrijednošću x .

Moguća su tri slučaja:

- ako je srednji element jednak vrijednosti x , pretraga se završava;
- ako je srednji element veći od x , traženi element se može nalaziti samo u lijevoj polovini niza;
- ako je srednji element manji od x , pretraga se nastavlja u desnoj polovini niza.

Na taj način se u svakom koraku prostor pretrage prepolovljava.

Rekurzivna implementacija binarne pretrage prikazana je u nastavku.

```
def binary_search(arr, x):  
  
    if len(arr) == 0:  
        return False  
  
    if len(arr) == 1:
```

```

        return arr[0] == x
    mid = len(arr) // 2

    if arr[mid] == x:
        return True
    elif arr[mid] < x:
        return binary_search(arr[mid + 1:], x)
    else:
        return binary_search(arr[:mid], x)

# primjer poziva

arr = [2, 8, 10, 12, 15]
x = 10
found = binary_search(arr, x)
print(found)

```

Analiza složenosti. Neka je $T(n)$ vrijeme izvršavanja algoritma za niz dužine n . U svakoj iteraciji pretražuje se samo jedna polovina niza, pa vrijedi rekurentna relacija $T(n) = T\left(\frac{n}{2}\right) + O(1)$. Primjenom Master teoreme dobija se $T(n) = O(\log n)$. Dakle, binarna pretraga je značajno efikasnija od linearne pretrage čija je složenost $O(n)$.

Binarna pretraga se posebno koristi kod velikih sortiranih skupova podataka, gdje njena logaritamska složenost omogućava veoma brzo pronalaženje traženih elemenata.

3.4.3 Pretraga preskakanjem

Pretraga preskakanjem (eng. *jump search*) predstavlja kompromis između linearne i binarne pretrage. Kao i binarna pretraga, zahtijeva da podaci budu sortirani.

Osnovna ideja algoritma jeste da se niz ne pretražuje element po element, već da se kroz njega kreće u koracima određene dužine. Na taj način

brzo se pronalazi manji interval u kojem se potencijalno nalazi traženi element, nakon čega se unutar tog intervala primjenjuje linearna pretraga.

Pretpostavimo da je dat sortirani niz dužine n i da je veličina koraka jednaka m . Algoritam provjerava elemente na pozicijama $0, ; m, ; 2m, ; 3m, \dots$ sve dok ne pronađe interval koji sadrži traženi element x .

Preciznije, traži se prvi indeks k za koji vrijedi $\text{niz}[k \cdot m] \geq x$. Nakon toga vrši se linearna pretraga unutar prethodnog bloka.

Posmatrajmo niz

$[0, 1, 1, 2, 3, 5, 8, 12, 24, 37, 55, 82, 140, 433, 567, 622]$

i pretpostavimo da je potrebno naći vrijednost $x = 55$, koristeći korak $m = 4$. Algoritam izvršava sljedeće korake:

1. provjerava element na indeksu 0;
2. prelazi na indeks 4;
3. prelazi na indeks 8;
4. prelazi na indeks 12;
5. pošto je vrijednost na indeksu 12 jednaka $140 > 55$, zaključuje da se traženi element nalazi između indeksa 8 i 12;
6. vrši linearnu pretragu unutar tog intervala i pronalazi vrijednost 55.

Analiza složenosti. U najgorem slučaju potrebno je izvršiti približno $\frac{n}{m}$ skokova, a zatim još najviše $m - 1$ poređenja tokom linearne pretrage unutar pronađenog bloka.

Ukupan broj poređenja iznosi $f(m) = \frac{n}{m} + m - 1$. Minimum ove funkcije postiže se za $m = \sqrt{n}$. Zamjenom optimalne vrijednosti koraka dobija se vremenska složenost $O(\sqrt{n})$. Pretraga preskakanjem je sporija od binarne pretrage, ali jednostavnija za implementaciju u pojedinim strukturama podataka kod kojih nije praktično često dijeljenje intervala. Zbog toga predstavlja koristan kompromis između jednostavnosti linearne pretrage i efikasnosti binarne pretrage.

Rekurzivna implementacija pretage preskakanjem je data sljedećim kôdom.

```
def jump_search(niz, x, jump, k):

    if k * jump >= len(niz): #empty array
        return False

    if k * jump < len(niz) and (k+1) * jump >= len(niz):
        found = linear_search(niz[k*jump:], x, 0)
        return found
    #rekurzivni korak: oba kraja intervala su elementi niza:
    if niz[k * jump] <= x and x <= niz[(k+1) * jump]:
        found = linear_search(niz[k*jump: ((k+1)*jump + 1)], x, 0)
        return found
    else:
        return jump_search(niz, x, jump, k+1) # idemo na naredni interval

#instanca:
niz = [2, 10, 21, 33, 38, 41, 45, 52, 57, 70, 75]
jump_step = 3
x = 57
# poziv metoda:
found = jump_search(niz, 57, jump_step, 0)
print(found)
```

3.4.4 Interpolacijska pretraga

Interpolacijska pretraga (eng. *interpolation search*) predstavlja unapređenje binarne pretrage namijenjeno radu sa sortiranim nizovima čije su vrijednosti približno ravnomjerno raspoređene. Za razliku od binarne pretrage, koja uvijek ispituje srednji element trenutnog intervala, interpolacijska pretraga pokušava procijeniti poziciju traženog elementa na osnovu njegove vrijednosti.

Osnovna ideja algoritma zasniva se na analogiji sa pretragom riječi u rječniku. Ukoliko tražimo riječ koja počinje slovom „Š“, nećemo otvoriti rječnik na sredini, već bliže njegovom kraju. Slično tome, ako je vrijednost traženog elementa bliža najvećem elementu niza, algoritam će pretragu započeti bliže kraju niza.

Pretpostavimo da je dat sortirani niz $[niz[0], , niz[1], , \dots, niz[n - 1]]$ i da tražimo element vrijednosti x . Pozicija na kojoj će se izvršiti naredno poređenje procjenjuje se linearnom interpolacijom između krajnjih elemenata trenutnog intervala pretrage.

Vrijednost indeksa pos računa se formulom $pos = \left\lfloor \frac{(n-1)(x-niz[0])}{niz[n-1]-niz[0]} \right\rfloor$.

Drugim riječima, ukoliko je vrijednost x bliža početku intervala, dobiće se manji indeks pos ; ako je bliža kraju intervala, dobiće se veći indeks.

Algoritam se sastoji od sljedećih koraka:

1. Izračunati procijenjenu poziciju pos koristeći interpolacionu formulu.
2. Ako je $niz[pos] = x$, pretraga se završava i vraća se pronađena pozicija.
3. Ako je $x < niz[pos]$, pretraga se nastavlja u lijevom dijelu niza.
4. Ako je $x > niz[pos]$, pretraga se nastavlja u desnom dijelu niza.
5. Koraci 1–4 ponavljaju se sve dok se element ne pronađe ili dok interval pretrage ne postane prazan.

Jedna moguća implementacija interpolacijske pretrage prikazana je u nastavku.

```
def interpolation_search(arr, x):

    left = 0
    right = len(arr) - 1

    while (left <= right and x >= arr[left] and \
           x <= arr[right]):

        if left == right:
            if arr[left] == x:
```

```
        return left
    return -1

pos = left + ((x - arr[left]) * \
             (right - left)) // (arr[right] - arr[left] )

    if arr[pos] == x:
        return pos
    elif arr[pos] < x:
        left = pos + 1
    else:
        right = pos - 1
return -1
```

Analiza složenosti. Ako su vrijednosti u nizu približno uniformno raspoređene, interpolacijska pretraga u prosjeku ostvaruje vremensku složenost $O(\log \log n)$, što je bolje od složenosti binarne pretrage, koja iznosi $O(\log n)$.

Međutim, ova prednost postoji samo kada je pretpostavka o približno ravnomjernoj raspodjeli podataka zadovoljena. U nepovoljnom slučaju, kada su podaci izrazito neravnomjerno raspoređeni, vremenska složenost može degradirati na $O(n)$, što odgovara složenosti linearne pretrage. Zbog toga se interpolacijska pretraga koristi prvenstveno kod velikih sortiranih skupova numeričkih podataka za koje se očekuje približno uniformna raspodjela vrijednosti podataka.

3.4.5 Eksponencijalna pretraga

Eksponencijalna pretraga (eng. *exponential search*) predstavlja algoritam pretraživanja namijenjen sortiranim nizovima. Naziv algoritma potiče od činjenice da se granice prostora pretrage proširuju eksponencijalno, odnosno u koracima veličine 1, 2, 4, 8, 16, ...

Osnovna ideja algoritma sastoji se od dvije faze:

1. pronalaženja intervala u kojem se traženi element može nalaziti korištenjem eksponencijalnih skokova;
2. primjene binarne pretrage unutar pronađenog intervala.

Pretpostavimo da tražimo element x u sortiranom nizu. Najprije provjeravamo elemente na pozicijama $1, 2, 4, 8, 16, \dots, 2^i$, sve dok ne pronađemo indeks i za koji važi $niz[2^{i-1}] \leq x \leq niz[2^i]$, ili dok ne dostignemo kraj niza. Na taj način određujemo interval u kojem se traženi element može nalaziti. Nakon toga nad pronađenim intervalom primjenjujemo binarnu pretragu.

Drugim riječima, eksponencijalna pretraga najprije veoma brzo sužava prostor pretrage eksponencijalnim povećavanjem indeksa, a zatim koristi binarnu pretragu da precizno odredi poziciju traženog elementa. Ovakav pristup omogućava efikasno pretraživanje velikih sortiranih nizova, posebno kada se traženi element nalazi blizu početka niza.

Jedna moguća implementacija eksponencijalne pretrage data je sljedećim kodom.

```
def exponential_search(niz, x, l):

    if len(niz) <= 1:
        if x in niz:
            return l + int(x in niz)
    else:
        return -1

    if l == 0:
        pos = 2
    else:
        pos = l * 2

    if pos >= len(niz):
        ind_found = binary_search(niz[l:], x)
        # ako x ne postoji u sufiksu
        if ind_found == -1:
            return -1
        else:
            return l + binary_search(niz[l:], x)
```

```

    else:
        if niz[l] <= x and x <= niz[pos] :
            return l + binary_search(niz[l: pos+1], x)
        else:
            return exponential_search(niz, x, pos)
# instance:
niz = [2, 10, 22, 38, 44, 51, 100, 220, 550, 880]
x = 10
# poziv funkcije:
print(exponential_search(niz, x, 0))

```

Kompleksnost algoritma. U prvoj fazi algoritam eksponencijalnim koracima pronalazi odgovarajući interval. Broj potrebnih skokova je najviše $O(\log n)$. Nakon toga se nad pronađenim intervalom izvršava binarna pretraga, čija je složenost takođe $O(\log n)$.

Prema tome, ukupna vremenska složenost eksponencijalne pretrage iznosi $O(\log n)$. Prostorna složenost je $O(1)$ ukoliko se koristi iterativna implementacija binarne pretrage.

Eksponencijalna pretraga je posebno korisna kod veoma velikih sortiranih nizova, naročito kada veličina niza nije unaprijed poznata ili kada se očekuje da se traženi element nalazi blizu početka niza. U takvim situacijama često postiže bolje praktične performanse od klasične binarne pretrage, jer brže sužava prostor pretraživanja na relevantan interval.

3.5 Podijeli pa zavladaaj

Paradigma *podijeli pa zavladaaj* zasniva se na ideji da se složen problem podijeli na više manjih podproblema istog tipa, koji se zatim rješavaju nezavisno. Nakon što se dobiju rješenja podproblema, ona se kombinuju kako bi se dobilo rješenje početnog problema.

Ova paradigma se sastoji od tri osnovna koraka:

1. *podijeli* početni problem na manje, međusobno nezavisne podprobleme;
2. *rekurzivno riješi* svaki od podproblema;

3. *kombinuj* rješenja podproblema da bi se dobilo rješenje originalnog problema.

Pseudokod ove paradigme dat je Algoritmom 5.

Algoritam 5 Paradigma Podijeli pa zavladaaj.

```

1: procedure DC( $n$ : veličina ulaza)
2:   if  $n \leq n_0$  then
3:     Direktno riješiti problem
4:   else
5:     Podijeliti problem na  $r$  podproblema veličine približno  $n/k$ 
6:     for svaki podproblem do
7:       Pozvati D&C( $n/k$ )
8:     end for
9:     Kombinovati rješenja podproblema u rješenje početnog problema
10:  end if
11: end procedure

```

Uspješna primjena ove paradigme zahtijeva da podjela problema i kombinovanje parcijalnih rješenja budu dovoljno efikasni. U suprotnom, trošak ovih operacija može poništiti prednosti dobijene podjelom problema na manje dijelove.

Jedan od najpoznatijih primjera primjene paradigme podijeli pa zavladaaj jeste algoritam sortiranja spajanjem (tzv. *merge sort*), koji sortira niz od n elemenata u vremenu $O(n \log n)$.

Osnovna ideja ovog algoritma je sljedeća:

- podijeliti niz na dva približno jednaka i disjunktna dijela;
- rekurzivno sortirati lijevi i desni podniz;
- spojiti dva sortirana podniza u jedan sortirani niz.

Ključna prednost ovog pristupa je činjenica da se operacija spajanja dva već sortirana niza može izvršiti u linearnom vremenu, tj. u $O(n)$ koraka.

Implementacija algoritma sortiranja spajanjem je data u nastavku.

```
def mergeSort(niz):

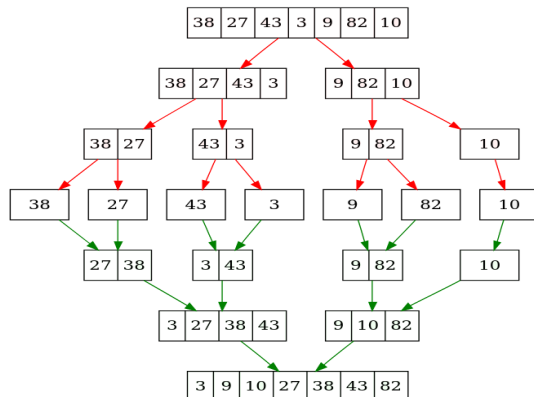
    if len(niz) <= 1:
        return niz
    else:
        #partitioning:
        nizLeftSort = niz[0 : (len(niz)//2) ]
        nizRightSort = niz[ (len(niz) // 2) :]
        mergeSort(nizLeftSort)
        mergeSort(nizRightSort)
        ## combine
        niz.clear()
        indexL = indexR = 0
        while(indexL < len(nizLeftSort) and indexR < len(nizRightSort)):
            if(nizLeftSort[ indexL ] >= nizRightSort[ indexR ]):
                niz.append(nizLeftSort[indexL])
                indexL = indexL + 1
            else:
                niz.append(nizRightSort[indexR])
                indexR = indexR + 1

        if(indexL < len(nizLeftSort)):
            niz += nizLeftSort[indexL : ]
        elif (indexR < len(nizRightSort)):
            niz += nizRightSort[indexR : ]

    return niz
```

Spoljašnja `while`-petlja u rekurziji izvršava spajanje dva sortirana dijela niza (lijevi i desni). Nezavisna dva (pod)niza `nizLeftSort` i `nizRightSort` se rekurzivno sortiraju, a potom spajaju (u linearnom vremenu). Rekurzivna relacija koja odgovara vremenu izvršavanja algoritma je data u Primjeru 1.12.

Primjer 3.6 (*Množenje cijelih brojeva*). Poznati algoritam za množenje dva n -cifrena broja, koji se uči u osnovnoj školi i zasniva se na množenju cifru po cifru, ima vremensku složenost $O(n^2)$. Konstruisati efikasniji al-



Slika 3.5: Sortiranje niza spajanjem pomoću Podijeli-pa-zavladaj paradigme: korak po korak

goritam zasnovan na paradigmi podijeli pa zavladaj za problem množenja dva cijela broja.

Rješenje. Radi jednostavnosti, bez smanjenja opštosti, pretpostavimo da su ulazni brojevi zapisani u binarnom obliku i da imaju isti broj cifara. Neka su dati brojevi x i y , svaki dužine n bitova. Podijelimo ih na lijevu i desnu polovinu: $x = x_1 2^{n/2} + x_2$, $y = y_1 2^{n/2} + y_2$, gdje x_1 , x_2 , y_1 i y_2 sadrže $n/2(\pm 1)$ bitova. Tada vrijedi

$$xy = (x_1 2^{n/2} + x_2) \cdot (y_1 2^{n/2} + y_2) = x_1 y_1 2^n + (x_1 y_2 + x_2 y_1) 2^{n/2} + x_2 y_2.$$

Na osnovu ove formule potrebno je izračunati četiri proizvoda brojeva dužine $\approx n/2$: $x_1 y_1$, $x_1 y_2$, $x_2 y_1$, $x_2 y_2$.

Rekurzija složenosti tada ima oblik $T(n) = 4T(n/2) + O(n)$, pa prema Master teoremi dobijamo $T(n) = O(n^{\log_2 4}) = O(n^2)$, što ne predstavlja poboljšanje u odnosu na klasični algoritam.

Da bismo smanjili broj rekurzivnih množenja, koristimo sljedeću ideju:

1. Izračunamo $[z_1 = x_1 y_1]$.

2. Izračunamo

$$z_2 = x_2 y_2.$$

3. Izračunamo $z_3 = (x_1 + x_2)(y_1 + y_2)$.

Tada važi $z_3 - z_1 - z_2 = x_1y_2 + x_2y_1$, pa se mješoviti član može dobiti bez dodatnih množenja. Prema tome, $xy = z_12^n + (z_3 - z_1 - z_2)2^{n/2} + z_2$.

Na ovaj način broj rekurzivnih množenja smanjuje se sa četiri na tri, pa rekurzija postaje $T(n) = 3T(n/2) + O(n)$. Primjenom Master teoreme dobijamo $T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$, što je asimptotski brže od klasičnog algoritma množenja (Pogledati Primjer 1.13). Ovaj pristup poznat je kao *Karacubinov algoritam* (eng. Karatsuba multiplication algorithm).

3.6 Algoritamska paradigma vraćanja unazad

Vraćanje unazad predstavlja algoritamsku paradigmu zasnovanu na rekurzivnoj izgradnji rješenja korak po korak. Tokom konstrukcije rješenja algoritam u svakom koraku provjerava da li trenutno parcijalno rješenje može biti prošireno u validno konačno rješenje. Ukoliko se utvrdi da neki izbor narušava ograničenja problema ili ne može dovesti do dopustivog rješenja, algoritam odustaje od tog pravca pretrage i vraća se na prethodnu odluku kako bi ispitao druge mogućnosti.

Ova paradigma može se posmatrati kao unaprijeđena verzija pristupa grube sile. Dok algoritmi grube sile ispituju sva moguća rješenja, algoritmi vraćanja unazad nastoje što ranije eliminisati dijelove prostora pretrage za koje se može zaključiti da ne sadrže validno rješenje. Na taj način često se postižu značajne uštede u vremenu izvršavanja.

Osnovna ideja paradigme sastoji se u tome da se rješenje gradi inkrementalno, dodavanjem jedne komponente u svakom koraku. Nakon svakog proširenja parcijalnog rješenja provjerava se da li ono i dalje zadovoljava uslove problema. Ako je odgovor negativan, pretraga se vraća na prethodni korak i pokušava drugačiji izbor.

Paradigma vraćanja unazad posebno je pogodna za probleme kod kojih:

- rješenje možemo graditi postepeno;
- nakon svakog koraka možemo provjeriti da li je parcijalno rješenje dopustivo;

- postoji veliki broj mogućih kombinacija koje treba sistematski istražiti.

Tipični primjeri primjene ove paradigme uključuju Problem osam dama, bojenje grafa, sudoku, problem ruksaka, generisanje permutacija i kombinacija sa određenim ograničenjima, te mnoge druge kombinatorne optimizacione probleme. Opšti pseudokod paradigme vraćanja unazad dat je Algoritmom 6.

Algoritam 6 Opšti obrazac paradigme vraćanja unazad.

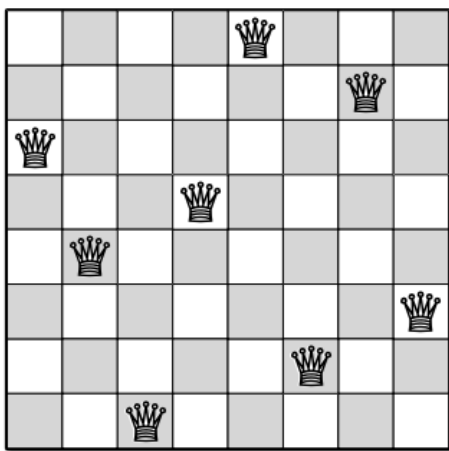
```
1: procedure BACKTRACK( $x$ )
2:   if  $x$  nije dopustivo parcijalno rješenje then
3:     return
4:   end if
5:   if  $x$  predstavlja potpuno rješenje then
6:     Sačuvaj ili obradi rješenje  $x$ 
7:     return
8:   end if
9:   for svaku moguću odluku  $e$  do
10:    BACKTRACK(proširi  $x$  sa  $e$ )
11:   end for
12: end procedure
```

Implementacija ove paradigme prirodno vodi ka konstrukciji *stabla stanja* (eng. *state-space tree*). Svaki čvor stabla predstavlja jedno parcijalno rješenje, dok grane odgovaraju mogućim proširenjima tog rješenja. Rekurzivni pozivi algoritma odgovaraju obilasku ovog stabla. Kada se utvrdi da određena grana ne dovodi do validnog rješenja, dalji obilazak te grane se prekida, čime se izbjegava nepotrebno pretraživanje.

U najgorem slučaju algoritmi vraćanja unazad i dalje mogu imati eksponencijalnu vremensku složenost, jer je ponekad potrebno istražiti veoma veliki broj mogućih kombinacija. Ipak, zahvaljujući ranom odbacivanju nedopustivih parcijalnih rješenja, oni su u praksi često znatno efikasniji od pristupa grube sile.

Primjer 3.7 (Problem N kraljica). Neka je data šahovska ploča dimenzija $N \times N$. Postaviti N kraljica na ploču tako da se nikoje dvije kraljice međusobno ne napadaju. Riješiti problem primjenom paradigme vraćanja unazad.

Rješenje. Posmatrajmo najpoznatiju varijantu ovog problema, problem osam kraljica ($N = 8$). Jedno dopustivo rješenje prikazano je na Slici 3.6.



Slika 3.6: Primjer jednog rješenja problema osam kraljica.

Pošto se dvije kraljice ne smiju nalaziti u istom redu, svako rješenje možemo predstaviti nizom dužine N . Element na poziciji i označava kolonu u kojoj je postavljena kraljica u i -tom redu šahovske ploče. Na primjer, raspored kraljica prikazan na Slici 3.6 može se predstaviti nizom $[5, 7, 1, 4, 2, 8, 6, 3]$. Vrijednost na svakoj poziciji određuje kolonu u kojoj se smiješta kraljica u odgovarajući red.

Algoritam vraćanja unazad započinje od praznog rješenja. U svakom koraku pokušava se postaviti nova kraljica u naredni red šahovske ploče. Drugim riječima, parcijalno rješenje se proširuje dodavanjem jedne nove komponente, odnosno izborom kolone u kojoj će se nalaziti kraljica u trenutno

obrađivanom redu. Nakon svakog dodavanja provjerava se da li novopostavljena kraljica napada neku od prethodno postavljenih kraljica. Konflikt postoji ukoliko se dvije kraljice nalaze:

- u istoj koloni;
- na istoj glavnoj dijagonali;
- na istoj sporednoj dijagonali.

Ako se utvrdi postojanje konflikta, trenutno parcijalno rješenje ne može biti dio nijednog dopustivog konačnog rješenja. U tom slučaju algoritam odbacuje posljednji izbor, vraća se na prethodni nivo pretrage i pokušava postaviti kraljicu u neku drugu kolonu istog reda.

Proces se nastavlja sve dok:

- ne bude konstruisano dopustivo rješenje dužine N , čime su kraljice uspješno postavljene u sve redove šahovske ploče; ili
- ne budu iscrpljene sve mogućnosti pretrage, kada zaključujemo da rješenje ne postoji.

Problem N kraljica predstavlja jedan od najpoznatijih primjera primjene paradigme vraćanja unazad, jer omogućava veoma efikasno odbacivanje velikog broja parcijalnih rješenja koja ne mogu dovesti do validnog rasporeda kraljica.

Implementacija problema je data narednim kôdom.

```
def NQueens(niz, N):  
    if len(niz) == N: # kompletno rješenje  
        return niz  
    else:  
        for col in range(N):  
            kandidat = niz + [col]  
            if valid(kandidat):
```

```

        rezultat = NQueens(kandidat, N)
        if rezultat is not None:
            return rezultat
    return None

#main
N = 8
sol = NQueens([], N)
if sol != None:
    print("Rješenje je: ", sol)
else:
    print("Nema rješenja") #npr. za N=3

```

Napomenimo da ovdje nismo prikazali implementaciju funkcije `valid()`, koja vraća *True* ako su pozicije kraljica validne, u protivnom vraća *False*. Ovo ostavljamo čitaocu kao zadatak za vježbanje.

Primjer 3.8 (*Problem sume 4 kvadrata*). Lagranžova teorema o 4 kvadrata kaže da se svaki prirodni broj može napisati kao zbir četiri broja koji su kvadrati nekih brojeva. Neka je dat broj n . Naći četiri kvadratna broja koji u zbiru daju broj n pomoću algoritma vraćanja unazad. Npr. za $n = 2000$, vrijedi $1764 + 196 + 36 + 4 = 2000$.

Rješenje. Rješenje problema predstavlja četvorka $sol = (a, b, c, d)$ prirodnih brojeva, tako da vrijedi $a^2 + b^2 + c^2 + d^2 = n$. Ideja rekursije je odabrati svaki od brojeva jedan po jedan, dok ne nađemo četvorku koja odgovara uslovima zadatka. Svaki put kada dodamo novi broj x u rješenje sol , naredni rekurzivni poziv ispituje trenutno zbir kvadrata brojeva u proširenom rješenju sol . Ako vrijednost prelazi n , algoritam vrši vraćanje unazad i bira novog kandidata za proširivanje rješenja.

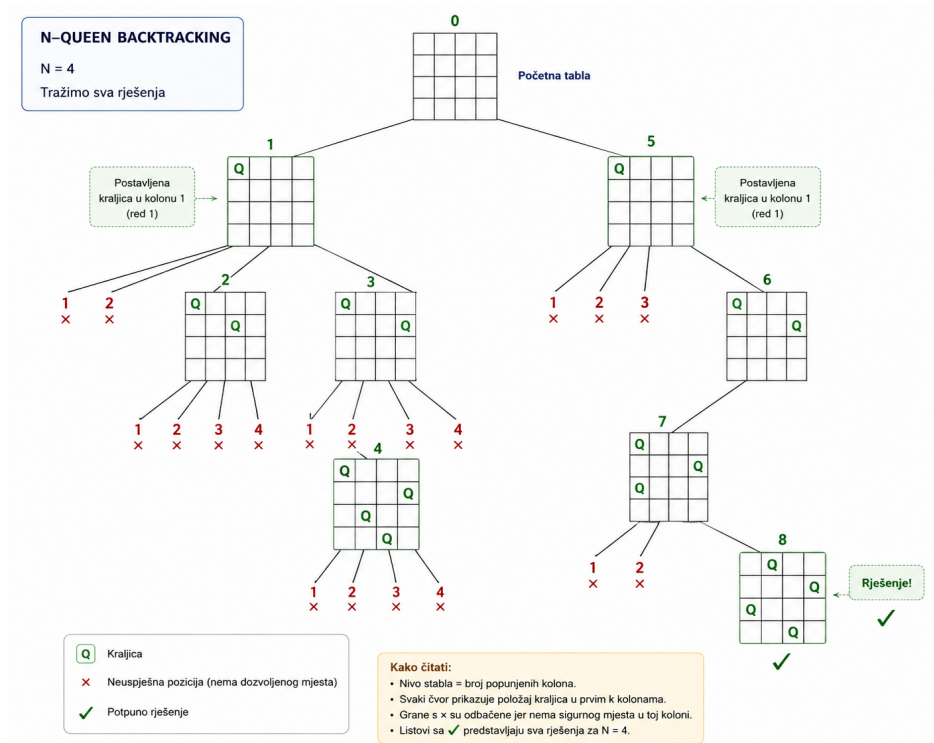
Implementacija algoritma vraćanja unazad za ovaj problem je dat narednim kôdom.

```

from math import sqrt, pow

def FourSquareSum(sol):

```



Slika 3.7: Primjer stabla stanja formiranog algoritmom vraćanja unazad za problem 4 kraljice.

```

    sum = 0
    for s in sol:
        sum += pow(s, 2)
    return sum

def FourSumSquares(sol, n):
    if len(sol) == 4:
        if FourSquareSum(sol) == n:
            print(sol)
            return
    else:
        for i in range(1, int(sqrt(n))+2):
            solI = sol + [i]
            sum_sol_i = FourSquareSum(solI)
            if sum_sol_i <= n-(4-len(solI)): # else backtrack
                FourSumSquares(solI, n)

# poziv metode
FourSumSquares([], 20)

```

Kao što primijećujemo u prethodnom kodu, ako je *sol* kardinalnosti 4, i ako je zbir kvadrata njegovih elemenata jednak *n*, ispisujemo *sol* kao rješenje. U protivnom, ako je $|sol| < 4$, (parcijalno) rješenje *sol* proširujemo ubacujući naredni broj (varijabla $i \in \{1, \dots, \lfloor \sqrt{n} \rfloor + 1\}$), dobijajući niz *solI*. Ako je zbir kvadrata elemenata iz *sol* veći od $n - (4 - |solI|)$, ovakva ekstenzija nikada neće voditi ka rješenju koje zadovoljava uslove zadatka, pa pretraga ide unazad, rekurzivno, ispitujući druge kandidate za proširenje trenutnog rješenja *sol*. Inicijalno, rješenje *sol* je predstavljeno praznim nizom.

3.7 Pohlepni algoritmi

Pohlepni algoritmi predstavljaju algoritamsku paradigmu u kojoj se u svakom koraku donosi lokalno najbolja odluka o proširenju parcijalnog rješenja prema unaprijed definisanom kriterijumu. Osnovna ideja je da se problem rješava nizom lokalno optimalnih izbora, uz pretpostavku da će takvi izbori dovesti do kvalitetnog, a ponekad i optimalnog globalnog rješenja.

Za razliku od algoritama vraćanja unazad ili dinamičkog programiranja, pohlepni algoritmi ne preispituju ranije donesene odluke. Jednom kada se neka komponenta uključi u rješenje, ona se više ne uklanja niti mijenja. Zbog toga su pohlepni algoritmi obično veoma efikasni, ali ne garantuju uvijek pronalazak optimalnog rješenja.

Drugim riječima, algoritam u svakom koraku bira opciju koja je trenutno najpovoljnija, bez razmatranja mogućih posljedica tog izbora na buduće korake. Zbog toga, uspješnost ovog pristupa zavisi od strukture problema. Za određene klase problema može se dokazati da lokalno optimalni izbori vode ka globalno optimalnom rješenju, dok kod drugih problema pohlepni pristup daje samo aproksimativna/heuristička rješenja.

U neformalnom smislu, pohlepni algoritam započinje od praznog ili djelimično izgrađenog rješenja i zatim ga iterativno proširuje komponentama koje imaju najveću trenutnu korist. Proces se nastavlja sve dok se ne dobije potpuno rješenje ili dok se ne iscrpe sve mogućnosti za dopustivo proširenje.

Opšta shema pohlepnog pristupa prikazana je Algoritmom 7. Ulaz u algoritam čine instanca problema $\mathcal{P}$ i pohlepna funkcija g , koja procjenjuje kvalitet mogućih proširenja trenutnog rješenja.

U svakoj iteraciji izvršavaju se sljedeći koraci:

- određuje se skup svih komponenti koje na dopustiv način mogu proširiti trenutno parcijalno rješenje;
- među njima se bira komponenta sa najboljom vrijednošću pohlepne funkcije;
- izabrana komponenta se dodaje trenutnom rješenju.

Proces se ponavlja sve dok postoji barem jedna komponenta koja može dopustivo proširiti trenutno rješenje. Nakon toga algoritam vraća konstruisano rješenje.

Da bi se neki problem mogao rješavati pohlepnim pristupom, potrebno je definisati:

1. skup komponenti od kojih se gradi rješenje;

Algoritam 7 Opšta shema pohlepnog algoritma.

```
1: Ulaz: instanca problema, pohlepna funkcija  $g$ 
2: Izlaz: (aproksimativno) rješenje problema
3:  $x \leftarrow$  inicijalno parcijalno rješenje
4:  $\mathcal{C} \leftarrow$  skup dopustivih proširenja rješenja  $x$ 
5: while  $\mathcal{C} \neq \emptyset$  do
6:    $e \leftarrow$  lokalno najbolja komponenta iz  $\mathcal{C}$  shodno vrijednosti funkc.  $g$ 
7:    $x \leftarrow$  proširi rješenje  $x$  komponentom  $e$ 
8:    $\mathcal{C} \leftarrow$  odredi nova dopustiva proširenja rješenja  $x$ 
9: end while
10: return  $x$ 
```

2. reprezentaciju parcijalnog rješenja;
3. kriterijum za izbor sljedeće komponente, odnosno pohlepnu funkciju;
4. uslove dopustivosti proširenja parcijalnog rješenja.

Najpoznatiji primjeri primjene pohlepnih algoritama uključuju problem izbora aktivnosti, konstrukciju Hafmanovog stabla, algoritme za pronalaženje minimalnog razapinjućeg stabla (Kruskalov i Primov algoritam), te određene varijante problema najkraćih puteva, kao što je Dijkstrin algoritam za grafove sa nenegativnim težinama grana.

Glavna prednost pohlepnih algoritama jeste njihova jednostavnost implementacije i mala vremenska složenost. Međutim, njihov osnovni nedostatak je što lokalno najbolji izbor ne mora uvijek voditi globalno optimalnom rješenju. Zbog toga je prije primjene ove paradigme potrebno dokazati da problem posjeduje tzv. *pohlepno svojstvo* (eng. *greedy-choice property*), odnosno da optimalno rješenje može biti izgrađeno nizom lokalno optimalnih odluka.

Ovu paradigmu koristimo kada je potrebno pronaći rješenje zadovoljavajućeg kvaliteta u relativno kratkom vremenu, uz jednostavnu implementaciju algoritma. Zbog svoje efikasnosti i konceptualne jednostavnosti, pohlepni

pristup često predstavlja prvi korak u rješavanju složenih kombinatornih i optimizacionih problema.

Pohlepni algoritmi se takođe često koriste za generisanje početnih rješenja koja se kasnije dodatno poboljšavaju naprednijim metodama, kao što su lokalna pretraga, metaheuristike ili egzaktni algoritmi. U tom smislu, oni pružaju korisnu polaznu osnovu za razumijevanje strukture problema i procjenu kvaliteta rješenja koja se mogu dobiti korištenjem jednostavnih konstruktivnih heuristika.

U nastavku prikazujemo nekoliko primjera primjene pohlepnih algoritama na različite kombinatorne probleme.

Primjer 3.9 (Problem razmjenjivanja novca).

Neka je dato X KM i skup apoena $x_1, x_2, \dots, x_k$, pri čemu je svaki apoen dostupan u neograničenoj količini. Pretpostavimo da su X i svi apoeni prirodni brojevi.

Na koji način razmijeniti iznos X tako da ukupan broj novčanica i kovanica bude minimalan? Problem riješiti primjenom pohlepnog pristupa.

Rješenje. Definišimo najprije strukturu rješenja. Svako rješenje može se predstaviti k -torkom $(p_1, p_2, \dots, p_k)$, gdje p_i označava broj iskorištenih apoena vrijednosti x_i . Rješenje je dopustivo ukoliko vrijedi $\sum_{i=1}^k p_i x_i = X$.

Parcijalno rješenje predstavlja djelimično formiranu kombinaciju apoena $(p_1, p_2, \dots, p_s)$, $s \leq k$, za koju vrijedi $\sum_{i=1}^s p_i x_i \leq X$. Takvo rješenje se može dalje proširivati dodavanjem novih apoena.

Osnovna ideja pohlepnog algoritma jeste da se u svakom koraku bira najveći apoen koji ne premašuje preostali iznos koji treba razmijeniti. Nakon izbora apoena uzima se maksimalan mogući broj novčanica ili kovanica tog apoena, a zatim se isti postupak ponavlja za preostali iznos.

Pretpostavimo da su apoeni sortirani opadajuće: $x_1 > x_2 > \dots > x_k$.

Neka je $X' = X - \sum_{j=1}^s p_j x_j$ preostali iznos koji još treba razmijeniti. Tada se bira najveći apoen x_i za koji važi $x_i \leq X'$, a broj njegovih pojavljivanja određuje se kao $p_i = \left\lfloor \frac{X'}{x_i} \right\rfloor$. Na taj način parcijalno rješenje se

proširuje novom komponentom, nakon čega se ažurira vrijednost preostalog iznosa. Postupak se ponavlja sve dok preostali iznos ne postane nula.

Implementacija algoritma ovog problema je data u nastavku.

```
def razmijeni(P, X):
    rjesenje = []
    i = 0
    usitnjeno, brojNovcanica = 0, 0
    sortApoeni = sorted(P) # sort apoene opadajuće
    while i < n and usitnjeno < X :
        pi = (X - usitnjeno) // P[i] #br. novcanica sortApoeni[i]
        if pi > 0:
            usitnjeno = usitnjeno + pi * sortApoeni[i]
            brojNovcanica = brojNovcanica + pi
        i = i + 1
    if usitnjeno == X:
        return brojNovcanica
    else:
        return -1 # ne može se usitniti
```

Vremenska složenost. Sortiranje algoritma zahtijeva $O(n \log n)$ vremena. Dalje, glavna petlja se izvršava u $O(n)$ vremenu, odakle slijedi da je ukupno vrijeme izvršavanja algoritma jednako $O(n \log n)$, što je, svakako, polinomijalno. Za kanonske sisteme apoena, kakvi se koriste u većini standardnih valuta, pohlepni algoritam daje optimalno rješenje.

Primjer 3.10 (Razlomljeni problem ruksaka). *Ovaj problem predstavlja relaksiranu verziju klasičnog problema ruksaka. Za razliku od standardne varijante, gdje se svaki proizvod mora uzeti u cijelosti ili se uopšte ne uzima, ovdje je dozvoljeno uzeti proizvoljan dio proizvoda. Takva pretpostavka ima smisla za proizvode koji se mogu dijeliti, kao što su šećer, brašno, ulje ili gorivo.*

Dat je skup proizvoda, pri čemu svaki proizvod i ima težinu w_i i vrijednost p_i , kao i ruksak kapaciteta C . Potrebno je odrediti koje proizvode, odnosno koje njihove dijelove, treba smjestiti u ruksak tako da ukupna vrijednost bude maksimalna, uz poštovanje ograničenja kapaciteta.

Problem riješiti primjenom pohlepnog pristupa.

Rješenje. Posmatrajmo instancu problema $n = 3, w = [10, 20, 30], p = [60, 100, 120]$, pri čemu je kapacitet ruksaka jednak $C = 50$.

Optimalno rješenje dobija se tako što se prvi i drugi proizvod uzimaju u potpunosti, dok se od trećeg proizvoda uzima samo dio. Ukupna težina prva dva proizvoda iznosi $10 + 20 = 30$, pa u ruksaku ostaje još 20 jedinica slobodnog prostora. Kako treći proizvod ima težinu 30, moguće je uzeti samo njegov dio $\frac{20}{30} = \frac{2}{3}$. Ukupna vrijednost ruksaka tada iznosi $60 + 100 + \frac{2}{3} \cdot 120 = 240$.

Struktura rješenja. Rješenje predstavljamo vektorom $c = (c_1, c_2, \dots, c_n)$, gdje je $0 \leq c_i \leq 1$ udio proizvoda i koji se nalazi u ruksaku. Pri tome mora važiti ograničenje kapaciteta $\sum_{i=1}^n c_i w_i \leq C$.

Parcijalna rješenja i komponente rješenja. Parcijalno rješenje čini bilo koji skup već odabranih proizvoda i njihovih udjela koji ne narušavaju kapacitet ruksaka. Komponente rješenja predstavljaju proizvodi koji još nisu razmatrani i koji mogu dopustivo proširiti trenutno rješenje.

Pohlepna funkcija. Ključna ideja je da prednost dajemo proizvodima koji ostvaruju najveću vrijednost po jedinici težine. Za svaki proizvod računamo omjer $\rho_i = \frac{p_i}{w_i}$. Vrijednost ρ_i predstavlja profit ostvaren po jedinici zauzetog kapaciteta i prirodno određuje prioritet proizvoda. Najprije se sortiraju proizvode opadajuće prema vrijednosti ρ_i . Neka je nakon sortiranja redoslijed proizvoda označen indeksima $1, 2, \dots, n$.

Algoritam zatim prolazi kroz proizvode tim redoslijedom. Za svaki proizvod uzima se najveći mogući dio koji još može stati u preostali kapacitet ruksaka. Ako proizvod može stati u cijelosti, postavlja se $c_i = 1$. U suprotnom, uzima se upravo onaj dio koji popunjava preostali kapacitet: $c_i = \frac{\text{preostali kapacitet}}{w_i}$. Nakon toga je ruksak popunjen i algoritam se završava.

Pohlepni algoritam za razlomljeni problem ruksaka ima vremensku složenost $O(n \log n)$, koja potiče od sortiranja proizvoda (prema omjeru $\frac{p_i}{w_i}$). Za razliku od klasičnog problema ruksaka, kod ove relaksirane varijante može se dokazati da opisani pohlepni pristup uvijek daje optimalno rješenje.

Implementacija algoritma je data naredim kôdom.

```
def fraction(i):
    return P[i] / W[i]

def fraction_knapsack():

    products = [i for i in range(len(W))] #lsit of prods
    products.sort(key = fraction, reverse=True)
    #print(products)
    current_C = C
    sol, parts = [], []

    for i in products:

        part = min(current_C / W[i], 1)
        if part > 0:
            sol.append(i)
            parts.append(part)
            current_C -= part * W[i]

        else: # knapsack filled
            return sol, parts

    return sol, parts

#instanca:
W = [10, 20, 30]
P = [60, 100, 120]
C = 50
sol, parts = fraction_knapsack()
print(sol, " parts: ", parts)
```

Kompleksnost algoritma. Najviše vremena je utrošeno na sortiranje proizvoda, što iznosi $O(n \log n)$ vremena. Dalje, u glavnoj petlji, koja se vrti najviše n puta, svaka iteracija se izvršava u konstantnom vremenu. Dakle, vrijeme izvršavanja petlje je $O(n)$. Prema tome, algoritam se izvršava u $O(n \log n)$ vremenu.

Primjer 3.11 (Problem pokrivanja skupa). *Neka je dat konačan skup X i familija njegovih podskupova $\mathcal{F} \subseteq P(X)$, pri čemu važi $\bigcup_{F \in \mathcal{F}} F = X$.*

Potrebno je pronaći podfamiliju $S \subseteq \mathcal{F}$ minimalne kardinalnosti takvu da vrijedi $\bigcup_{A \in S} A = X$.

Drugim riječima, cilj je odabrati što manji broj skupova iz familije $\mathcal{F}$ tako da njihova unija pokrije sve elemente skupa X . Problem riješiti primjenom pohlepnog pristupa.

Rješenje. Posmatrajmo instancu $X = \{1, 2, 3, 4, 5\}$, i familiju skupova $\mathcal{F} = \{\{1, 2, 3\}, \{1, 4\}, \{2, 4, 5\}, \{2, 3\}, \{4, 5\}, \{1, 5\}\}$.

Jedno optimalno rješenje je $S = \{\{1, 2, 3\}, \{2, 4, 5\}\}$, jer unija skupova u rješenju pokriva čitav skup X , a pri tome je $|S| = 2$.

Problem pokrivanja skupa ima brojne praktične primjene. Na primjer, može predstavljati zadatak izbora minimalnog broja stručnjaka u komisiji tako da svaka potrebna oblast bude zastupljena barem jednim članom komisije. Slične primjene javljaju se u planiranju mreža senzora, raspoređivanju resursa i izboru lokacija za pružanje usluga.

Struktura rješenja. Rješenje problema je predstavljeno podfamilijom skupova $S \subseteq \mathcal{F}$ takva da njihova unija pokriva sve elemente skupa X .

Parcijalna rješenja i komponente rješenja. Parcijalno rješenje predstavlja proizvoljna podfamilija $S \subseteq \mathcal{F}$, koja ne mora nužno pokrivati čitav skup X . Komponente koje mogu proširiti trenutno rješenje jesu svi skupovi iz familije $\mathcal{F} \setminus S$. Proširenje parcijalnog rješenja vrši se dodavanjem novog skupa: $S' = S \cup C, C \in \mathcal{F} \setminus S$.

Pohlepna funkcija. Neka trenutno parcijalno rješenje S pokriva skup elemenata $X_S = \bigcup_{A \in S} A$. Preostali nepokriveni elementi dati su skupom $X \setminus X_S$. Intuitivno, najbolji izbor u datom koraku jeste skup koji pokriva najveći broj trenutno nepokrivenih elemenata. Zbog toga definišemo pohlepnu funkciju

$$g(S, C) = |C \cap (X \setminus X_S)|,$$

koja mjeri broj novih elemenata koje skup C doprinosi trenutnom rješenju. U svakoj iteraciji bira se skup $C^* = \arg \max_{C \in \mathcal{F} \setminus S} g(S, C)$, odnosno skup koji pokriva najveći broj još nepokrivenih elemenata.

Pohlepni algoritam započinje praznim rješenjem $S = \emptyset$. U svakoj iteraciji bira skup C^* koji maksimizuje funkciju g , dodaje ga u rješenje i ažurira skup pokrivenih elemenata. Postupak se ponavlja sve dok svi elementi skupa X ne budu pokriveni ili ne postoji više skupova koji se mogu dodati u rješenje. Važno je naglasiti da opisani pohlepni algoritam ne garantuje uvijek optimalno rješenje. Ipak, zbog svoje jednostavnosti i efikasnosti veoma je često korišten u praksi i predstavlja jednu od najpoznatijih aproksimativnih metoda za problem pokrivanja skupa.

Radi efikasnije implementacije, umjesto samih skupova u rješenju se čuvaju odgovarajući indeksi unutar familije $\mathcal{F}$.

Implementacija algoritma je data u nastavku sljedećim kôdom.

```
def g(X_S, C):
    return len((X.difference(X_S)).intersection(C))

def cover_X():
    X_S = set()
    S = [] # solution

    while(X_S != X):
        C_star = istar = None
        g_best = 0

        for i in range(len(F)):# iteration best-next
            if i not in S:
                g_i = g(X_S, F[i])
                if g_best < g_i:
                    g_best = g_i
                    i_star = i
        X_S = X_S.union(F[i_star]) #update cover
        S.append(i_star)

    return S

#instanca:
X = {1, 2, 3, 4, 5}
F = [{1, 2, 3}, {1, 4}, {2, 4, 5}, {2, 3}, {4, 5}, {1, 5}]
sol = []
C = 50
```

```
sol = cover_X()
print(sol)
```

Vremenska složenost. Glavna **while**-petlja izvršava najviše n iteracija. U unutrašnjoj **for**-petlji, koja se izvršava $|\mathcal{F}|$ puta, najskuplja operacija je poziv pohlepne funkcije g , koja se izvršava u linearnom vremenu, tj. u $O(n)$. Dakle, kompleksnost čitave **for** petlje je $O(n \cdot |\mathcal{F}|)$. Zajedno, kompleksnost algoritma je $O(n^2 \cdot |\mathcal{F}|)$. Na osnovu dobijenog, zaključujemo da efikasnost algoritma najviše zavisi od veličine skupa $\mathcal{F}$.

Napomenimo da ovako dizajniran algoritam ne garantuje nalazak optimalnog rješenja kao u prethodna dva problema. Uzmimo npr. instancu $X = \{1, 2, 3, 4, 5, 6\}$, te

$$\mathcal{F} = \{\{1, 2\}, \{3, 4\}, \{4, 5\}, \{3, 6\}, \{5\}, \{6\}\}.$$

Prva iteracija u algoritmu će odabrati, prvi skup $\{1, 2\}$ i dodati ga u parcijalno rješenje. Naredna iteracija odabire skup $\{3, 4\}$. Nakon toga, bira se $\{4, 5\}$, pa $\{6\}$. Dakle, dobijeno rješenje je kardinalnosti 4. Međutim, ono očigledno ne predstavlja optimalno rješenje.

Optimalno rješenje je veličine 3 i dato je sa $\{\{1, 2\}, \{3, 6\}, \{4, 5\}\}$.

Primjer 3.12 (Problem sekvencijalnog raspoređivanja poslova).

Dat je skup poslova, pri čemu je za svaki posao poznat krajnji rok izvršavanja (eng. deadline) i profit koji se ostvaruje ukoliko posao bude završen najkasnije do tog roka. Pretpostavimo da je za izvršavanje svakog posla potrebno tačno jedna jedinica vremena, te da svi poslovi mogu započeti od trenutka $t = 0$.

Potrebno je odabrati podskup poslova i odrediti njihov raspored izvršavanja tako da ukupan ostvareni profit bude maksimalan. Pri tome vrijedi ograničenje da se u svakoj jedinici vremena može izvršavati najviše jedan posao. Problem riješiti primjenom paradigme pohlepnih algoritama.

Rješenje. Posmatrajmo sljedeću instancu problema.

<i>Posao</i>	<i>Krajnji rok</i>	<i>Profit</i>
1	4	20
2	1	10
3	1	40
4	1	30

Optimalno rješenje u ovom slučaju čine poslovi $\{3, 1\}$, čiji je ukupan profit jednak $40 + 20 = 60$.

Posmatrajmo sada drugu instancu problema.

<i>Posao</i>	<i>Krajnji rok</i>	<i>Profit</i>
1	2	100
2	1	19
3	2	27
4	1	25
5	3	15

Jedno optimalno rješenje dobija se izvršavanjem poslova $\{3, 1, 5\}$. Poslovi se mogu rasporediti u vremenske intervale tako da poštuju zadate rokove, pri čemu je ukupan profit $27 + 100 + 15 = 142$.

Prva ideja za rješavanje ovog problema bila bi potpuna enumeracija svih podskupova poslova. Za svaki generisani podskup provjerila bi se dopustivost rasporeda i izračunao ostvareni profit. Međutim, broj mogućih podskupova jednak je 2^n , pa takav pristup postaje neupotrebljiv već za umjerene vrijednosti n .

Zbog toga razmatramo pohlepni pristup.

Struktura rješenja. Rješenje problema predstavlja skup odabranih poslova zajedno sa njihovim rasporedom izvršavanja. Rješenje je dopustivo ako:

- svaki odabrani posao bude završen najkasnije do svog krajnjeg roka;
- u svakom vremenskom trenutku bude izvršavan najviše jedan posao.

Parcijalno rješenje i njegovo proširenje. Parcijalno rješenje predstavlja bilo koji dopustiv raspored dijela poslova. Komponente koje mogu proširiti parcijalno rješenje jesu poslovi koji još nisu raspoređeni, a koji se mogu dodati bez narušavanja ograničenja problema. Proširenje parcijalnog rješenja podrazumijeva dodavanje novog posla u neki slobodan vremenski interval tako da njegov krajnji rok ostane zadovoljen. Recimo, slobodan vremenski interval bi mogao biti onaj koji je najkasniji mogući a ne narušava ni krajnji rok izvršavanja konkretnog posla, niti postoji konflikt između poslova koji bi se simultano izvršavali u istom vremenskom intervalu.

Pohlepna funkcija. Intuitivno je razumno dati prednost poslovima koji donose veći profit. Zbog toga se poslovi najprije sortiraju po profitu u opadajućem poretku.

Neka je trenutno parcijalno rješenje označeno sa I_s . U svakoj iteraciji bira se neraspoređeni posao sa najvećim profitom. Ukoliko je moguće pronaći slobodan vremenski interval prije krajnjeg roka, posao se dodaje u raspored. U suprotnom, posao se odbacuje i razmatra se naredni posao iz sortirane liste. Da bi se ostavilo što više prostora za buduće poslove, odabrani posao se smiješta u *posljednji* (najkasniji) slobodan vremenski interval koji ne prelazi njegov krajnji rok.

Algoritam se može opisati sljedećim koracima:

1. Sortirati poslove po profitu u opadajućem poretku.
2. Poštujući redoslijed sortiranja, idući od posla do posla, pronaći (najkasniji) slobodan vremenski interval prije krajnjeg roka.
3. Ako takav interval postoji, posao rasporediti u njega.
4. U suprotnom, posao odbaciti i preći na drugi.
5. Nakon razmatranja svih poslova vratiti dobijeni raspored.

Pohlepni algoritam za ovaj problem izvršava se znatno brže od potpune enumeracije, a pritom daje optimalno rješenje. Ključna ideja sastoji se u

tome da se najprofitabilniji poslovi raspoređuju što je moguće kasnije, čime se ostavlja maksimalna fleksibilnost za raspoređivanje preostalih poslova.

Implementacija algoritma je data sljedećim kôdom.

```
def ProfitSort(i):
    return Profit[i]

# možemo li dodati i-ti proizvod ili ne (-1) u I_s
def find_next_interval(I_s_covered, i):

    for index, covered in enumerate(I_s_covered):
        rev_index = len(I_s_covered)-index
        if rev_index > Deadline[i]:
            return continue
        if covered == False:
            return rev_index
    return -1

def sequetial_scheduling():

    I_s_covered = [False] * max(Deadline)
    I_s = []
    #sortiranje jobs u odnosu na Profit (preprocessing):
    Jobs.sort(key=ProfitSort, reverse=True)

    for i in Jobs:

        interval = find_next_interval(I_s_covered, i)
        if interval >= 0: #nadjena (dopustiv) interval:
            I_s_covered[interval] = True #zauzet
            I_s.append(i)

    return I_s, sum([ Profit[I_s[i]]  for i in range(len(I_s)) ])

#main:
num_jobs = 5
Jobs = [i for i in range(num_jobs)]
Deadline = [2, 1, 2, 1, 3]
Profit = [100, 19, 27, 25, 15]
```

```
# izvršavanje algoritma:
I_s, profit = sequetial_scheduling()
print(I_s, " profit: ", profit)
```

Vremenska kompleksnost. Sortiranje poslova se odvija u $O(n \log n)$. Dalje, glavna **while**-petlja se sastoji od n iteracija. U svakoj iteraciji, najskuplja operacija je poziv funkcije `find_next_interval` koja se izvršava u vremenu $O(n)$. Prema tome, glavna petlja se izvršava u $O(n^2)$. Zaključujemo da se implementacija algoritma izvršava u $O(n^2)$.

Napomenimo da se može pokazati da ovaj pohlepni algoritam garantuje nalazak optimalnog rješenja.

3.8 Dinamičko programiranje

Dinamičko programiranje predstavlja algoritamsku tehniku namijenjenu efikasnom rješavanju problema koji imaju dvije ključne karakteristike:

- *preklapajuće podprobleme* (eng. *overlapping subproblems*);
- *optimalnu podstrukturu* (eng. *optimal substructure*).

Pod *preklapajućim podproblemima* podrazumijevamo situaciju u kojoj se isti podproblem javlja više puta tokom rješavanja većeg problema. Umjesto da se rješenje podproblema svaki put iznova računa, ono se jednom izračuna i rezultat se čuva za kasniju upotrebu. Svojstvo *optimalne podstrukture* znači da se optimalno rješenje početnog problema može konstruisati kombinovanjem optimalnih rješenja njegovih podproblema. Drugim riječima, rješavanje podproblema na optimalan način vodi ka optimalnom rješenju originalnog problema.

Osnovna ideja dinamičkog programiranja sastoji se u izbjegavanju višestrukog rješavanja istih podproblema. Umjesto ponovnog računanja, rezultati se pohranjuju u memoriju i po potrebi ponovo koriste. Ovaj postupak naziva se *memoizacija*.

Memoizacija značajno poboljšava efikasnost rekurzivnih algoritama, jer se svaki podproblem rješava najviše jednom. Naravno, cijena ovog ubrzanja

jeste dodatni memorijski prostor potreban za pohranjivanje već izračunatih rezultata.

Da bismo primijenili dinamičko programiranje na određeni problem, potrebno je:

- formulirati problem u obliku rekurzivne relacije koja ga razlaže na manje podprobleme;
- identifikovati podprobleme čija se rješenja ponavljaju;
- sačuvati rješenja podproblema u odgovarajućoj memorijskoj strukturi radi njihove ponovne upotrebe.

3.8.1 Memoizacija (pristup odozgo prema dolje)

Memoizacija (eng. *top-down approach*) predstavlja implementaciju dinamičkog programiranja zasnovanu na rekurziji. Algoritam se izvršava na isti način kao i obična rekurzija, ali se rezultati već riješenih podproblema pohranjuju u pomoćnu strukturu podataka (npr. rječnik ili tabelu).

Prilikom svakog novog poziva prvo se provjerava da li je odgovarajući podproblem već riješen. Ukoliko jeste, prethodno izračunato rješenje se odmah vraća bez dodatnog računanja.

Prednosti memoizacije su:

- jednostavna implementacija kada je rekurzivna formulacija problema prirodna;
- računanje samo onih podproblema koji su zaista potrebni.

Nedostatak je dodatni trošak rekurzivnih poziva i memorije steka.

Tabuliranje (pristup odozdo prema gore)

Drugi način implementacije dinamičkog programiranja naziva se *tabuliranje* (eng. *tabulation*) ili pristup *odozdo prema gore* (eng. *bottom-up approach*).

Kod ovog pristupa problem se ne rješava rekurzivno. Umjesto toga, prvo se izračunavaju rješenja najmanjih podproblema, a zatim se ona sistematski koriste za konstruisanje rješenja većih podproblema, sve do konačnog rješenja. Rezultati se pohranjuju u tabelu (najčešće niz ili matricu), po čemu je ovaj pristup i dobio naziv. Prednosti tabuliranja su:

- nema troška rekurzivnih poziva;
- nema opasnosti od prekoračenja steka;
- često postiže bolje praktične performanse od memoizacije.

Sa druge strane, tabuliranje ponekad zahtijeva računanje i onih podproblema koji možda nisu neophodni za konačno rješenje.

Možemo zaključiti da memoizacija i tabuliranje predstavljaju dva različita načina implementacije iste ideje: izračunati svaki podproblem najviše jednom i sačuvati njegovo rješenje za kasniju upotrebu.

U nastavku ćemo razmotriti problem izračunavanja n -tog Fibonačijevog broja kako bismo ilustrovali razliku između memoizacije i tabuliranja, te pokazali kako dinamičko programiranje može dramatično smanjiti vrijeme izvršavanja u odnosu na klasičnu rekurziju.

```
def fib(n, cache={}): # memoizacija
    if n in cache:
        return cache[n]
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        cache[n] = fib(n-1) + fib(n-2)
        return cache[n]

def fib(n): # tabuliranje
    if n == 0:
        return 0
    elif n == 1:
```

```
        return 1
    else:
        tab = [0] * (n + 1) #init
        tab[0] = 0
        tab[1] = 1
        for i in range(2, n+1):
            tab[i] = tab[i-1] + tab[i-2]
        return tab[n]
```

U implementaciji memoizacije koristimo objekat klase `dict` naziva *cache* za čuvanje rezultata poziva funkcija, a rekurzija se koristi za izračunavanje rezultata. U implementaciji tabele koristimo niz nazvan *tab* za čuvanje rezultata podproblema, a koristimo iterativnu implementaciju za izračunavanje rezultata. Obje implementacije vraćaju isti rezultat, ali su zasnovane na drugačijoj implementaciji.

Da zaključimo, princip dinamičkog programiranja ima smisla koristiti gdje god je prepoznato da rekurzivni pristup ima ponovljive pozive za iste ulaze, redukujući ih uz pomoć ove algoritamske paradigme.

3.8.2 Rekurzija naspram dinamičkog programiranja

Dinamičko programiranje se najčešće primjenjuje na probleme koji se prirodno mogu formulisati rekurzivno. U velikom broju optimizacionih problema prvi korak jeste definisanje odgovarajuće rekurzivne relacije, dok se dinamičko programiranje koristi za njenu efikasnu implementaciju.

Međutim, nije svaki rekurzivni algoritam pogodan za primjenu dinamičkog programiranja. Ključni uslov jeste postojanje *preklapajućih podproblema*. Ukoliko se isti podproblem pojavljuje više puta tokom izvršavanja algoritma, tada ima smisla sačuvati njegovo rješenje i koristiti ga pri narednim pozivima.

Tipičan primjer takve situacije jeste rekurzivno računanje Fibonačijevih brojeva. Na primjer, pri izračunavanju vrijednosti $F(6)$ podproblem $F(4)$ se pojavljuje više puta, pa njegovo ponovno računanje predstavlja nepotreban trošak. Dinamičko programiranje eliminiše ovakva višestruka izračunavanja

pohranjivanjem već dobijenih rezultata.

Sa druge strane, postoje rekurzivni algoritmi kod kojih se podproblemi ne preklapaju. U takvim slučajevima dinamičko programiranje ne donosi značajno ubrzanje. Primjer je algoritam *sortiranja spajanjem* (tzv. *merge sort*), koji niz dijeli na dva disjunktna podniza. Svaki podproblem se rješava tačno jednom, pa nema potrebe za memorisanjem međurezultata.

Možemo zaključiti da je dinamičko programiranje najkorisnije kada su ispunjena dva uslova:

- problem posjeduje optimalnu podstrukturu;
- tokom rješavanja dolazi do ponavljanja rješavanja istih podproblema.

U suprotnom, klasična rekurzija ili paradigma *podijeli pa zavlada* često predstavljaju prikladnije rješenje.

3.8.3 Primjene dinamičkog programiranja

Primjer 3.13 *Neka je data kvadratna matrica $A = (a_{ij})$ prirodnih brojeva. Pijun se nalazi u gornjem lijevom uglu matrice i može se kretati:*

- *jedno polje prema dolje;*
- *jedno polje dijagonalno prema dolje-desno.*

Odrediti putanju od početne pozicije do nekog polja u posljednjem redu matrice tako da suma vrijednosti na svim posjećenim poljima bude maksimalna.

Rješenje. Na Slici 3.8 prikazan je primjer jedne dopustive putanje kretanja pijuna.

5	x	x	x
2	4	x	x
7	9	2	x
7	7	6	7

Slika 3.8: Primjer putanje pijuna. Ukupna vrijednost označene putanje iznosi 25.

Problem posjeduje optimalnu podstrukturu, jer optimalna putanja do polja (i, j) mora sadržavati optimalnu putanju do jednog od polja iz kojih je moguće doći u (i, j) .

Definišimo matricu M tako da vrijednost $M(i, j)$ predstavlja maksimalnu sumu koja se može ostvariti na putanji od početnog polja $(0, 0)$ do polja (i, j) .

Da bismo izračunali vrijednost $M(i, j)$, posmatramo dva moguća prethodna polja:

- polje $(i - 1, j)$;
- polje $(i - 1, j - 1)$.

Ako je pijun stigao iz jednog od tih polja, tada važi rekurzivna relacija:

$$M(i, j) = a_{ij} + \max\{M(i - 1, j), M(i - 1, j - 1)\}. \quad (3.1)$$

Bazni slučaj definisan je sa:

$$M(0, 0) = a_{00}. \quad (3.2)$$

Za pozicije koje nisu dostupne pijunu (npr. izvan granica matrice) možemo postaviti vrijednost $-\infty$, kako nikada ne bi bile odabrane kao dio optimalne putanje. Nakon popunjavanja matrice M , optimalna vrijednost rješenja dobija se kao:

$$\max_{0 \leq j < n} M(n-1, j), \quad (3.3)$$

odnosno kao najveća vrijednost u posljednjem redu matrice M . Na ovaj način svaki podproblem rješava se tačno jednom, pa je vremenska složenost algoritma jednaka $O(n^2)$, dok je potrební memorijski prostor takođe $O(n^2)$.

Implementacija ovog rješenje je data sljedećim kôdom (pristup odozgo prema dolje).

```
A = None #matrica sa svim -1
def init(n):

    A = [ [-1] * n for _ in range(n) ]
    A[0][0] = M[0][0] #bazni slučaj

    def solve(k, l):# najbolji put od (0.0) do (k, l)

        for i in range(1, k):
            for j in range(0, l):
                best = A[i-1][j]
                if j>0:
                    best = max(best, A[i-1][j-1])
                A[i][j] = best + A[i][j]

        return A[k][l]

    #instanca
    M=[[4, 5, 7, 0],
        [2, 3, 1, 3],
        [1, 1, 3, 1],
        [2, 3, 3, 3]
    ]

    max_sum_path = 0
    for j in range(len(M)):
        init(len(A))
        A = solve(len(A)-1, j)
        max_sum_path = max(max_sum_path, A[len(A)-1, j])
```

```
#ispis rezultata
print(max_sum_path)
```

Kompleksnost algoritma. Funkcija `solve()` se izvršava u vremenu od $O(n^2)$ kao najzahtjevniji dio implementacije. Spoljašnja petlja se izvršava n puta. Prema tome, ukupna kompleksnost algoritma je $O(n^2 \cdot n) = O(n^3)$.

Primjer 3.14 *Riješiti problem ruksaka primjenom dinamičkog programiranja.*

Rješenje. Posmatrajmo klasični problem ruksaka (0–1 ruksak). Ponovimo, neka je dato n proizvoda, pri čemu proizvod i ima težinu w_i sa vrijednosti p_i , $i = 1, \dots, n$. Kapacitet ruksaka iznosi C .

Definisanje podproblema. Neka $DP[i][j]$ čuva optimalnu vrijednost koju je moguće ostvariti korištenjem prvih i proizvoda, za raspoloživi kapacitet ruksaka jednak j . Dakle, podproblem je određen parom indeksa (i, j) , $i \in \{0, 1, \dots, n\}$, $j \in \{0, 1, \dots, C\}$.

Rekurzivna relacija. Posmatrajmo podproblem (i, j) . Postoje dva moguća slučaja:

1. Proizvod i ne ulazi u optimalno rješenje. Tada je $DP[i][j] = DP[i - 1][j]$.
2. Proizvod i ulazi u optimalno rješenje. To je moguće samo ako je njegova težina manja ili jednaka raspoloživom kapacitetu, odnosno ako važi $w_i \leq j$. U tom slučaju ostvarujemo vrijednost p_i , a preostali kapacitet postaje $j - w_i$. Tada je

$$DP[i][j] = \max\{DP[i - 1][j - w_i] + p_i, DP[i - 1][j]\}.$$

Birajući bolje od ova dva rješenja, dobijamo rekurzivnu relaciju:

$$DP[i][j] = \begin{cases} \max(DP[i - 1][j], DP[i - 1][j - w_i] + p_i), & \text{ako } w_i \leq j, \\ DP[i - 1][j], & \text{ako } w_i > j. \end{cases}$$

Bazni slučajevi. Ako je kapacitet ruksaka jednak nuli, nijedan proizvod ne može biti odabran, pa vrijedi $DP[i][0] = 0, i = 0, \dots, n$. Takođe, ako ne razmatramo nijedan proizvod, maksimalna ostvariva vrijednost jednaka je nuli bez obzira na kapacitet: $DP[0][j] = 0, j = 0, \dots, C$.

Konačno rješenje. Nakon popunjavanja matrice DP, optimalna vrijednost problema nalazi se u ćeliji $DP[n][C]$. Drugim riječima, ona predstavlja najveću moguću vrijednost koju možemo smjestiti u ruksak kapaciteta C razmatrajući svih n proizvoda.

Složenost algoritma. Matrica DP sadrži $(n + 1)(C + 1)$ elemenata, a svaka ćelija se računa u konstantnom vremenu. Zbog toga je vremenska složenost algoritma $O(n \cdot C)$, dok je prostorna složenost takođe $O(n \cdot C)$.

Implementacija paradigme dinamičkog programiranja za rješavanje problema ruksaka je data narednim kôdom.

```
def knapsack(n, C, W, p):
    DP = [[0 for x in range(C + 1)] for x in range(n + 1)]

    # tabuliranje
    for i in range(n + 1):
        for j in range(C + 1):
            if i == 0 or j == 0:
                DP[i][j] = 0
            elif W[i-1] <= j:
                DP[i][j] = max(p[i-1]
                               + DP[i-1][j-W[i-1]],
                               DP[i-1][j])
            else:
                DP[i][j] = DP[i-1][j]

    return DP[n][C]

#instanca:
p = [60, 100, 120]
W = [10, 20, 30]
C = 50
```

```

n = len(p)
#poziv funkcije
print knapsack(n, C, W, p, n)

```

Kompleksnost algoritma. Najskuplji dio algoritma su dvije ugnježdene for petlje, koje izvršavaju $O(n \cdot C)$ iteracija. Svaka od iteracija se izvršava u konstantnom vremenu, pa je i ukupna kompleksnost algoritma jednaka $O(n \cdot C)$.

Navedimo sada primjer konkretne instance problema i način popunjavanja DP tabele.

Neka je broj proizvoda $n = 4$, kapacitet ruksaka $C = 5$, a težine i vrijednosti proizvoda date nizovima $W = (2, 3, 4, 5)$, $P = (3, 4, 5, 6)$.

Formiramo matricu DP dimenzija $(n + 1) \times (C + 1)$, pri čemu vrijednost $DP[i][j]$ predstavlja maksimalnu vrijednost koju je moguće ostvariti korištenjem prvih i proizvoda i ruksaka kapaciteta j .

Prvi korak sastoji se od inicijalizacije baznih slučajeva. Kako ruksak kapaciteta 0 ne može sadržavati nijedan proizvod, vrijedi $DP[i][0] = 0$ za sve i . Takođe, ako ne razmatramo nijedan proizvod, ostvarena vrijednost je jednaka nuli, pa vrijedi $DP[0][j] = 0$ za sve j . Dobijamo početnu tabelu:

$j \rightarrow$	Proizvod	0	1	2	3	4	5
$i = 0$		0	0	0	0	0	0
$i = 1$	$w_1 = 2$ $p_1 = 3$	0					
$i = 2$	$w_2 = 3$ $p_2 = 4$	0					
$i = 3$	$w_3 = 4$ $p_3 = 5$	0					
$i = 4$	$w_4 = 5$ $p_4 = 6$	0					

Za $i = 1$ razmatramo samo prvi proizvod. Kako njegova težina iznosi 2, nije ga moguće smjestiti u ruksak kapaciteta 0 ili 1. Za sve kapacitete $j \geq 2$ optimalno je uzeti taj proizvod, pa druga vrsta tabele postaje:

$j \rightarrow$	Proizvod	0	1	2	3	4	5
$i = 0$		0	0	0	0	0	0
$i = 1$	$w_1 = 2$ $p_1 = 3$	0	0	3	3	3	3
$i = 2$	$w_2 = 3$ $p_2 = 4$	0					
$i = 3$	$w_3 = 4$ $p_3 = 5$	0					
$i = 4$	$w_4 = 5$ $p_4 = 6$	0					

Dalje popunjavamo treću vrstu ($i = 2$), pri čemu su na raspolaganju prva dva proizvoda.

$j \rightarrow$	Proizvod	0	1	2	3	4	5
$i = 0$		0	0	0	0	0	0
$i = 1$	$w_1 = 2$ $p_1 = 3$	0	0	3	3	3	3
$i = 2$	$w_2 = 3$ $p_2 = 4$	0	0	3	4	4	7
$i = 3$	$w_3 = 4$ $p_3 = 5$	0					
$i = 4$	$w_4 = 5$ $p_4 = 6$	0					

Razmotrimo ćeliju $DP[2][4]$. Postoje dvije mogućnosti:

- ne uzimamo drugi proizvod, pa je vrijednost jednaka $DP[1][4] = 3$;
- uzimamo drugi proizvod, pa preostaje kapacitet $4 - 3 = 1$, odnosno dobijamo vrijednost $DP[1][1] + 4 = 0 + 4 = 4$.

Prema tome, $DP[2][4] = \max\{3, 4\} = 4$. Vrijednost $DP[2][5] = 7$ dobija se uzimanjem oba proizvoda, jer vrijedi $DP[1][2] + 4 = 3 + 4 = 7$.

Popunjavanjem četvrte vrste ($i = 3$) dobijamo:

$j \rightarrow$	Proizvod	0	1	2	3	4	5
$i = 0$		0	0	0	0	0	0
$i = 1$	$w_1 = 2$ $p_1 = 3$	0	0	3	3	3	3
$i = 2$	$w_2 = 3$ $p_2 = 4$	0	0	3	4	4	7
$i = 3$	$w_3 = 4$ $p_3 = 5$	0	0	3	4	5	7
$i = 4$	$w_4 = 5$ $p_4 = 6$	0					

Na primjer, za ćeliju $DP[3][4]$ imamo $DP[2][4] = 4$ ako ne uzmemo treći proizvod, odnosno $DP[2][0] + 5 = 5$ ako ga uzmemo. Zato je $DP[3][4] = \max 4, 5 = 5$.

Konačno, nakon obrade četvrtog proizvoda dobijamo završnu tabelu:

$j \rightarrow$	Proizvod	0	1	2	3	4	5
$i = 0$		0	0	0	0	0	0
$i = 1$	$w_1 = 2$ $p_1 = 3$	0	0	3	3	3	3
$i = 2$	$w_2 = 3$ $p_2 = 4$	0	0	3	4	4	7
$i = 3$	$w_3 = 4$ $p_3 = 5$	0	0	3	4	5	7
$i = 4$	$w_4 = 5$ $p_4 = 6$	0	0	3	4	5	7

Posmatrajmo ćeliju $DP[4][5]$. Ako ne uzmemo četvrti proizvod, vrijednost je $DP[3][5] = 7$. Ako ga uzmemo, dobijamo $DP[3][0] + 6 = 6$. Stoga je $DP[4][5] = \max\{7, 6\} = 7$.

Kako se optimalna vrijednost problema nalazi u ćeliji $DP[n][C]$, zaključujemo da je optimalna vrijednost za posmatranu instancu jednaka $DP[4][5] = 7$. Ona se ostvaruje odabirom prvog i drugog proizvoda, čija je ukupna težina $2 + 3 = 5$, a ukupna vrijednost $3 + 4 = 7$.

Definicija 3.1 *String s je podniz stringa s_1 akko se s može dobiti brisanjem (nula ili više) karaktera iz stringa s_1 a čuvajući poredak ostalih karaktera.*

Npr. acd je podniz stringa $abbbcddef$. Prefiks stringa s dužine k je podstring stringa s koji se sastoji od prvih k karaktera. Npr. prefiks dužine tri stringa $abbbcddef$ je string abb .

Primjer 3.15 *(Problem najdužeg zajedničkog podniza (eng. Longest Common Subsequence Problem – LCS)). Neka su data dva stringa s_1 i s_2 . Potrebno je odrediti najduži string s koji je podniz oba ulazna stringa.*

Na primjer, za stringove $s_1 = aatdcccddc$ i $s_2 = agccddta$ jedno optimalno rješenje je $s = accdd$, čija je dužina jednaka 5.

Rješenje. Definišimo podproblem parom indeksa (i, j) , gdje je $0 \leq i \leq |s_1|, 0 \leq j \leq |s_2|$. Podproblem označen parom (i, j) odgovara određivanju dužine najdužeg zajedničkog podniza između prefiksa od s_1 dužine i i prefiksa od s_2 dužine j . Rješenja podproblema memorišemo u matrici DP, pri čemu vrijednost $DP[i][j]$ predstavlja dužinu najdužeg zajedničkog podniza pomenutih prefiksa.

Rekurzivna relacija. Posmatrajmo posljednje karaktere razmatranih prefiksa, odnosno karaktere $s_1[i-1]$ i $s_2[j-1]$. Moguća su dva slučaja.

- Ako je $s_1[i-1] = s_2[j-1]$, tada taj karakter može biti dio optimalnog rješenja. Optimalni podniz podproblema (i, j) dobija se proširivanjem optimalnog rješenja podproblema $(i-1, j-1)$ za jedan karakter. Stoga vrijedi $DP[i][j] = DP[i-1][j-1] + 1$.
- Ako je $s_1[i-1] \neq s_2[j-1]$, tada barem jedan od ta dva karaktera neće generisati optimalno rješenje. Zbog toga razmatramo dva podproblema $(i-1, j)$ i $(i, j-1)$, odnosno izostavljamo posljednji karakter prvog ili drugog prefiksa. Optimalna vrijednost dobija se izborom boljeg od ta dva podproblema:

$$DP[i][j] = \max\{DP[i-1][j], DP[i][j-1]\}.$$

Prema tome, rekurzija ima oblik

$$DP[i][j] = \begin{cases} DP[i-1][j-1] + 1, & \text{ako je } s_1[i-1] = s_2[j-1], \\ \max\{DP[i-1][j], DP[i][j-1]\}, & \text{inače.} \end{cases}$$

Bazni slučajevi. Ako je jedan od prefiksa prazan, tada zajednički podniz ne postoji, pa je njegova dužina jednaka nuli. Zbog toga vrijedi $DP[i][0] = 0, 0 \leq i \leq |s_1|$, i $DP[0][j] = 0, 0 \leq j \leq |s_2|$.

Nakon popunjavanja matrice DP, dužina najdužeg zajedničkog podniza nalazi se u ćeliji $DP[|s_1|][|s_2|]$. Sam podniz može se rekonstruisati prolaskom unazad kroz matricu, počevši od pozicije $(|s_1|, |s_2|)$ i prateći odluke koje su dovele do optimalne vrijednosti.

Implementacija paradigme dinamičkog programiranja za rješavanje LCSP je data narednim kôdom.

```
def LCS(s1, s2): #tabuliranje

    if len(s1)==0 or len(s2)==0:
        return 0

    DP = [ [0]*(len(s2)+1) for _ in range(len(s1)+1)]
    for i in range(1, len(s1)+1):
        for j in range(1, len(s2)+1):
            if s1[i-1] == s2[j-1]:
                DP[i][j] = DP[i-1][j-1] + 1
            else:
                DP[i][j] = max(DP[i-1][j], DP[i][j-1])
    return DP[len(s1)][len(s2)]

#instanca:
s1 = "abcdcdcdaa"
s2 = "abbaabccddd"
#poziv DP metoda
lcs = LCS(s1, s2)
print("Duzina LCS-a je: ", lcs)
```

Primijetimo da implementacija koristi princip odozdo-nagore.

Kompleksnost algoritma. Dvije for-petlje izvršavaju ukupno $O(|s_1| \cdot |s_2|)$ iteracija. Svaka od iteracija se izvršava u konstantnom vremenu, pa je shodno tome ukupno vrijeme izvršavanja algoritma jednako $O(|s_1| \cdot |s_2|)$. Ako je $n = \max\{|s_1|, |s_2|\}$, dobija se kvadratna $O(n^2)$ kompleksnost.

Primjer 3.16 Problem sume podskupa (eng. Subset Sum Problem). Neka je dat niz od n nenegativnih cijelih brojeva i ciljna vrijednost sum . Potrebno je odrediti da li postoji podskup elemenata datog niza čiji je zbir jednak vrijednosti sum .

Rješenje. Posmatrajmo primjer u kojem je $niz = [1, 7, 8, 2, 6]$ i $sum = 9$.

Odgovor je pozitivan, jer podskup 1, 2, 6 ima zbir jednak 9.

		0	1	2	3	4	5	6	7
		$\emptyset$	M	Z	J	A	W	X	U
0	$\emptyset$	0	0	0	0	0	0	0	0
1	X	0	0	0	0	0	0	1	1
2	M	0	1	1	1	1	1	1	1
3	J	0	1	1	2	2	2	2	2
4	Y	0	1	1	2	2	2	2	2
5	A	0	1	1	2	3	3	3	3
6	U	0	1	1	2	3	3	3	4
7	Z	0	1	2	2	3	3	3	4

Tabela 3.1: DP matrica za LCSP dva stringa $s_1 = \text{MZJAWXU}$ i $s_2 = \text{XMJYAUZ}$. Rješenje je dužine 4 (pogledati donji desni ugao matrice).

Da bismo riješili problem dinamičkim programiranjem, definišimo podproblem određen parom indeksa

$$(i, j), 0 \leq i \leq n, 0 \leq j \leq \text{sum}.$$

Podproblem (i, j) odgovara pitanju:

Da li među prvih i elemenata niza postoji podskup čiji je zbir jednak j ?

Rješenja podproblema memorišemo u matrici DP, gdje je

$$\text{DP}[i][j] = \begin{cases} \text{True}, & \text{ako takav podskup postoji,} \\ \text{False}, & \text{inače.} \end{cases}$$

Rekurzivna relacija. Posmatrajmo podproblem označen parom (i, j) i element $\text{niz}[i - 1]$. Postoje dva moguća slučaja.

- Element $\text{niz}[i - 1]$ pripada rješenju.

U tom slučaju preostali elementi moraju formirati zbir $j - niz[i - 1]$. Stoga vrijedi $DP[i][j] = DP[i - 1][j - niz[i - 1]]$, pod uslovom da je $j \geq niz[i - 1]$.

- Element $niz[i - 1]$ ne uzimamo u rješenje. Tada se problem svodi na prvih $i - 1$ elemenata: $DP[i][j] = DP[i - 1][j]$.

Pošto je dovoljno da bude zadovoljen bilo koji od ova dva slučaja, dobijamo rekurziju

$$DP[i][j] = DP[i - 1][j] \vee DP[i - 1][j - niz[i - 1]],$$

ako je $j \geq niz[i - 1]$. U suprotnom, element ne može biti dio rješenja, pa vrijedi $DP[i][j] = DP[i - 1][j]$.

Bazni slučajevi. Zbir 0 uvijek možemo ostvariti izborom praznog podskupa, pa vrijedi $DP[i][0] = True, 0 \leq i \leq n$. S druge strane, ako ne raspoložemo niti jednim elementom, nijedan pozitivan zbir ne može biti formiran $DP[0][j] = False, 1 \leq j \leq sum$.

Nakon popunjavanja matrice DP, odgovor na početni problem nalazi se u ćeliji $DP[n][sum]$.

Ako je vrijednost ove ćelije *True*, postoji podskup čiji je zbir jednak zadanoj vrijednosti *sum*; u suprotnom takav podskup ne postoji.

Implementacija paradigme dinamičkog programiranja za rješavanje Problema *sume podskupa* je data narednim kôdom.

```
def sum_array(niz, suma):
    #tabulation:
    DP = [[False] * (suma + 1) for _ in range(len(niz) + 1)]
    for i in range(len(niz)+1):
        DP[i][0] = True

    for i in range(1, len(niz)+1):
        for j in range(1, suma+1):
            if j >= niz[i-1]:
                DP[i][j] = DP[i-1][j-niz[i-1]] or DP[i-1][j]
```

```

        else:
            DP[i][j] = DP[i-1][j]

    return DP[len(niz)][suma]

#instanca:
niz = [3, 34, 4, 12, 5, 2]
suma = 9
postoji = sum_array(niz, suma)
print("Postoji podskup sa sumom" if postoji else
      "Ne postoji podskup sa sumom")

```

Primijetimo da smo dinamičko programiranje implementirali pomoću pristupa *odozdo prema gore* (tabuliranje).

Implementirajmo sada dinamičko programiranje za ovaj problem pristupom *odozgo prema dolje*.

```

def init(niz, suma):
    global DP
    DP = [[-1] * (suma + 1) for _ in range(len(niz) + 1) ]

def sum_array_top_down(niz, suma, i, j, DP = [ ]):

    if DP[i][j] != -1:
        return DP[i][j]

    if j == 0:
        DP[i][0] = True
        return True
    if i == 0:
        DP[0][j] = False
        return False

    #recursion:
    if j >= niz[i-1]:
        DP[i][j] = sum_array_top_down(niz, suma, i-1, j, DP) or
        sum_array_top_down(niz, suma, i-1, j-niz[i-1], DP)
        return DP[i][j]
    else:
        DP[i][j] = sum_array_top_down(niz, suma, i-1, j, DP)
        return DP[i][j]

```

Kompleksnost algoritma. Broj iteracija koje se izvršavaju je jednak $O(n \cdot \text{sum})$. Svaka iteracija se izvršava u konstantnom vremenu, pa je i ukupna kompleksnost algoritma jednaka $O(n \cdot \text{sum})$. Dakle, ako je vrijednost sum ogromna, ekponencijalno velika u odnosu na veličinu niza, ovakav algoritam će biti neefikasan.

Primjer 3.17 (*Problem rezanja štapa*). *Dat je štap dužine n i niz cijena price , pri čemu $\text{price}[i - 1]$ predstavlja cijenu štapa dužine i , za svako $1 \leq i \leq n$.*

Potrebno je odrediti način rezanja štapa na manje dijelove tako da ukupan ostvareni profit bude maksimalan.

Rješenje. Posmatrajmo primjer instance problema date sa

$$\text{price} = [1, 5, 8, 9, 10, 17, 17, 20].$$

Pretpostavimo da je dužina štapa $n = 4$. Ako se štap ne reže, ostvaruje se profit jednak 9. Međutim, ako ga podijelimo na dva dijela dužine 2, dobijamo profit $5 + 5 = 10$, što predstavlja bolje (u ovom slučaju optimalno) rješenje za tu instancu.

Da bismo problem riješili dinamičkim programiranjem, definišimo podproblem dužinom štapa i , gdje je $0 \leq i \leq n$.

Neka $\text{DP}[i]$ označava maksimalni profit koji se može ostvariti rezanjem štapa dužine i . Vrijednost $\text{DP}[i]$ memorišemo u nizu DP na poziciji i .

Rekurzivna relacija. Posmatrajmo štap dužine i . Podijelimo ovaj štap na dio dužine k , gdje je $1 \leq k \leq i$. Na taj način štap dijelimo na:

- dio dužine k , čija je vrijednost $\text{price}[k - 1]$;
- preostali dio dužine $i - k$, čiji je optimalni profit jednak $\text{DP}[i - k]$.

Za svaku moguću vrijednost k dobijamo kandidatsko rješenje $\text{price}[k - 1] + \text{DP}[i - k]$. Optimalna vrijednost za štap dužine i dobija se izborom najboljeg među svim mogućim prvim rezovima:

$$DP[i] = \max_{1 \leq k \leq i} \{price[k-1] + DP[i-k]\}.$$

Bazni slučaj. Štap dužine 0 nema vrijednost, pa vrijedi $DP[0] = 0$.

Nakon popunjavanja niza DP, optimalna vrijednost početnog problema nalazi se u ćeliji $DP[n]$. Vrijednost $DP[n]$ predstavlja maksimalan profit koji se može ostvariti rezanjem štapa dužine n .

Vremenska složenost. Za svaku dužinu i razmatra se najviše i mogućih pozicija prvog reza. Ukupan broj operacija jednak je $\sum_{i=1}^n i = \frac{n(n+1)}{2}$, pa je vremenska složenost algoritma $O(n^2)$. Prostorna složenost iznosi $O(n)$, jer se koristi jednodimenzionalni niz DP veličine $n + 1$.

Implementacija paradigme dinamičkog programiranja za rješavanje problema rezanja štapa je data narednim kôdom.

```
def stick_cut(d, price):

    DP = [-1] * (d+1) #init
    DP[0], DP[1] = 0, price[0]

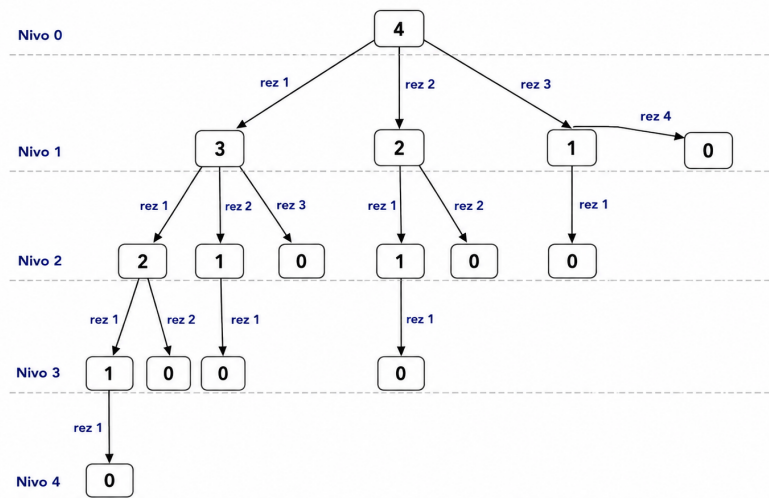
    for i in range(2, d+1):
        for k in range(1, i+1):
            if DP[i] < DP[i-k] + price[k-1]:
                DP[i] = DP[i-k] + price[k-1]

    return DP[d]

#instanca
d = 4
price = [1, 5, 8, 9, 10, 17, 17, 20]
max_profit = stick_cut(d, price)
#poziv
print(max_profit)
```

Napomena. Pogledajmo drvo rekurzije koje se kreira algoritmom grube sile na Slici 3.9 pri rješavanju problema rezanja štapa za $d = 4$. Ovaj pristup enumeriše sva moguća rješenja za rezanje štapa dužine $d = 4$. Prvo uzima

cio štap u razmatranje (i njegovu cijenu), zatim otkida dio štapa dužine 1, dok se ostatak štapa (dužine 3) rješava (dijeli) rekursivno itd. Kompletно rješenje dobijamo u listovima stabla (označeni sa 0).



Slika 3.9: Drvo grananja paradigme grube sile za rješavanje problema reza-
nja štapa.

Npr. put koji posjećuje čvorove sa oznakama $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$, odgovara rješenju u kojem se štap siječe na 4 manja štapa dužine 1 (cijene $4 \times price[0]$). Slično, za put koji posjećuje čvorove sa oznakama $4 \rightarrow 3 \rightarrow 1 \rightarrow 0$, odgovara rješenju u kojem se štap siječe na dva manja štapa dužine 1, te jedan dužine 2 (cijene $2 \times price[0] + price[1]$).

Jasno se vidi da se podproblemi isprepliću, tj. da se, npr. rješavanje problema štapa dužine dva računa više (dva) puta. U slučaju dinamičkog

programiranja implementiran pristupom odozgo prema dolje, stablo će biti značajno redukovano, jer neće postojati rekalkulacije istih podproblema (dakle, nema identičnih kopija podstabala u DP stablu).

Zadaci

1. Riješiti problem trgovačkog putnika pomoću algoritamske paradigme grube sile.
2. Riješiti problem ruksaka pomoću algoritamske paradigme grube sile.
3. Dati su brojevi $0 \leq M < N \leq 1,000,000$. Odrediti najmanji broj sastavljen samo od neparnih cifara, koji pri dijeljenju sa N daje ostatak M . Ukoliko traženi broj ne postoji, vratiti 1.
4. Neka su dati prirodni brojevi n , koji označava broj promjenljivih, i s , koji označava sumu promjenljivih. Uz pomoć rekurzivnih algoritama napisati program koji pronalazi sva moguća rješenja za promjenljive kojih ima n i čija suma treba biti jednaka s .

Instanca: $n = 5, s = 4$; *Rješenje:* 70.

Napomena. Instanca predstavlja jednačinu $x_1 + x_2 + x_3 + x_4 + x_5 = 4, x_i \in \mathbb{N}$. Sva moguća rješenja su: $(1\ 1\ 1\ 1\ 0), (1\ 0\ 1\ 1\ 1), (0\ 1\ 1\ 1\ 1), \dots, (2\ 1\ 0\ 0\ 1), \dots, (2\ 2\ 0\ 0\ 0)$ itd.

5. Pronaći podskup elemenata niza tako da proizvod elemenata u podskupu bude minimalan. Koristiti paradigmu pohlepni algoritama.
Instanca: $niz = [-1, -1, -2, 4, 3]$; *Rješenje:* $-24 (= (-2) \cdot (-1) \cdot (-1) \cdot 4 \cdot 3)$.
6. Svaki pozitivan razlomak možemo predstaviti kao zbir jedinstvenih jediničnih razlomaka. Razlomak je jedinični ako je brojnik 1, a nazivnik pozitivan cijeli broj; npr. $1/3$ je jedinični razlomak. Npr. $2/3 = 1/2 + 1/6, 3/7 = 1/3 + 1/11 + 1/231$. Uz pomoć paradigme pohlepni algoritama, za proizvoljan razlomak, naći jedinične razlomke koje u zbiru daju taj razlomak.

7. Riješiti problem nalaska najdužeg zajedničkog podniza dva stringa pomoću dinamičkog programiranja pristupom odozgo prema dolje.
8. U ulazu je dat niz brojeva. Konstruisati algoritam dinamičkog programiranja koji daje odgovor na pitanje li postoji particionisanje niza na dva (disjunktna) skupa tako da je zbir elemenata u obje particije jednak.
9. Implementirati metod dinamičkog programiranja za problem rezanja štapa koristeći pristup odozgo prema dolje.
10. Neka su u ulazu data dva stringa s_1 i s_2 . Uz pomoć dinamičkog programiranja, pronaći najkraći string s tako da su stringovi s_1 i s_2 podnizovi stringa s .
11. Data je riječ i riječnik (skup riječi). Ispitati da li se riječ može podijeliti na segmente (podstringove) tako da oni pripadaju riječniku.

Instanca:

`dict = {this, th, is, famous, Word, break, b, r, e, a, k, br, bre, brea, ak, problem}`

te

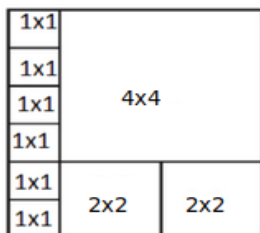
`word = Wordbreakproblem.`

Rješenje: Word break problem, kao jedno od rješenja.

12. Neka je dat pravougaonik dimenzija $N \times M$. Pretpostavimo da je na raspolaganju beskonačan broj pločica formata $2^i \times 2^i$, $i = 0, 1, 2, 3, \dots$

Napisati program koji rekurzivno pronalazi minimalan broj pločica koji je potreban da bi se popločao pravougaonik datih dimenzija.

Instanca: $N = 6, M = 5$; *Rješenje:* 9 (pločica) – vidjeti narednu sliku.

Slika 3.10: Optimalno pokrivanje ploče 6×5

13. Napisati program koji generiše sve mogućnosti da se stavi $+$ ili $-$ ili niti jedna od ove dvije opcije između brojeva $1, 2, \dots, 9$ (ovim redom) tako da se kao rezultat izvršenja operacija dobije vrijednost 100. Npr. jedno takvo rješenje je $1 + 2 + 3 - 4 + 5 + 6 + 78 + 9 = 100$.
14. Za dva stringa, napisati program koji ispisuje najkraći niz umetanja i brisanja karaktera u prvi string čijom se primjenom dobija drugi string.

Glava 4

Primjena algoritamskih paradigmi na probleme iz aritmetike

U ovoj glavi razmatramo primjenu algoritamskih paradigmi predstavljених u prethodnom poglavlju na odabrane probleme iz aritmetike i teorije brojeva. Cilj je pokazati kako se različiti pristupi dizajnu algoritama, poput grube sile, podijeli-pa-zavladaj, dinamičkog programiranja i drugih paradigmi, mogu koristiti za efikasno rješavanje matematičkih problema.

Nakon uspješnog savladavanja sadržaja ove glave student će biti u stanju da:

- primijeni prethodno obrađene algoritamske paradigme na rješavanje problema iz elementarne aritmetike i teorije brojeva;
- objasni i implementira Eratostenovo sito za efikasno određivanje prostih brojeva u zadanom intervalu;
- implementira postupke faktORIZACIJE cijelih brojeva i analizira njihovu vremensku složenost;

- izračunava binomne koeficijente primjenom rekurzivnog pristupa i dinamičkog programiranja te uporedi njihove performanse;
- objasni i implementira Hornerov metod za efikasno izračunavanje vrijednosti polinoma;
- analizira vremensku i prostornu složenost algoritama za razmatrane aritmetičke probleme i uporedi različite pristupe njihovom rješavanju.

4.1 Eratostenovo sito

Definicija 4.1 Broj $n \in \mathbb{N}$ naziva se *prostim brojem* ako je djeljiv isključivo sa 1 i samim sobom.

Na primjer, prvih pet prostih brojeva su: 2, 3, 5, 7, 11. Broj 6 nije prost jer je, pored brojeva 1 i 6, djeljiv i sa 2 i sa 3.

Posmatrajmo sljedeći problem:

Odrediti sve proste brojeve manje ili jednake vrijednosti N .

Jedan jednostavan pristup zasniva se na paradigmi grube sile. Za svaki broj iz intervala $[2, N]$ provjeravamo da li je prost koristeći sljedeću proceduru:

```
def prost(n):
    for i in range(2, n // 2 + 1):
        if n % i == 0:
            return False
    return True
```

Ova procedura u najgorem slučaju izvršava $O(n)$ operacija za broj n . Kako se poziva za približno N različitih brojeva, ukupna vremenska složenost ovakvog pristupa iznosi $O(N^2)$.

Za veće vrijednosti parametra N ovakav pristup postaje neefikasan.

Efikasnije rješenje daje algoritam poznat kao *Eratostenovo sito*. Njegova osnovna ideja zasniva se na sljedećem zapažanju:

Ako je broj p prost, tada nijedan njegov višekratnik $2p, 3p, 4p, \dots$ ne može biti prost.

Zbog toga, nakon što identifikujemo prost broj p , svi njegovi višekratnici mogu se odmah eliminisati iz daljeg razmatranja.

Algoritam počinje skupom svih brojeva od 2 do N . Najmanji broj koji još nije eliminisan proglašava se prostim, nakon čega se svi njegovi višekratnici označavaju kao složeni brojevi. Postupak se zatim ponavlja za naredni neoznačeni broj. Na taj način se progresivno filtriraju svi složeni brojevi, dok u “situ” ostaju upravo prosti brojevi.

Proces se završava kada više nije potrebno eliminisati nove višekratnike. Preostali neoznačeni brojevi predstavljaju sve proste brojeve manje ili jednake vrijednosti N .

Implementacija Eratostenovog sita u programskom jeziku Pajton data je u nastavku.

```
def eratostenovo_sito(N):  
  
    prost = [True] * (N + 1)  
    prost[0] = prost[1] = False  
    p = 2  
  
    while p * p <= N:  
        if prost[p]:  
            for k in range(p * p, N + 1, p):  
                prost[k] = False  
            p += 1  
  
    return [i for i in range(2, N + 1) if prost[i]]
```

Kompleksnost algoritma. Eratostenovo sito ima vremensku složenost $O(N \log \log N)$, dok je prostorna složenost jednaka $O(N)$, zbog pomoćnog niza kojim se označava da li je broj prost ili složen.

Zahvaljujući svojoj efikasnosti, Eratostenovo sito predstavlja jedan od najpoznatijih algoritama za generisanje prostih brojeva i često se koristi kao osnovna tehnika u problemima iz teorije brojeva i algoritamske matematike. Implementacija algoritma je data narednim kôdom.

```
def sieve(N):

    sieve = [True] * (N+1)
    sieve[0] = sieve[1] = False
    p = 2
    while p <= N:

        if sieve[p]:
            index = p * p
            while index <= N:
                sieve[index] = False
                index += p
            p = p + 1
    for i in range(N+1):
        if sieve[i]: #broj je prost
            print(i)

#main:
N = 1000
sieve(N)
```

Kompleksnost algoritma. Može se pokazati da je kompleksnost prethodne implementacije jednaka $O(N \log \log N)$.

Demonstrirajmo prvih nekoliko iteracija ovog algoritma u tabelarnom zapisu, za $N = 50$.

U početnoj iteraciji, inicira se niz (tabela) brojeva gdje su svi brojevi označeni kao prosti (na poziciji koja odgovara broju se čuva vrijednost *True*).

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Tabela 4.1: Inicijalizacija.

Dalje, u prvoj iteraciji, nalazimo se na poziciji 2, gdje elemente na pozicijama $2 \cdot k$, $k = 2, \dots, 50$ eliminišemo kroz sito (kao oni koji nisu prosti, dodijeljujući im vrijednost *False*). Vizuelno, preostali brojevi za ispitivanje su oni neoznačeni:

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Tabela 4.2: Eratostenovo sito: iteracija 1.

Dalje, u drugoj iteraciji, prelazimo na poziciju 3. Kako je ona označena sa *True* (prost broj), elemente na pozicijama $3 \cdot k$, $k = 3, \dots, 16$ eliminišemo kroz sito (jer nisu prosti). Preostali brojevi za ispitivanje su oni neoznačeni u narednoj tabeli.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Tabela 4.3: Eratostenovo sito: iteracija 2.

U narednoj iteraciji prelazimo na poziciji 4. Kako to nije prost broj

(dakle filtriran prethodno), prelazimo na naredni broj (5), koji je prost, nakon čega ponavljamo operacije opisane prethodnim iteracijama.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Tabela 4.4: Eratostenovo sito: iteracija 3.

4.2 Faktorizacija broja

Fundamentalna teorema aritmetike kaže da se svaki prirodan broj $n > 1$ može zapisati kao proizvod prostih brojeva na jedinstven način, do na redoslijed faktora. Drugim riječima, za svaki broj n postoje prosti brojevi $p_1, p_2, \dots, p_k$ i prirodni eksponenti $a_1, a_2, \dots, a_k$ takvi da vrijedi $n = \prod_{i=1}^k p_i^{a_i}$. Ovaj zapis naziva se *prosta faktorizacija* broja n . Na primjer, $120 = 2^3 \cdot 3 \cdot 5$.

Sa algoritamskog stanovišta postavlja se pitanje kako efikasno odrediti prostu faktorizaciju datog broja. Naivan pristup podrazumijeva isprobavanje svih mogućih djelilaca, što za velike brojeve može biti veoma neefikasno.

Efikasniji pristup može se zasnovati na idejama korištenim u Eratostenovom situ. Osnovna ideja je sljedeća: za dati broj n pronađemo njegov najmanji prosti djelilac p . Tada vrijedi $n = p \cdot \frac{n}{p}$. Nakon toga isti postupak rekursivno primjenjujemo na broj n/p , sve dok ne dođemo do vrijednosti 1. Na taj način broj postepeno razlažemo na proste faktore.

Ovakav pristup prirodno slijedi paradigmu *podijeli pa zavlada*. Problem faktorizacije broja n svodi se na faktorizaciju manjeg broja n/p , pri čemu se rezultat dobija dodavanjem prostog faktora p u konačno rješenje.

Da bi se postupak dodatno ubrzao, moguće je unaprijed konstruisati pomoćnu strukturu podataka koja za svaki broj i čuva njegov najmanji

prosti faktor. Ova struktura se može izgraditi modifikacijom Eratostenovog sita. Neka je niz $\text{SPF}[0], \text{SPF}[1], \dots, \text{SPF}[N]$ (eng. *Smallest Prime Factor*) definisan tako da vrijednost $\text{SPF}[i]$ predstavlja najmanji prosti faktor broja i . Na primjer, $\text{SPF}[12] = 2, \text{SPF}[15] = 3, \text{SPF}[49] = 7$. Za proste brojeve vrijedi $\text{SPF}[p] = p$.

Nakon izgradnje niza SPF , faktORIZACIJA broja n postaje veoma jednostavna: u svakom koraku uzimamo vrijednost $\text{SPF}[n]$, dodajemo je u listu faktora i zamjenjujemo n vrijednošću $n \leftarrow \frac{n}{\text{SPF}[n]}$.

Postupak se ponavlja sve dok ne dobijemo $n = 1$.

Na primjer, za broj 120 dobijamo: $120 \rightarrow 60 \rightarrow 30 \rightarrow 15 \rightarrow 5 \rightarrow 1$, odnosno proste faktore 2, 2, 2, 3, 5, što odgovara faktORIZACIJI $120 = 2^3 \cdot 3 \cdot 5$.

Kompleksnost algoritma. Izgradnja niza najmanjih prostih faktora do granice N može se izvršiti u vremenu reda $O(N \log \log N)$, dok se faktORIZACIJA pojedinačnog broja n izvršava u vremenu proporcionalnom broju njegovih prostih faktora, odnosno $O(\log n)$.

Zbog toga je ovaj pristup naročito pogodan kada je potrebno faktorisati veliki broj različitih brojeva iz istog intervala. Implementacija ovakve (nizovne) strukture je data narednim kôdom.

```
def sieve_adaptation(n):

    sieve_fact = [0] * (n+1)
    p = 2
    while p <= n:
        if sieve_fact[p] == 0:
            index = p * p
            while index <= n:
                sieve_fact[index] = p
                index += p
        p = p + 1

    return sieve_fact
```

```

def factorization(n):

    factorize_min_p = sieve_fact(n)
    F = []
    while True:
        i = factorize_min_p[n]
        if i == 0:
            F.append(n)
            return F
        else:
            n = n // i
            F.append(i)
    return F

#ulaz:
n = 120
F = factorization(n)
print("Lista faktora je: ", F)

```

Kompleksnost algoritma. Najskuplja operacija u algoritmu je adaptacija algoritma Eratostenovog sita (funkcija `sieve_adaptation`) i ona se izvršava u $O(n \log \log n)$ vremenu. Glavna `while`-petlja se izvršava u linearnom $O(n)$ vremenu. Dakle, čitav algoritma se izvršava u $O(n \log \log n)$ vremenu.

4.3 Izračunavanje binomnih koeficijenata

Binomni koeficijenti igraju bitnu ulogu u kombinatorici. Npr. broj različitih grupa od k ljudi može izabrati iz skupa od n ljudi je predstavljen binomnim koeficijentom $\binom{n}{k}$.

Definicija 4.2 Binomni koeficijent $\binom{n}{k}$ gdje je $n, k \in \mathbb{N}, k > 0$ je dat sa

$$\binom{n}{k} = \frac{n(n-1) \cdots (n-k+1)}{k!}$$

pri čemu je $\binom{n}{0} := 1$. Binomni koeficijent $\binom{n}{1} = n$, dok je $\binom{n}{n} = 1$ za sve $n \in \mathbb{N}$. Takođe, ako je $n < k$, vrijedi $\binom{n}{k} = 0$. Lako se može pokazati da vrijedi Paskalova jednakost:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k},$$

što nam daje rekurziju za računanje binomnih koeficijenata. Dakle, ako označimo $\text{binom}(n, k) = \binom{n}{k}$, vrijedi:

$$\text{binom}(n, k) = \text{binom}(n-1, k-1) + \text{binom}(n-1, k).$$

Implementacija rekurzivnog pristupa je data sljedećim kôdom.

```
def binom(n, k):
    if k == 0:
        return 1
    if n < k:
        return 0
    return binom(n-1, k-1) + binom(n-1, k)
```

Kompleksnost algoritma. Faktor grananja rekurzije je 2. To znači da na svakoj narednoj dubini rekurzije imamo (najviše) duplo više rekurzivnih poziva nego je to slučaj na dubini prije. Kako je dubina rekurzije maksimalno n (vrijednost prvog ili drugog atributa se smanjuje za 1 u narednom pozivu rekurzije), zaključujemo da je maksimalni broj poziva rekurzije eksponencijalan i reda veličine $O(2^n)$.

Optimizujmo ovu naivnu implementaciju rekurzije za računanje binomnih koeficijenata. Jasno se vidi da se većina podproblema rješava više puta u rekurzivnim pozivima. Zbog toga ima smisla upotrebiti princip memoizacije. Označimo strukturu u kojoj čuvamo rješenja matricom $B[i][j] = \binom{i}{j}$. Bazi slučaj je dat sa $B(n, 0) = 1 = B(n, n)$. Ako je $i < j$, tada je $B[i][j] = 0$.

Implementacija principa memoizacije za izračunavanje binomnog koeficijenta $\binom{n}{k}$ je data narednim kôdom.

```

def initialization(n, k):
    Binom = [[-1] * (k+1) for _ in range(n+1)]
    return Binom

def binom_memoization(i, j, Binom):

    if Binom[i][j] != -1: # već računat
        return Binom[i][j]
    if j == 0:
        Binom[i][j] = 1
        return Binom[i][j]
    if i < j:
        Binom[i][j] = 0
        return Binom[i][j]

    Binom[i][j] = binom_memoization(i-1, j-1, Binom)
                  + binom_memoization(i-1, j, Binom)
    return Binom[i][j]

#poziv metode:
n = 10
k = 4
Binom = initialization(n, k)
n_over_k = binom_memoization(n, k, Binom)
print(n_over_k)

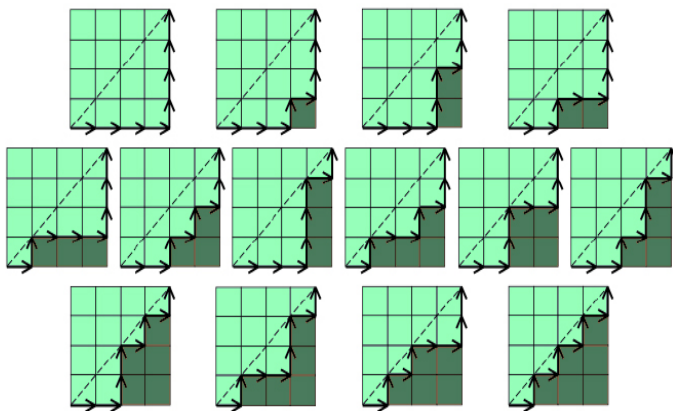
```

Kompleksnost algoritma. Kreiranje strukture se izvršava u $O(n \cdot k)$ vremenskoj kompleksnosti. Dalje, broj podproblema koji se rješavaju je jednak $(n+1) \cdot (k+1)$. U svakom rekurzivnom pozivu, izvršava se konstantan broj operacija. Sveukupno, kompleksnost algoritma je jednaka $O(n \cdot k)$.

Katalanovi brojevi pojavljuju se u velikom broju interesantnih kombi-

natornih problema. U nastavku navodimo nekoliko njihovih poznatih interpretacija.

- **Brojanje putanja na pravougaonoj mreži.** Broj različitih putanja od donjeg lijevog do gornjeg desnog ugla mreže dimenzija $n \times n$, pri čemu se u svakom koraku smije kretati samo udesno ili nagore, a putanja ne smije prelaziti iznad glavne dijagonale (može je samo dodirivati), jednak je n -tom Katalanovom broju.



Slika 4.1: Putanje na mreži dimenzija 4×4 .

- **Permutacije bez određenog paterna.** Broj permutacija skupa $\{1, 2, \dots, n\}$ koje ne sadrže patern 123 (ili, opštije, bilo koji fiksirani patern dužine 3) takođe je dat Katalanovim brojevima. Na primjer, za $n = 3$ dozvoljene su permutacije 132, 213, 231, 312, 321, dok permutacija 123 nije dozvoljena. Dakle, postoji ukupno 5 takvih permutacija, što odgovara vrijednosti $C_3 = 5$.
- **Pravilno ugniježdene zagrade.** Katalanov broj C_n predstavlja broj načina da se formira izraz sa n parova zagrada tako da je svaka otvorena zagrada ispravno zatvorena. Na primjer, za $n = 3$ postoji pet takvih izraza: $((()))$, $((()()))$, $((())())$, $(())(())$, $(())()()$.

Prvih nekoliko Katalanovih brojeva su dati sa nizom: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ... Katalanovi brojevi zadovoljavaju sljedeću rekursivnu relaciju:

$$C(0) = 1, \text{ i } C(n+1) = \sum_{i=0}^n C(i)C(n-i), \quad n \geq 0.$$

Ova rekurzija prirodno proizlazi iz činjenice da se mnogi objekti koji se broje Katalanovim brojevima mogu podijeliti na dva nezavisna dijela, čiji se doprinosi zatim množe i sabiraju preko svih mogućih podjela.

Primijenimo sada paradigmu grube sile, odnosno direktnu rekursivnu implementaciju, za računanje n -tog Katalanovog broja. Rekurzija slijedi neposredno iz prethodne formule:

```
def catalan(n):

    if n == 0:
        return 1

    result = 0
    for i in range(n):
        result += catalan(i) * catalan(n - 1 - i)
    return result
```

Kompleksnost algoritma. Ovakva implementacija generiše veliki broj ponovljenih izračunavanja istih podproblema. Zbog toga je njena vremenska složenost eksponencijalna. Upravo zbog izraženog preklapanja podproblema, računanje Katalanovih brojeva predstavlja prirodnu primjenu dinamičkog programiranja, koje značajno smanjuje vrijeme izvršavanja memoizacijom ili tabuliranjem prethodno izračunatih vrijednosti. Implementacija ovog pristupa je data sljedećim kôdom.

```
def catalan(n):
    if n == 0:
```

```

    return 1
catalan_n = 0
for i in range(n):
    catalan_n += catalan(i) * catalan(n-1-i)
return catalan_n

```

Kompleksnost algoritma. Faktor grananja rekurzije je (najviše) n . Dubina rekurzije je $n+1$. Dakle, kompleksnost ovog algoritma je eksponencijalna i iznosi $O(n^n)$.

Optimizujemo ovaj naivni rekurzivni pristup za računanje Katalanovih brojeva. Kako je primijetno svaki podproblem izražunava iznova više puta, ponovo iskoristimo princip memoizacije. Čuvajmo vrijednost i -tog Katalanovog broja u nizovnoj strukturi $\text{Cat}(i)$. Svojstvo optimalne podstrukture je dato rekurzijom $\text{Cat}(n+1) = \sum_{i=0}^n \text{Cat}(i) \cdot \text{Cat}(n-i)$. Bazni slučaj je dat sa $\text{Cat}(0) = 1$.

Implementacija memoizacije je data sljedećim kôdom.

```

def initialization(n):
    Cat = [ -1 for _ in range(n+1) ]
    Cat[0]=1
    return Cat

def catalan_i(i, Cat):

    if Cat[i] != -1:
        return Cat[i]

    catalan_i = 0
    for k in range(i):
        catalan_i += catalan_memoization(k, Cat) *
                     catalan_memoization(i-1-k, Cat)

```

```

        Cat[i] = catalan_i
    return Cat[i]

#poziv metode:
n = 10
Cat = initialization(n)

cat_n = catalan_n(n, Cat)
print(cat_n)

```

Kompleksnost algoritma. Dubina rekurzije je (najviše) $n + 1$. U svakoj rekurziji, broj iteracija koji se izvršava je linearne kompleksnosti $O(n)$. Prema tome, kompleksnost algoritma je $O(n) \cdot O(n) = O(n^2)$.

Posmatrajmo sljedeću proceduru konstruisanja niza brojeva.

- Počinjemo od niza prirodnih brojeva 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, ...
- U prvom koraku brišemo svaki drugi broj. Dobija se niz 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, ...
- Iz novodobijenog niza zatim brišemo svaki treći broj, pa dobijamo 1, 3, 7, 9, 13, 15, 19, ...
- Postupak se nastavlja tako što se iz trenutnog niza briše svaki četvrti broj, zatim svaki peti broj itd.

Brojevi koji nikada ne budu obrisani nazivaju se *srećnim brojevima* (eng. *lucky numbers*).

U ulazu je dat prirodan broj n . Potrebno je utvrditi da li je n srećan broj, odnosno da li će preživjeti prethodno opisani proces eliminacije.

Rješenje. Posmatrajmo položaj broja n tokom izvođenja postupka. U prvoj iteraciji briše se svaki drugi element niza. Prema tome, svi brojevi djeljivi sa 2 bivaju uklonjeni.

Ako je n paran broj, zaključujemo da će biti obrisani već u prvom koraku i vraćamo rezultat *False*. U suprotnom, broj n preživljava ovu iteraciju. Pošto je uklonjena približno polovina elemenata, njegova nova pozicija postaje

$$n \leftarrow n - \left\lfloor \frac{n}{2} \right\rfloor.$$

U drugoj iteraciji briše se svaki treći element trenutnog niza. Ako je nova pozicija broja n djeljiva sa 3, broj se uklanja i vraća se rezultat *False*. U suprotnom, broj ostaje u nizu, a njegova pozicija se ažurira na

$$n \leftarrow n - \left\lfloor \frac{n}{3} \right\rfloor.$$

Opštije, u i -toj iteraciji briše se svaki $(i + 1)$ -vi element trenutnog niza. Neka je trenutna pozicija posmatranog broja jednaka n .

- Ako je $n < i + 1$, broj više ne može biti obrisani ni u jednoj narednoj iteraciji, jer se uklanjaju samo elementi na pozicijama koje su višekratnici broja $i + 1$. U tom slučaju vraćamo rezultat *True*.
- Ako je n djeljiv sa $i + 1$, broj će biti obrisani u tekućoj iteraciji i vraćamo rezultat *False*.
- U suprotnom, broj preživljava iteraciju, a njegova nova pozicija postaje

$$n \leftarrow n - \left\lfloor \frac{n}{i + 1} \right\rfloor,$$

jer se prije njega uklanja upravo $\left\lfloor \frac{n}{i+1} \right\rfloor$ elemenata.

Postupak se ponavlja sve dok se ne utvrdi da je broj obrisani ili dok njegova trenutna pozicija ne postane manja od koraka eliminacije. U prvom slučaju broj nije srećan, dok u drugom zaključujemo da će preživjeti sve naredne iteracije i da jeste srećan broj.

Implementacija algoritma je data narednim kôdom.

```

from math import ceil
def lucky_number(n):

    for i in range(1, n): #iteracije
        if i+1 > ceil(n/i):
            return True
        if ceil(n/i) % (i+1) == 0:
            return False
    return True

#poziv funkcije
print(lucky_number(7)) #True

```

4.4 Hornerov metod

Definicija 4.3 *Hornerov metod za izračunavanje vrijednosti polinoma.* Neka je dat polinom $P(x) = c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0$, kao i tačka $x \in \mathbb{R}$. Potrebno je odrediti vrijednost polinoma P u tački x .

Pretpostavimo da je polinom predstavljen nizom *coef*, pri čemu element *coef*[*i*] predstavlja koeficijent uz monom x^{n-i} , za $i \in \{0, \dots, n\}$.

Rješenje. Posmatrajmo primjer instance: *coef* = [1, -3, 2, -1], $x = 3$. U ovom slučaju riječ je o polinomu $P(x) = x^3 - 3x^2 + 2x - 1$. Vrijednost polinoma u tački $x = 3$ iznosi $P(3) = 27 - 27 + 6 - 1 = 5$.

Naivan pristup podrazumijeva zasebno računanje svakog stepena x^i . Ako se vrijednost x^i računa iterativno, potrebno je $O(i)$ operacija, pa ukupna vremenska složenost evaluacije polinoma iznosi $O(1+2+\dots+n) = O(n^2)$. Hornerov metod omogućava izračunavanje vrijednosti polinoma u linearnom vremenu, odnosno u složenosti $O(n)$.

Ideju algoritma ilustrujmo na prethodnom primjeru. Polinom možemo zapisati u obliku $P(x) = ((x - 3)x + 2)x - 1$. Drugim riječima, $P(x) =$

$((c_3x + c_2)x + c_1)x + c_0$.

Računanje se odvija sukcesivno:

1. Počinje se od vodećeg koeficijenta $c_3 = 1$.
2. Trenutna vrijednost množi se sa $x = 3$ i dodaje se naredni koeficijent:
 $1 \cdot 3 + (-3) = 0$.
3. Dobijena vrijednost ponovo se množi sa x i dodaje se sljedeći koeficijent:
 $0 \cdot 3 + 2 = 2$.
4. Postupak se ponavlja: $2 \cdot 3 + (-1) = 5$.

Dobijamo konačnu vrijednost polinoma $P(3) = 5$.

U opštem slučaju vrijedi identitet

$$c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0 = (\dots ((c_n x + c_{n-1})x + c_{n-2})x + \dots + c_1)x + c_0.$$

Na ovaj način izbjegava se eksplicitno računanje stepena broja x , pa se polinom evaluira korištenjem samo n množenja i n sabiranja.

Bazni slučaj. Ako je $n = 0$, polinom se svodi na konstantu $P(x) = c_0$, pa algoritam neposredno vraća vrijednost c_0 .

Kompleksnost algoritma. Hornerov metod izvršava tačno n množenja i n sabiranja, pa je njegova vremenska složenost jednaka $O(n)$, dok je prostorna složenost $O(1)$.

Implementacija Hornerovog metoda je data narednim kôdom.

```
def horner_metod(coef, x):

    rezultat = 0
    for c in coef:
        rezultat = rezultat * x + c

    return rezultat
```

```

# poziv funkcije
# Evaluacija:  $x^3 - 3x^2 + 2x - 1$ ,  $x = 3$ 
coef = [1, -3, 2, -1]
x = 3
print(horner_metod(coef, x))

```

Zadaci

1. Broj je poluprost (eng. *semiprime*) ako se može predstaviti kao proizvod dva (ne obavezno različita) prosta broja. Npr. 4, 6, 9, 10, ... su neki od poluprostih brojeva. Konstruisati (efikasan) algoritam koji ispituje da li je broj poluprost.
2. Broj je savršen akko je jednak zbiru svojih djelilaca (ne računajući njega samog). Npr. 6 i 28 su savršeni jer je $6 = 1 + 2 + 3$; $28 = 1 + 2 + 4 + 7 + 14$. Napisati program koji ispituje da li je broj savršen korištenjem Eratostenovog sita. Kolika je vremenska složenost algoritma? Uporediti ovaj algoritam sa algoritamskom paradigmom brutalne sile.
3. Broj je ružan akko su mu jedini prosti faktori 2 ili 3 ili 5. Niz prvih ružnih brojeva je dat sa 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, ... (1 je po konvenciji uključen). U ulazu je dat broj n . Izračunati n -ti ružni broj. Koristiti paradigmu dinamičkog programiranja.
4. Sfenički brojevi su pozitivni cijeli brojevi koji se dobijaju kao proizvod tačno 3 različita prosta faktora. Npr. 30, 42, 66, 70, 78, 102, 105, 110, 114, ... su neki od takvih brojeva. U ulazu je dat broj n . Ispitati da li je n sfenički?

Primjer. $30 = 2 \cdot 3 \cdot 5$ jeste sfenički, dok $60 = 2^2 \cdot 3 \cdot 5$ nije sfenički.
5. Za broj se kaže da je slobodan od kvadrata ako ga nijedan prosti faktor ne dijeli više od jednom, tj. najveći stepen prostog faktora koji

dijeli n je jedan. Prvih nekoliko slobodnih kvadrata brojeva su

$$1, 2, 3, 5, 6, 7, 10, 11, 13, 14, 15, 17, 19, 21.$$

Na ulazu je dat broj n . Ispitati da li je on broj slobodnog kvadrata.

6. Neka je dat broj n . Ispitati da li je to *lažni* broj ili ne. Pod lažnim brojem podrazumijevamo složeni broj, čiji je zbir cifara jednak zbiru cifara njegovih različitih prostih faktora.
7. Dat je skup od n elemenata, pronaći broj načina particionisanja broja (Belovi brojevi). Implementirati efikasnu proceduru računanja ovakvih brojeva u zavisnosti od n .

Bibliografija

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, Cambridge, MA, 2022.
- [2] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed., Springer, Cham, 2020.
- [3] J. Kleinberg and E. Tardos, *Algorithm Design*, Pearson/Addison-Wesley, Boston, 2006.
- [4] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, Boston, 2012.
- [5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley Professional, Boston, 2011.
- [6] J. Erickson, *Algorithms*, 1st ed., 2019. Available online at: <https://jeffe.cs.illinois.edu/teaching/algorithms/>.
- [7] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, *Algorithms*, McGraw-Hill, New York, 2008.
- [8] E. Horowitz, S. Sahni, and S. Anderson-Freed, *Fundamentals of Data Structures in C*, W. H. Freeman and Company, New York, 1992.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed., Addison-Wesley, Reading, MA, 1997.

- [10] S. Baase and A. Van Gelder, *Computer Algorithms: Introduction to Design and Analysis*, 3rd ed., Addison-Wesley, Boston, 2000.
- [11] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*, Wiley, Hoboken, NJ, 2013.
- [12] P. Brass, *Advanced Data Structures*, Cambridge University Press, Cambridge, 2008.
- [13] U. Manber, *Introduction to Algorithms: A Creative Approach*, Addison-Wesley, Reading, MA, 1989.
- [14] Python Software Foundation, *Python Documentation*, Python Software Foundation. Available online at: <https://docs.python.org/3/>.
- [15] Python Software Foundation, *The Python Language Reference*. Available online at: <https://docs.python.org/3/reference/>.
- [16] Python Software Foundation, *The Python Standard Library*. Available online at: <https://docs.python.org/3/library/>.
- [17] M. Lutz, *Learning Python*, 6th ed., O'Reilly Media, Sebastopol, CA.
- [18] L. Ramalho, *Fluent Python: Clear, Concise, and Effective Programming*, 2nd ed., O'Reilly Media, Sebastopol, CA, 2022.
- [19] D. Beazley and B. K. Jones, *Python Cookbook*, 3rd ed., O'Reilly Media, Sebastopol, CA, 2013.
- [20] R. Sedgewick, K. Wayne, and R. Dondero, *Introduction to Programming in Python: An Interdisciplinary Approach*, Addison-Wesley Professional, 2015.
- [21] Z. Ognjanović and N. Krdžavac, *Uvod u teorijsko računarstvo*, Fakultet organizacionih nauka, Univerzitet u Beogradu, Beograd, 2004.
- [22] D. Živković, *Uvod u algoritme i strukture podataka*, Univerzitet Singidunum, Beograd, 2010.

- [23] M. Živković, *Algoritmi*, Matematički fakultet, Univerzitet u Beogradu, Beograd, 2000.
- [24] R. Manger and M. Marušić, *Strukture podataka i algoritmi*, Prirodoslovno-matematički fakultet, Sveučilište u Zagrebu, Zagreb.
- [25] Python Software Foundation, *Python Package Index (PyPI)*. Available online at: <https://pypi.org/>.
- [26] GeeksforGeeks, *Data Structures and Algorithms*. Available online at: <https://www.geeksforgeeks.org/>.
- [27] A. Neumann, *Simple Programming Problems*. Available online at: https://adriann.github.io/programming_problems.html.